

基于遗传算法的无人机投弹策略优化

摘要

为应对来袭导弹对固定圆柱体目标的打击，本文围绕“无人机投放球状烟雾”的软杀伤方案，提出一套基于遗传算法并辅以贪心算法优化，采用从几何评估到协同优化的一体化方法。本文构建了导弹匀速，无人机等高匀速直线运动，干扰弹平抛运动，球状烟雾下沉的运动学模型。并且构建了基于圆柱体目标表面采样的“严格全覆盖”判据，形成通用评估内核；在此基础上，还结合遗传算法的“粗筛+细筛”，使用贪心算法分解问题，采用上界剪枝与结果缓存等工程化策略，进行系统求解。

问题一：统一构建“导弹——无人机——干扰弹”运动学模型，给出导弹所在点到圆柱体上每一点实现均被烟雾球遮挡的严格遮蔽判据；在统一时间窗内返回遮蔽区间并计时，得到单弹确定参数的情况下的遮蔽时间为 **1.39s**，由此确定后续各题共用的评估内核与输出格式。

问题二：采用遗传算法 **全局搜索**，粗评用较大步长与快速充分判据预筛，精评以小步长按严格口径重估，并通过可行化修复满足起爆时限与最短间隔，找到最优的航向，速度，投放时刻与起爆延迟，得到单机单弹最长遮蔽时间为 **4.58s**。

问题三：在问题二的框架内优化三次投放时序与延迟，以遮蔽区间并集计分（重叠不重复计时）；编码采用差分间隔以保证最短投放间隔，评估仍调用问题 1 的严格判据。得到最长遮蔽时间为 **6.40s**，验证“拼窗延时”策略的有效性与稳健性。

问题四：本题将问题扩展至三架无人机，因此本文先以贪心策略分别求出三机单机最优并筛为高价值种子，再注入遗传算法做协同细化；总体指标为三机遮蔽区间的并集长度。最终得到三机并集遮蔽时间为 **11.38s**。

问题五：本题扩展五机三干扰弹三导弹，极大增加了复杂度，本文采用贪心策略将问题分解，对每枚导弹分别计算各机三弹的并集时长，再对三枚导弹求和作为每机得分，最后汇总五机总分；为加速评估与迭代，引入上界剪枝与结果缓存并保留严格口径终评。最终获得总有效遮蔽 **22.27s**。

关键字： 优化问题 遗传算法 智能优化算法 贪心算法

一、问题重述

1.1 问题背景

现代战争中，精确制导武器对高价值固定目标的打击能力日益增强，传统硬杀伤拦截手段面临成本高、响应慢、拦截窗口短等现实瓶颈。烟幕干扰作为一种典型的软杀伤防护手段，通过在短时间内形成高浓度气溶胶云团，对敌方光学、红外、激光等制导系统实施遮蔽与误导，具有成本低、部署快、效费比高等显著优势，已成为要地防空、目标防护的重要补充手段。

近年来，随着无人机技术的迅猛发展，利用具有长续航能力的无人机挂载烟幕干扰弹实施前置式、精准式干扰成为研究热点。相较于传统地面固定发射或有人机临空抛撒，无人机具备任务灵活性强、部署空域广、响应速度快等优势，可在导弹来袭航路前方预设干扰走廊，实现“以逸待劳”式遮蔽，显著延长有效干扰时间窗口。

然而，无人机烟幕干扰弹的实战效能受多重因素耦合影响：一方面，导弹飞行速度快、航路预测误差小，要求干扰云团必须在时空维度上精准遮蔽其制导视野；另一方面，烟幕云团以恒定速度下沉、有效遮蔽半径有限、持续时间短暂，需通过多弹协同、起爆时序优化、航路动态规划等手段，实现对真目标的有效遮蔽时长的最大化。此外，无人机在受领任务后可瞬时调整航向与速度，但一旦确定即不再变更，这使得投放点、起爆点、航路参数必须在任务初始阶段一体化优化，构成了典型的多约束、多阶段、多目标决策问题。

在此背景下，如何利用有限的无人机与干扰弹资源，针对多枚来袭导弹，设计时空协同的投放策略，使烟幕云团在真目标周围形成持续、稳定、高效的遮蔽屏障，成为提升要地生存概率的核心科学问题。本研究围绕该问题，构建**多弹协同遮蔽模型**，融合遗传算法与贪心策略，系统优化无人机飞行参数、干扰弹投放时序与起爆位置，为无人机软杀伤作战提供量化决策支持与工程可实现方案。

1.2 问题提出

问题 1：在无人机 FY1 飞行速度、飞行方向、烟幕干扰弹投放点和烟幕干扰弹起爆点均固定的情况下，如何计算烟幕干扰弹所形成的球形烟幕云团对导弹进行遮蔽的有效遮蔽时间。

问题 2：在无人机 FY1 的飞行速度、飞行方向、烟幕干扰弹投放点和烟幕干扰弹起爆点均为自变量的情况下，如何合理选择飞行速度、飞行方向、烟幕干扰弹投放点和烟幕干扰弹起爆点的数值，使得遮蔽时间最长。

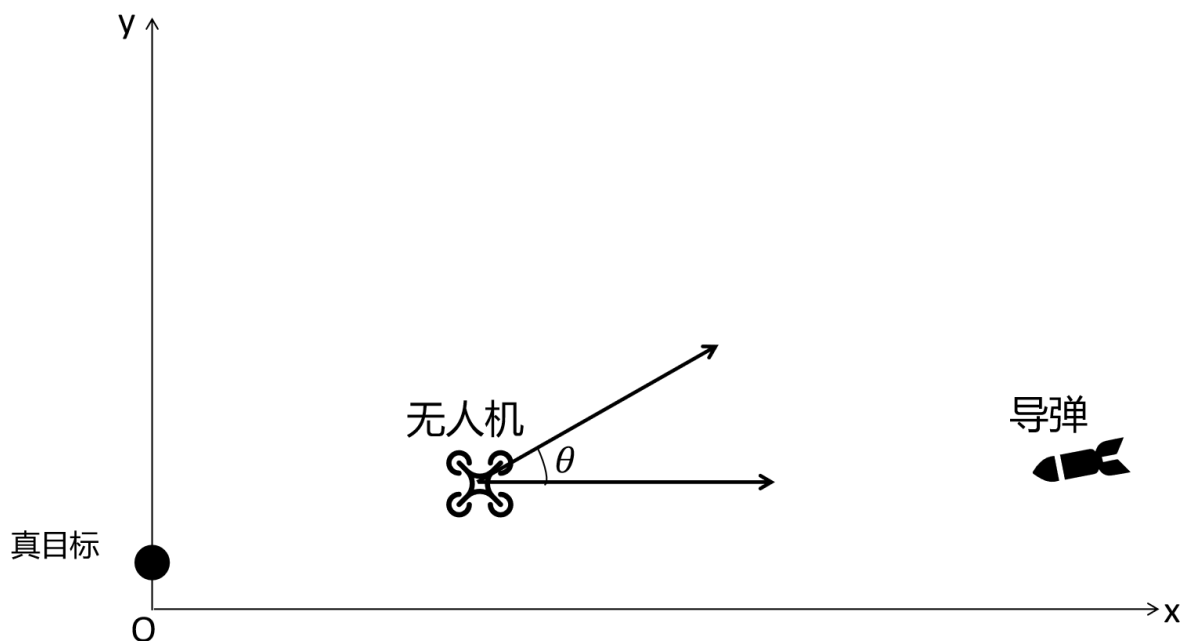


图 1 航向角说明

问题 3: 在无人机 FY1 的飞行速度、飞行方向、三颗烟幕干扰弹分别的投放点和烟幕干扰弹起爆点均为自变量的情况下，如何合理选择飞行速度、飞行方向、三颗烟幕干扰弹分别的投放点和烟幕干扰弹起爆点的数值，使得三颗烟幕干扰弹的遮蔽时间的并集最大。并且有限制条件：每架无人机投放两枚烟幕干扰弹至少间隔 1 s。

问题 4: 在无人机 FY1、FY2、FY3 的飞行速度、飞行方向、各自的烟幕干扰弹投放点和烟幕干扰弹起爆点均为自变量的情况下，如何合理选择飞行速度、飞行方向、各自的烟幕干扰弹投放点和烟幕干扰弹起爆点的数值，使得三颗烟幕干扰弹的遮蔽时间的并集最大。由于本题是三架无人机各自只携带一枚烟幕干扰弹，故没有投弹时间间隔的限制，且这三架无人机有各自的飞行轨迹，故对于投弹的位置选择更为灵活。

问题 5: 利用 5 架无人机，他们的飞行速度、飞行方向、各自的三枚烟幕干扰弹投放点和烟幕干扰弹起爆点均为自变量，如何合理选择飞行速度、飞行方向、各自的三枚烟幕干扰弹投放点和烟幕干扰弹起爆点的数值，使得对于 M1、M2、M3 三枚导弹的有效遮蔽时间最长。由于有三枚导弹，故有效遮蔽要求对这三枚导弹都要进行遮蔽。

二、问题分析

2.1 问题一分析

问题 1 要求在给定无人机 FY1 飞行速度、飞行方向、烟幕干扰弹投放点和烟幕干扰弹起爆点的条件下，计算单枚烟幕干扰弹对导弹 M1 的有效遮蔽时间。核心思路是建立导弹匀速直线飞行、干扰弹平抛运动与延时起爆、烟幕云团匀速下沉的运动模型，并

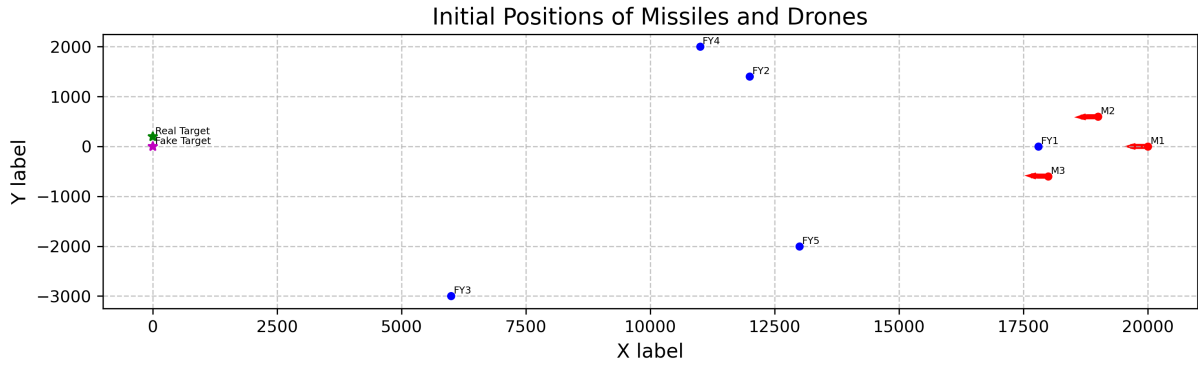


图 2 无人机和导弹初始位置示意图

结合遮蔽判据判断导弹到真目标这个圆柱上的任意一点的视线是否被烟雾云团遮挡。通过在有效时间窗内统计视线被遮挡的时段，即可得到总遮蔽时长。

2.2 问题二分析

问题 2 要求在单枚烟幕干扰弹的条件下，通过优化无人机 FY1 的飞行速度、飞行方向、烟幕干扰弹投放点和烟幕干扰弹起爆点，使得烟幕对导弹 M1 的有效遮蔽时间尽可能长。与问题 1 的“计算给定数据”不同，这里需要建立一个优化问题：决策变量为

$$x = [v, \theta, t_{\text{drop}}, \text{delay}],$$

目标函数为对应条件下的有效遮蔽时长。

由于变量连续，有复杂约束，且遮蔽判据高度非线性，难以用解析方法直接求解，因此我们的代码中采用了 **遗传算法 (Genetic Algorithm, GA)**。其基本流程为：随机初始化种群，在交叉、变异等操作下迭代演化，通过适应度函数评估个体遮蔽时长，并逐代逼近最优方案。这样能在多变量搜索空间中找到近似最优的无人机投放策略。

2.3 问题三分析

问题三在问题二的基础上，进一步扩展为单架无人机投放三枚烟幕干扰弹，目的是通过合理设计三次投放与起爆时序，使得总的有效遮蔽时间最长。与单弹优化不同，这里不仅需要确定无人机的飞行方向，飞行速度，还要设计三枚烟幕干扰弹的投放时间、间隔时间和起爆时间，以实现三枚烟幕干扰弹遮蔽时间并集的最大化。

由于多个干扰弹的效果具有时间叠加和并集特性，问题三实际上是一个多变量的全局优化问题。笔者采用遗传算法求解：以染色体表示航向角、飞行速度以及三次投放与起爆的参数，适应度函数为三团云雾遮蔽区间的并集时长。为提高效率，算法先用快速几何判据进行可行性筛选，再用严格判据进行高精度评估。这样既能保证搜索速度，又能在复杂的参数空间中逼近最优的多弹投放策略。

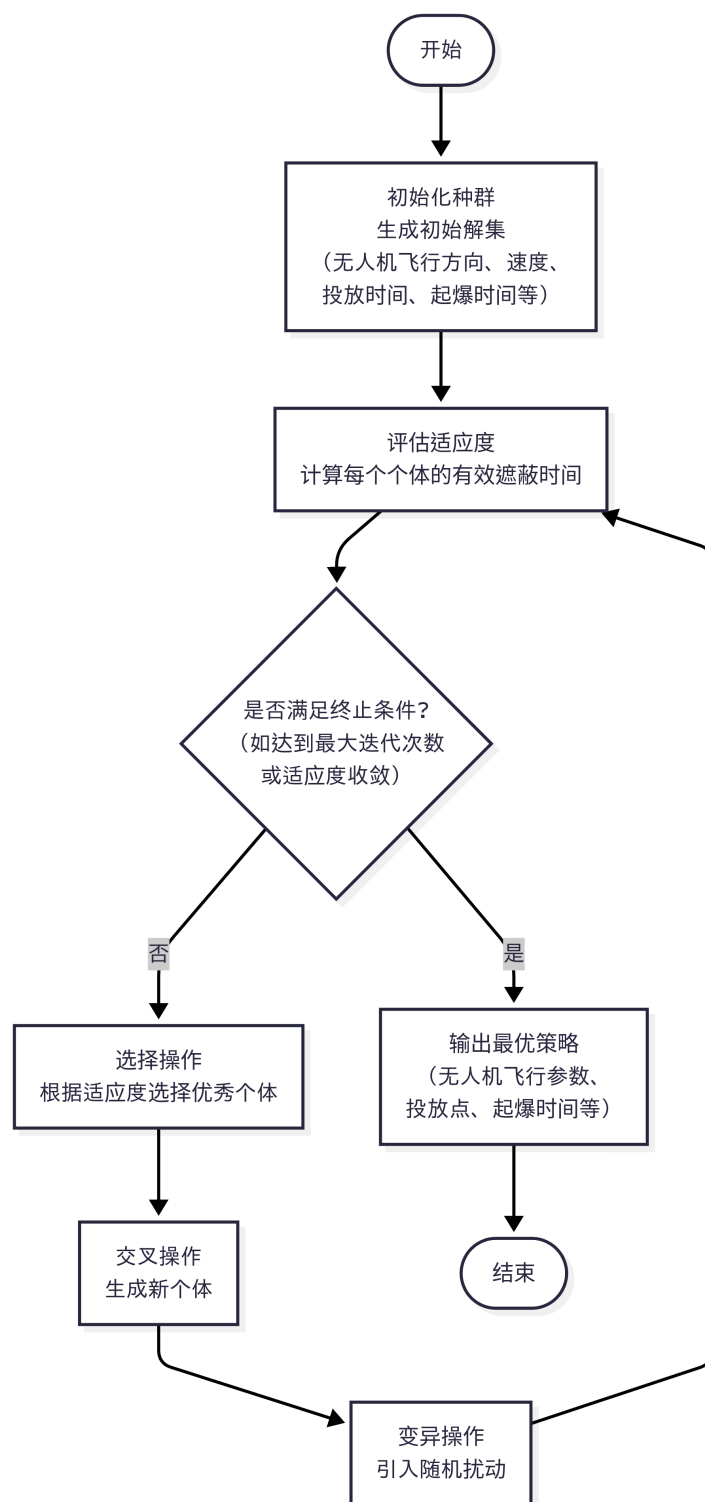


图 3 遗传算法流程图

2.4 问题四分析

问题 4 在前面单机场景的基础上，引入了多无人机协同：共有三架无人机 (FY1、FY2、FY3)，每架各投放一枚烟幕干扰弹，对导弹 M1 进行联合干扰。与问题 2、问题 3 的单机优化不同，这里每架无人机都有独立的初始位置和可调参数，需要分别确定飞

行方向、飞行速度、投放时刻和延时起爆。

建模思路是将问题分解为三个独立的 **单机优化子问题**：先用遗传算法分别为 FY1、FY2、FY3 求解单弹最优投放方案，再将结果汇总，输出三机的运动参数，投放点与起爆点坐标以及对应的有效干扰时长。最终结果上三枚烟幕干扰弹的遮蔽时段刚好错开，三个最优解之和一定为全局最优解。这种“分而治之”的方法避免了直接在高维搜索空间中进行全局优化，降低了计算复杂度，同时能保证每架无人机都尽可能发挥最大作用，最终的遮蔽时间由三者的贡献叠加而成。

2.5 问题五分析

问题五在问题三和问题四的基础上，要求使用五架无人机 (FY1、FY2、FY3、FY4、FY5) 各自配备三枚烟幕干扰弹来联合干扰三枚导弹 (M1、M2、M3)。不同于前面的问题，问题五的目标函数不能由一个导弹的有效遮蔽时间简单确定，面对多枚导弹我们需要考虑各个导弹的威胁程度，哪个无人机适合用来投放烟幕干扰弹阻碍哪个导弹，综合这些因素来对投弹策略进行优化，来最大化三枚导弹有效遮蔽时间并集的长度。

基于遗传算法，问题五中的基因变量包含了五架无人机的航向、速度、三枚烟幕干扰弹的投放时间、引爆时间。过多的基因变量会使遗传算法的计算量陡增、计算时间过长和计算准确度下降，因此我们首先利用贪心算法将问题初步分解为五个独立的单机遮蔽三弹的优化子问题，再将结果汇总处理，不断迭代，使三枚导弹的有效遮蔽时间并集尽可能长。

三、模型假设与符号说明

3.1 模型假设

1 导弹运动轨迹假设：导弹朝假目标以定速度 300m/s 定飞行方向飞行，导弹的飞行轨迹是一条定直线，可以精准地用参数方程表示。

2 无人机运动轨迹假设：无人机在指定位置接受任务后可以立即起飞，起飞时确定飞行方向，与飞行速度（速度范围为 70-140m/s），且只能登高飞行，飞行方向与速度一经确定便不再改变。无人机的飞行轨迹是平行于 xy 平面的定直线，可以精准地用参数方程表示。

3 烟幕干扰弹投弹运动假设：烟幕干扰弹从无人机上投放后做平抛运动，水平方向上继承无人机的飞行速度，竖直方向上仅受重力作用，忽略空气阻力。

4 球状烟雾云团假设：假设烟雾干扰弹起爆后瞬间产生半径为 10m 的球状烟雾云团，云团一经形成便立即有遮蔽效果。球状烟雾云团形成后便以 3m/s 匀速下沉，下沉过程中形状不变仍然保持球状，且半径不变，在整个球状烟雾范围内一直有遮蔽效果，直到起爆后 20s 后完全消散，不再具有任何遮蔽效果。

5 有效遮蔽的定义：在问题 1 到问题 5 中，用极小的步长遍历圆柱体的侧面与上顶面，并且对所有点实施是否有效遮蔽检测，即验证是否导弹与圆柱体上每一个点的连线，均与球状烟雾相交。只有满足所有点均被遮蔽才认为真目标被遮蔽。

6 真实目标假设：根据题目描述，真实目标是一个底面半径为 7m，高为 10m 的圆柱体。

7 投弹时间间隔假设：根据题目描述，同一架无人机投放的每两枚烟雾干扰弹的投弹时间间隔需大于等于 1s，且每架无人机都可以精准控制其投弹时间。

8 起爆时间假设：假设起爆时间一到，烟雾干扰弹便可以立即爆炸产生球状烟雾，达到遮蔽效果，起爆时间不得晚于烟雾干扰弹平抛运动落地时刻。

9 遮蔽效果假设：假设多枚烟雾干扰弹产生的球状烟雾可以配合实现对一枚或多枚导弹遮蔽视线的效果，一枚干扰弹产生的球状烟雾也可以对多枚导弹产生遮蔽作用。

10 计算时间假设：假设无人机出发前确定飞行方向、飞行速度，烟幕干扰弹投放点和烟幕干扰弹起爆点的计算思考时间无限快，即认为经过思考时间后，导弹的各个参数以及位置不变，排除因计算时间导致的导弹位置的变化，避免计划跟不上变化。

3.2 符号说明

表 1 符号说明

符号	说明	单位
g	重力加速度	m/s^2
v_{missile}	导弹的速度	m/s
v_{cloud}	云团下沉速度	m/s
v_{drone}	无人机的速度	m/s
v_{min}	无人机飞行最低速度	m/s
v_{max}	无人机飞行最高速度	m/s
$R_{\text{effective}}$	有效遮蔽半径	m
$T_{\text{effective}}$	有效遮蔽时间窗长度	s
interval	最短投弹时间间隔	s
T_{end}	导弹到达假目标时刻	s
t_{drop}	无人机投弹时间	s
delay	干扰弹从投放到爆炸的延时	s
t_{burst}	干扰弹爆炸时刻	s
P_{burst}	干扰弹爆炸位置	m
M_i^0	导弹 i 在初始时刻的位置	m

接下页

续表 1

符号	说明	单位
FY_i^0	无人机 i 在初始时刻的位置	m
M_i^t	导弹 i 在 t 时刻的位置	m
FY_i^t	无人机 i 在 t 时刻的位置	m
fake	假目标的位置	m
real	真目标圆柱底面中心	m
R	真目标圆柱的半径	m
H	真目标圆柱的高度	m
PopSize	遗传算法设定的种群大小	
nGen	遗传算法设定的迭代代数	
θ	FY1 的水平航向角（弧度）	rad
$[s_j, e_j]$	第 j 个遮蔽时间区间的起止时刻	s
B	真目标可见表面上的采样点	m
α	退火因子	
$t_{\text{rel}}^{(k)}$	第 k 枚干扰弹的投放时刻	s
$\text{delay}^{(k)}$	第 k 枚干扰弹的延时	s
$t_{\text{burst}}^{(k)}$	第 k 枚干扰弹的起爆时刻	s
$P_{\text{burst}}^{(k)}$	第 k 枚干扰弹的起爆位置	m
$[s_j^{(k)}, e_j^{(k)}]$	第 k 枚弹的遮蔽时间区间	s
$\mathcal{U}$	三弹遮蔽区间的并集	
$C(t)$	云团中心在 t 时刻的位置	m
$\text{CoverTime}^{(i)}$	第 i 架无人机的有效遮蔽总时长	s
$\mathcal{U}_{\text{all}}$	三机（FY1-FY3）遮蔽区间的并集	
$\text{CoverTime}_{\text{all}}$	三机联合有效遮蔽总时长（并集的测度）	s
$T_{\cap}$	评估时间上限（由 T_{end} 与导弹到达时刻综合裁剪所得）	s
$\mathcal{U}_m^{(i)}$	第 i 架无人机对第 m 枚导弹的遮蔽区间并集	

注：凡带上标 (i) 表示第 i 架无人机相关量（ $i = 1, 2, 3$ ）；凡带上标 (k) 表示第 k 枚干扰弹相关量（ $k = 1, 2, 3$ ）。

四、模型的建立与求解

4.1 问题一模型的建立与求解

4.1.1 模型的建立

导弹从 M_1^0 进入警戒范围，飞行速度为 v_{missile} ，方向为“指向 fake 的单位方向”，导弹一直匀速直线飞行：

$$M_1^t = M_1^0 + v_{\text{missile}} t \frac{\text{fake} - M_1^0}{\|\text{fake} - M_1^0\|}. \quad (1)$$

无人机 FY1 等高匀速直线飞行：

$$FY_1^t = FY_1^0 + v_{\text{drone}} t \frac{\text{fake} - FY_1^0}{\|\text{fake} - FY_1^0\|}, \quad (2)$$

并在 t_{drop} 投放烟雾干扰弹，经过延时 delay 起爆，有 $t_{\text{burst}} = t_{\text{drop}} + \text{delay}$ 。干扰弹在起爆延期内做平抛运动，水平方向速度继承了无人机的速度，竖直方向上只受重力作用，忽略空气阻力。起爆位置

$$P_{\text{burst}} = FY_1^{t_{\text{drop}}} + v_{\text{drone}} \text{delay} \frac{\text{fake} - FY_1^0}{\|\text{fake} - FY_1^0\|} + (0, 0, -\frac{1}{2}g \text{delay}^2). \quad (3)$$

云团以 v_{cloud} 竖直匀速直线下沉，令

$$(x_c(t), y_c(t), z_c(t)) = (P_{\text{burst},x}, P_{\text{burst},y}, P_{\text{burst},z} - v_{\text{cloud}}(t - t_{\text{burst}})), \quad t \geq t_{\text{burst}}. \quad (4)$$

评估时间窗取

$$t \in [t_{\text{burst}}, \min(t_{\text{burst}} + T_{\text{effective}}, T_{\text{end}})]. \quad (5)$$

遮蔽判据要求严格全覆盖：将真目标（底面中心 $real$ 、半径 R 、高度 H 的圆柱体）可见表面离散为采样点集。若在时刻 t 对任意采样点 B 均有

$$\min_{s \in [0,1]} \|s M_1^t + (1-s) B - (x_c(t), y_c(t), z_c(t))\| \leq R_{\text{effective}}, \quad (6)$$

即表示导弹到圆柱体表面上任意一点 B 的直线均与球状烟雾相交，则判定真目标在该时刻被完全遮蔽。

但是这个判据的缺点是计算量较大。为加速计算，可先用快速判据预筛。

快速判据的几何解释为：设导弹和云团中心的连线与和云团的切线的夹角为 β ，导弹和圆柱体中心的连线与和圆柱体外接球的切线的夹角为 α ，导弹和云团中心的连线与导弹和圆柱体中心的连线的夹角为 θ 。若 $\beta \geq \alpha + \theta$ ，则导弹到圆柱体上任意一点的连线均与云团相交，真目标被遮蔽。

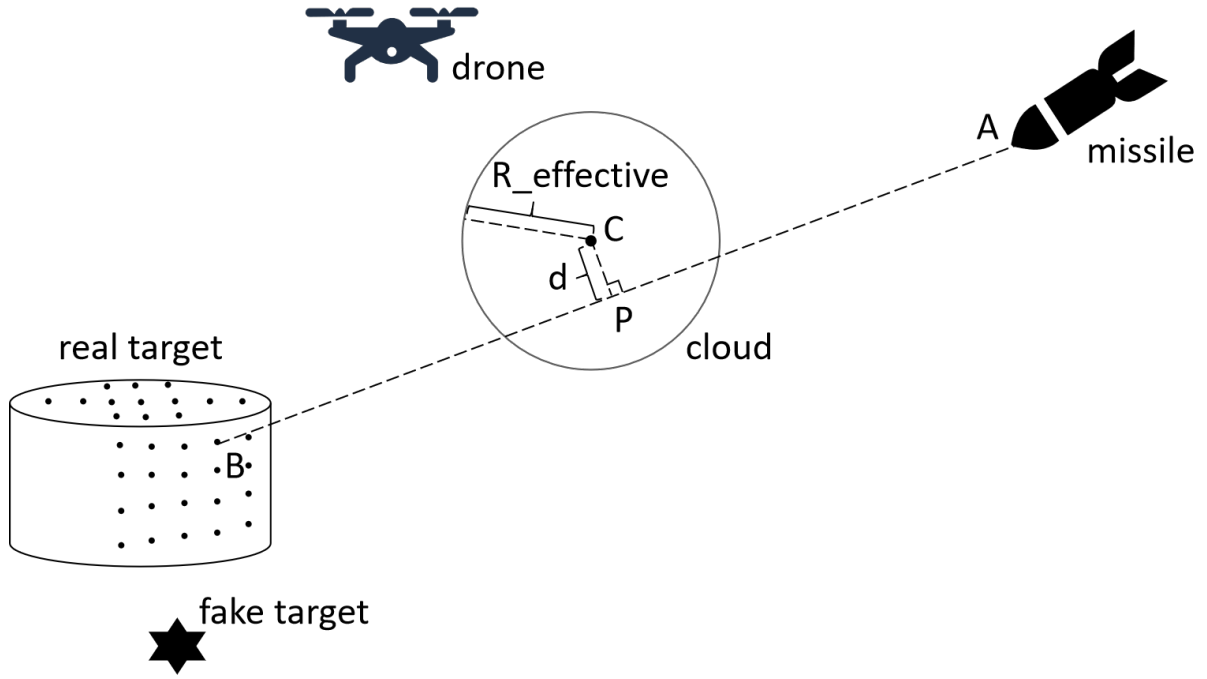


图 4 精确判据推导

但是这个判据其实是一个充分不必要条件，就是说，当不满足 $\beta \geq \alpha + \theta$ 时，真目标也可能被遮蔽，因此这个判据只能用来预筛，不能作为最终的判据。

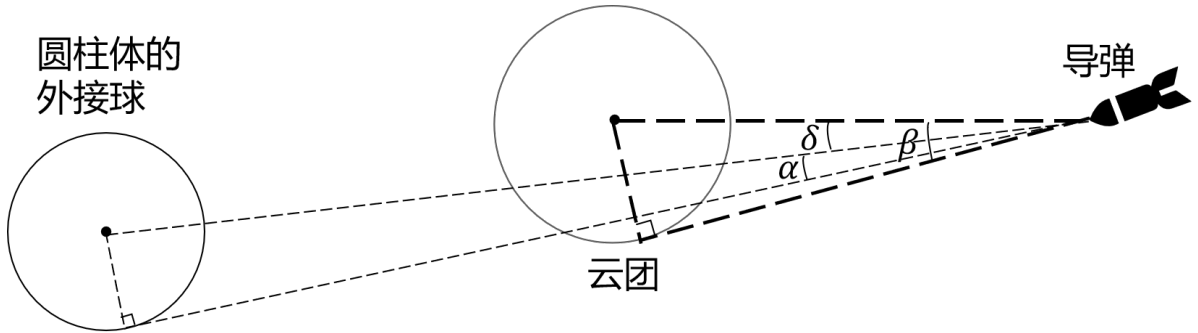


图 5 快速判据推导

4.1.2 模型的求解

- (1) 由 v_{drone} , t_{drop} , delay 求得 t_{burst} 与 P_{burst} ;
- (2) 在 $[t_{\text{burst}}, \min(t_{\text{burst}} + T_{\text{effective}}, T_{\text{end}})]$ 进行固定步长离散;
- (3) 逐时刻计算 M_1^t 与 $(x_c(t), y_c(t), z_c(t))$, 按遮蔽判据检查所有采样点是否均满足 $\leq R_{\text{effective}}$, 满足则记为被遮蔽;

(4) 合并相邻“被遮蔽”时刻为若干区间 $[s_j, e_j]$ ，总有效遮蔽时长为

$$\sum_j (e_j - s_j); \quad (7)$$

(5) 输出 P_{burst} 、各区间 $[s_j, e_j]$ 与总时长.

(6) 最终输出为

Burst point (m): x=17188.000, y=0.000, z=1736.431

[Total] dur=1.390

intervals=[(8.049999999999937, 9.439999999999909)]

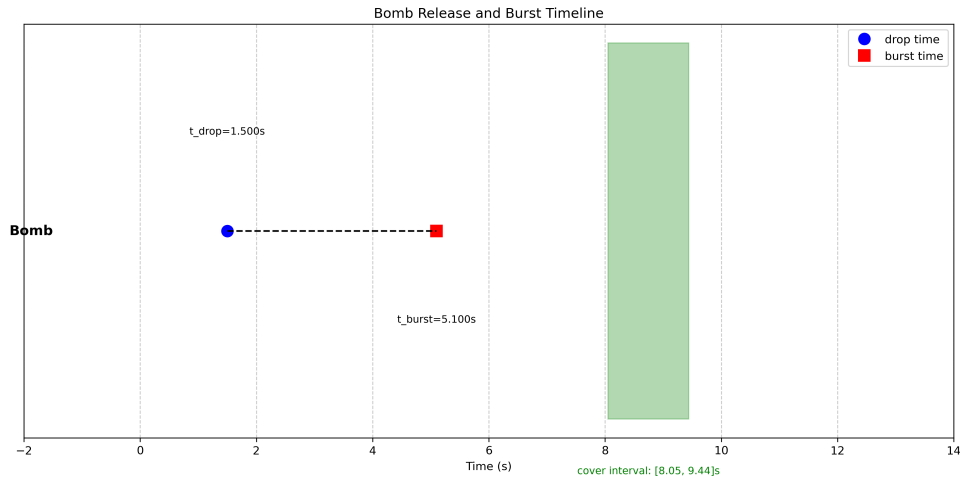


图 6 问题 1 投弹时间线

4.2 问题二模型的建立与求解

4.2.1 模型的建立

在单枚干扰弹的设定下，需要寻找最佳的无人机 FY1 的水平航向角 θ 、飞行速度 v_{drone} 、投放时刻 t_{drop} 与延时起爆 delay ，并令

$$t_{\text{burst}} = t_{\text{drop}} + \text{delay}.$$

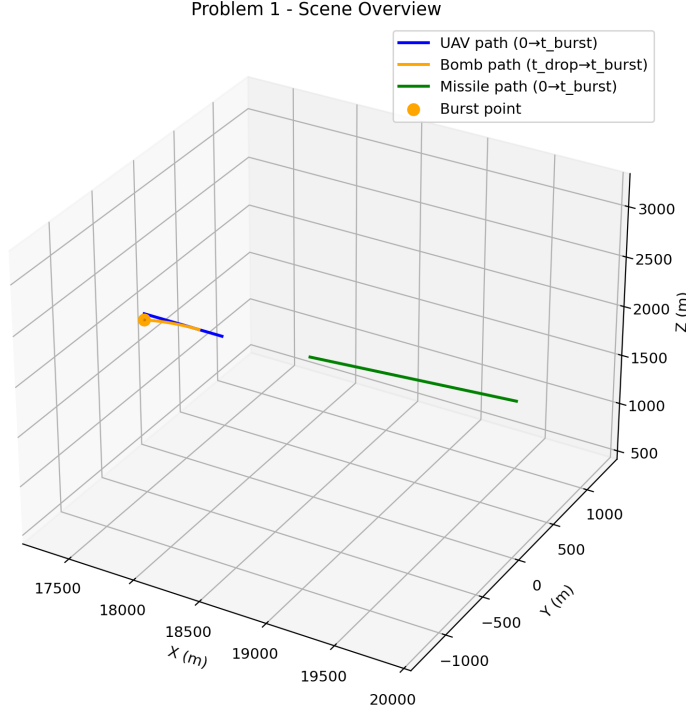


图 7 问题 1 的三维场景图

约束条件为

$$v_{\min} \leq v_{\text{drone}} \leq v_{\max}, \quad (8)$$

$$-\pi \leq \theta \leq \pi, \quad (9)$$

$$0 \leq t_{\text{drop}} \leq 30 \text{ s}, \quad (10)$$

$$0 \leq \text{delay} \leq T_{\text{effective}}, \quad (11)$$

$$t_{\text{burst}} \leq T_{\text{end}}. \quad (12)$$

对违反 $t_{\text{burst}} \leq T_{\text{end}}$ 的情形，进行可行化修复：

$$\text{delay} \leftarrow \max(0, T_{\text{end}} - t_{\text{drop}}), \quad t_{\text{burst}} = t_{\text{drop}} + \text{delay}.$$

运动学关系：

导弹从 M_1^0 出发，以速度 v_{missile} 指向 fake 匀速直线飞行：

$$M_1^t = M_1^0 + v_{\text{missile}} t \frac{\text{fake} - M_1^0}{\|\text{fake} - M_1^0\|}. \quad (13)$$

FY1 等高匀速直线飞行，其水平航向为 θ ，干扰弹在 t_{drop} 时刻投放，在起爆延时

delay 后于 t_{burst} 起爆。起爆位置为

$$P_{\text{burst}} = FY_1^{t_{\text{drop}}} + v_{\text{drone}} \cdot \text{delay} \cdot (\cos \theta, \sin \theta, 0) + (0, 0, -\frac{1}{2}g \text{delay}^2). \quad (14)$$

起爆后云团以 v_{cloud} 竖直匀速下沉：

$$(x_c(t), y_c(t), z_c(t)) = (P_{\text{burst},x}, P_{\text{burst},y}, P_{\text{burst},z} - v_{\text{cloud}}(t - t_{\text{burst}})), \quad t \geq t_{\text{burst}}, \quad (15)$$

并在时间窗

$$t \in [t_{\text{burst}}, \min(t_{\text{burst}} + T_{\text{effective}}, T_{\text{end}})] \quad (16)$$

内进行评估。

遮蔽判据（严格全覆盖） 将真目标（底面中心 $real$ 、半径 R 、高度 H 的圆柱体）可见表面进行离散采样。若在某一时刻 t ，对每个采样点 B 都有

$$\min_{s \in [0,1]} \|s M_1^t + (1-s) B - (x_c(t), y_c(t), z_c(t))\| \leq R_{\text{effective}}, \quad (17)$$

则判定在 t 时刻被完全遮蔽。对整个时间窗进行逐时刻判定，将收集所有“为真”的时间片段取并集。

4.2.2 模型的求解

以总有效遮蔽时长为目标，设所有被遮蔽的时间片段合并为区间 $\{[s_j, e_j]\}$ ，则本题等价于在满足边界条件与可行化约束的条件下，进行求解

$$\max_{v_{\text{drone}}, \theta, t_{\text{drop}}, \text{delay}} \sum_j (e_j - s_j) \quad (18)$$

采用实数编码的遗传算法求解：个体为 $[v_{\text{drone}}, \theta, t_{\text{drop}}, \text{delay}]$ ，下界与上界分别为 $[v_{\text{min}}, -\pi, 0, 0]$ 与 $[v_{\text{max}}, \pi, 30, T_{\text{effective}}]$ 。解码时计算 t_{burst} 并执行可行化修复，然后调用问题一的评估内核得到目标值（严格遮蔽判据）。为兼顾效率与精度，采用“两阶段评估”：先以较大的步长进行粗评并预筛个体，再以较小的步长对候选与精英进行严格复评。进化操作包括锦标赛选择、SBX 交叉（ $\eta = 12.0$ 、交叉概率 0.9）与高斯扰动变异；对基因向量 $[v_{\text{drone}}, \theta, t_{\text{drop}}, \text{delay}]$ 的扰动尺度取

$$[0.05(v_{\text{max}} - v_{\text{min}}), 0.06\pi, 0.6\alpha, 0.5\alpha],$$

其中 α 为按代数线性递减（不低于 0.45）的退火因子。达到迭代上限或收敛阈值后，输出最优 $(v_{\text{drone}}, \theta, t_{\text{drop}}, \text{delay}, t_{\text{burst}})$ ，并给出对应的 P_{burst} 、遮蔽区间 $\{[s_j, e_j]\}$ 及其

总长度用于展示与复现。

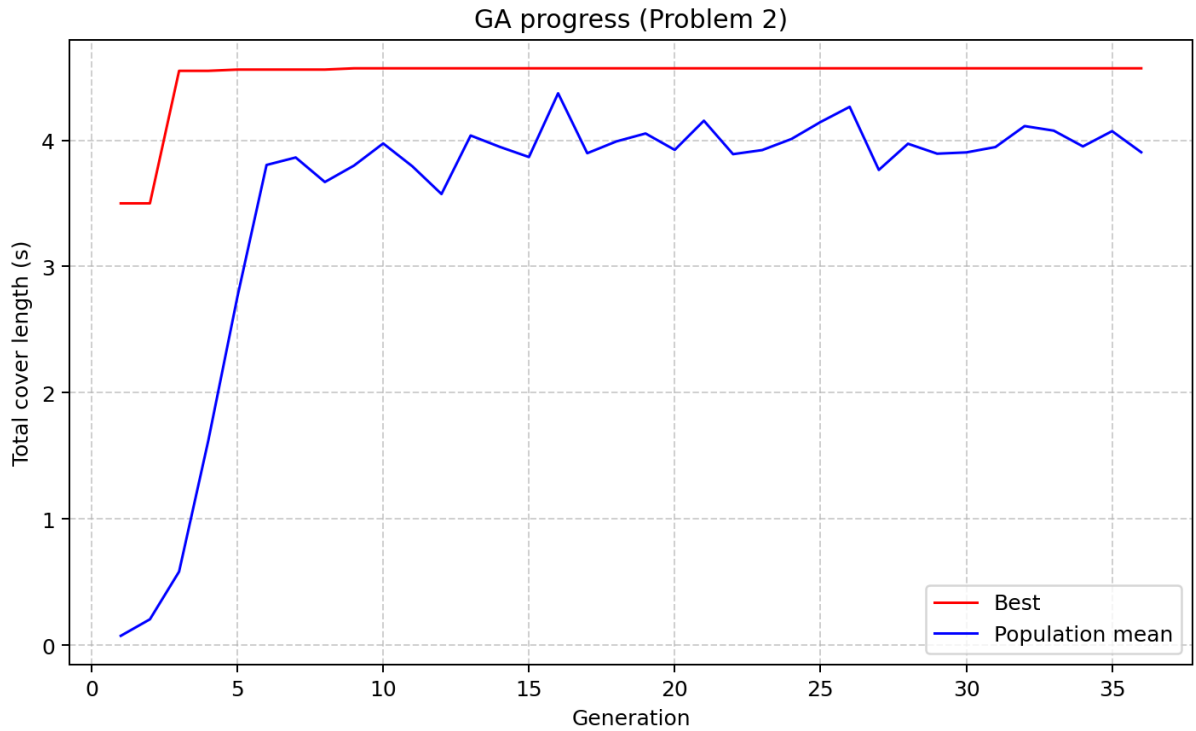


图 8 问题 2 的遗传算法进展图

最终结果为：

=== Problem 2 —GA Best Plan (final, total-length objective) ===

Total cover length: 4.570 s

Speed $v = 74.623$ m/s, Heading $\theta = 6.371^\circ$

Drop time: 0.461 s,

Delay: 0.793 s,

Burst: 1.254 s

Burst pos: (17892.987, 10.382, 1796.919) m

Effective cover intervals:

Interval 1: 1.540 s to 6.110 s (duration: 4.570 s)

4.3 问题三模型的建立与求解

4.3.1 模型的建立

问题 3 在只有一架无人机 FY1 投放三枚干扰弹的条件下，需全局搜索最优的水平航向角 θ 、飞行速度 v_{drone} ，以及三枚烟雾干扰弹分别的投弹时间与起爆延时 $\{t_{\text{rel}}^{(k)}, \text{delay}^{(k)}\}_{k=1}^3$ ，其中

$$t_{\text{burst}}^{(k)} = t_{\text{rel}}^{(k)} + \text{delay}^{(k)}.$$

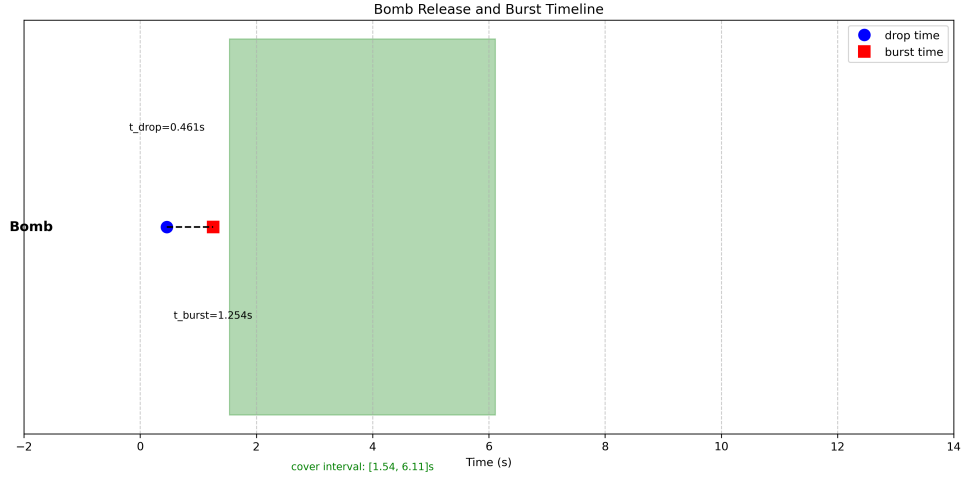


图 9 问题 2 投弹时间线

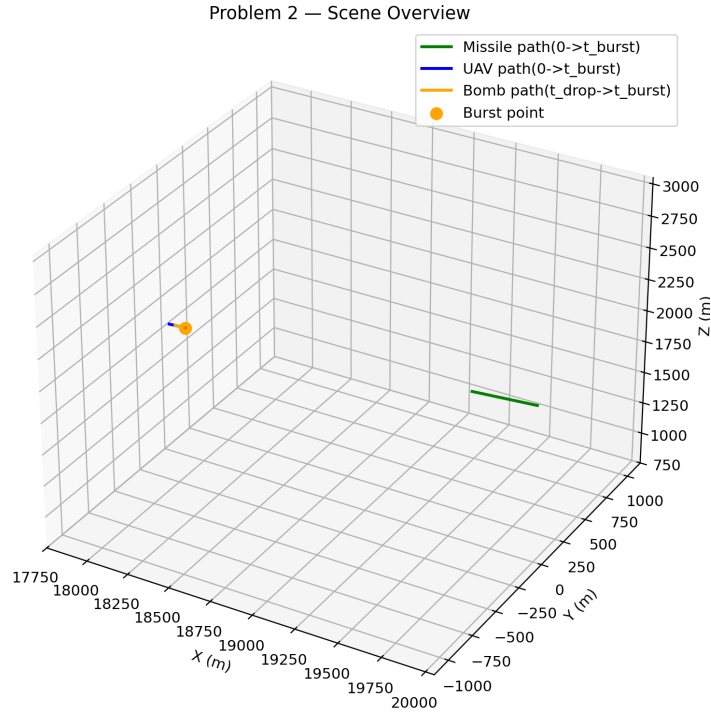


图 10 问题 2 的三维场景图

决策还需要满足下列物理与边界约束：

$$v_{\min} \leq v_{\text{drone}} \leq v_{\max}, \quad (19)$$

$$-\pi \leq \theta \leq \pi, \quad (20)$$

$$t_{\text{rel}}^{(1)} \geq 0, \quad (21)$$

$$t_{\text{rel}}^{(2)} - t_{\text{rel}}^{(1)} \geq \text{interval}, \quad (22)$$

$$t_{\text{rel}}^{(3)} - t_{\text{rel}}^{(2)} \geq \text{interval}, \quad (23)$$

$$0 \leq \text{delay}^{(k)} \leq 6 \text{ s} \quad (k = 1, 2, 3), \quad (24)$$

$$t_{\text{burst}}^{(k)} = t_{\text{rel}}^{(k)} + \text{delay}^{(k)} \leq T_{\text{end}} \quad (k = 1, 2, 3). \quad (25)$$

导弹从 M_1^0 出发，以 v_{missile} 指向 **fake** 匀速直线飞行：

$$M_1^t = M_1^0 + v_{\text{missile}} t \frac{\text{fake} - M_1^0}{\|\text{fake} - M_1^0\|}. \quad (26)$$

无人机 FY1 等高直线匀速，水平方向由 θ 给出。第 k 枚弹在 $t_{\text{rel}}^{(k)}$ 投放，延时 $\text{delay}^{(k)}$ 后于 $t_{\text{burst}}^{(k)}$ 起爆，其起爆位置为

$$P_{\text{burst}}^{(k)} = FY_1^0 + v_{\text{drone}} t_{\text{burst}}^{(k)} (\cos \theta, \sin \theta, 0) + (0, 0, -\frac{1}{2}g [t_{\text{burst}}^{(k)} - t_{\text{rel}}^{(k)}]^2). \quad (27)$$

起爆后云团以 3m/s 的速度竖直匀速下沉，且每枚烟雾干扰弹的评估时间窗为

$$t \in [t_{\text{burst}}^{(k)}, \min(t_{\text{burst}}^{(k)} + T_{\text{effective}}, T_{\text{end}})], \quad (28)$$

$$C^{(k)}(t) = (P_{\text{burst},x}^{(k)}, P_{\text{burst},y}^{(k)}, P_{\text{burst},z}^{(k)} - v_{\text{cloud}} [t - t_{\text{burst}}^{(k)}]). \quad (29)$$

遮蔽判据沿用问题一的“严格全覆盖”：在任一时刻 t ，对真目标（底面中心 $real$ 、半径 R 、高度 H 的圆柱体）可见表面离散采样。若对每个采样点 B 都有

$$\min_{s \in [0,1]} \|s M_1^t + (1-s) B - C^{(k)}(t)\| \leq R_{\text{effective}}, \quad (30)$$

则判定“第 k 枚弹在时刻 t 形成完全遮蔽”。将满足条件的时刻并为区间集合 $\{[s_j^{(k)}, e_j^{(k)}]\}$ 。

4.3.2 模型的求解

三枚弹的遮蔽区间取 **并集**

$$\mathcal{U} = \bigcup_{k=1}^3 \bigcup_j [s_j^{(k)}, e_j^{(k)}], \quad (31)$$

目标是在满足上述边界条件与物理约束的条件下，最大化并集的总长度，即总有效遮蔽时长：

$$\max_{v_{\text{drone}}, \theta, \{t_{\text{rel}}^{(k)}, \text{delay}^{(k)}\}_{k=1}^3} \text{len}(\mathcal{U}) \quad (32)$$

求解上采用遗传算法进行连续全局搜索。染色体使用实数编码：

$$\mathbf{x} = [\theta, v_{\text{drone}}, t_{\text{rel}}^{(1)}, \text{delay}^{(1)}, t_{\text{rel}}^{(2)}, \text{delay}^{(2)}, t_{\text{rel}}^{(3)}, \text{delay}^{(3)}].$$

每个个体在解码阶段被强制满足间隔约束与 $t_{\text{burst}}^{(k)} \leq T_{\text{end}}$ ($k = 1, 2, 3$)，适应度由严格判据下的区间并集长度给出。

为兼顾效率与精度，采用“粗筛+细筛”的两阶段评估：先用较大步长进行粗评快速筛选；再对精英及候选以较小步长进行严格复评，最终返回最优解及对应的 $P_{\text{burst}}^{(k)}$ 与并集区间。

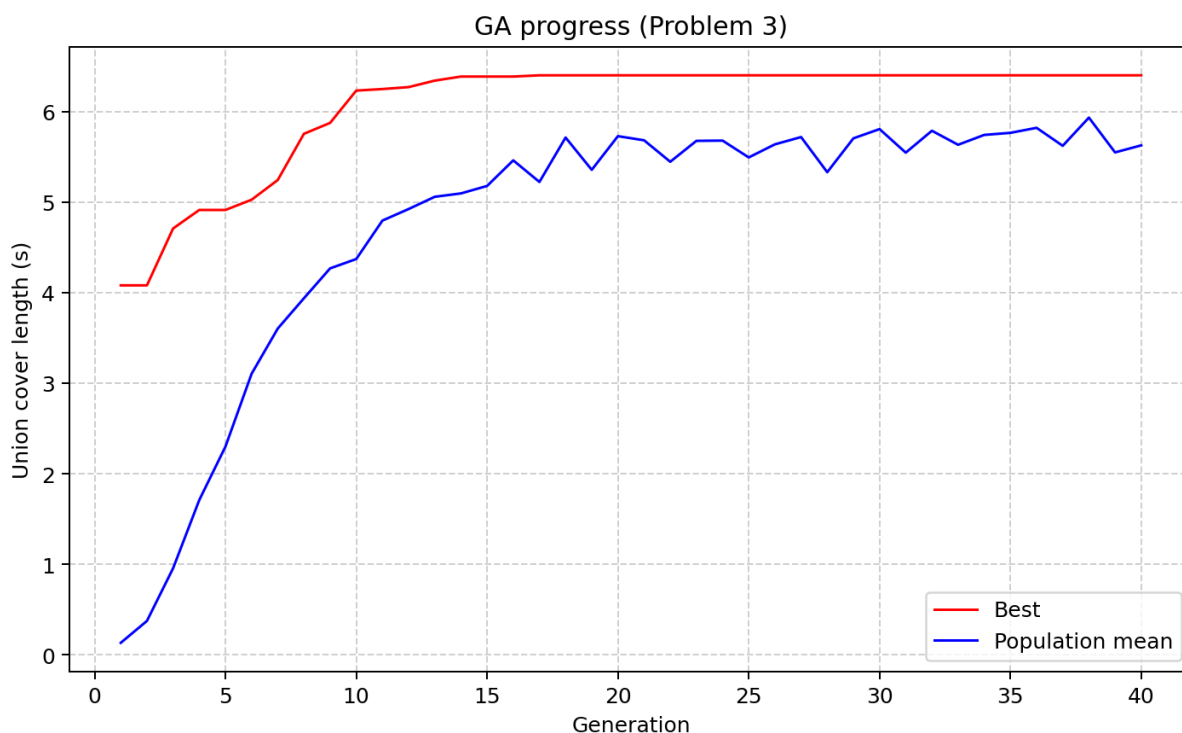


图 11 问题 3 的遗传算法进展图

最终结果为：

表 2 问题三的结果

烟雾干扰弹编号	投放时间 (s)	投放点坐标 (m)	起爆时间 (s)	起爆点坐标 (m)	覆盖区间 (s)	有效遮蔽时间 (s)
弹 #1	0.00	(17800.00, 0.00, 1800.00)	0.00	(17800.00, 0.00, 1800.00)	[(4.85, 7.4)]	2.55
弹 #2	1.00	(17937.12, 11.89, 1800.00)	1.00	(17937.12, 11.89, 1800.00)	[(1.0, 5.37)]	4.37
弹 #3	2.86	(18192.04, 33.99, 1800.00)	4.12	(18365.23, 49.01, 1792.18)	[]	0.00

问题三总体参数：

- 无人机航向 = 4.96° ，速度 = 137.6 m/s
- 三团烟雾干扰时间并集（严格）= **6.400 s**
- 并集区间 = [(1.0, 7.4)]

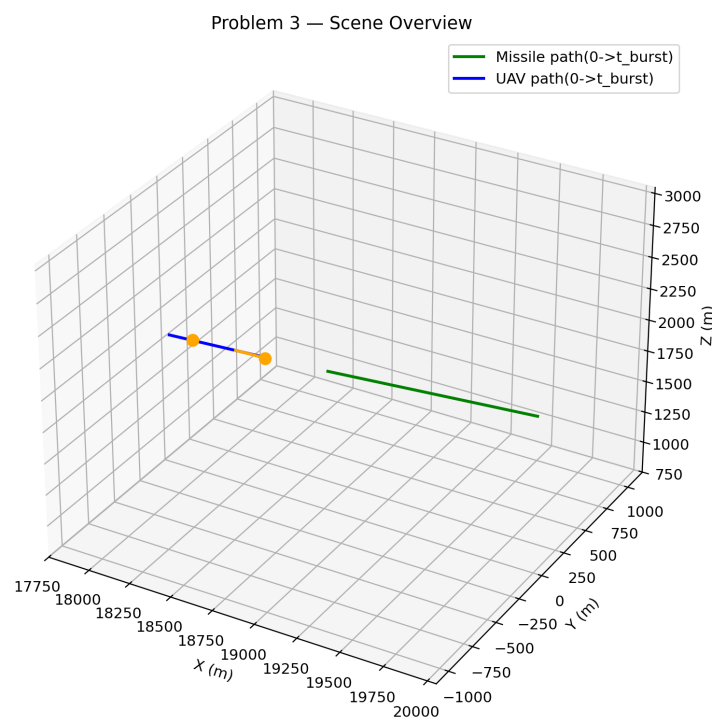


图 12 问题 3 的三维场景图

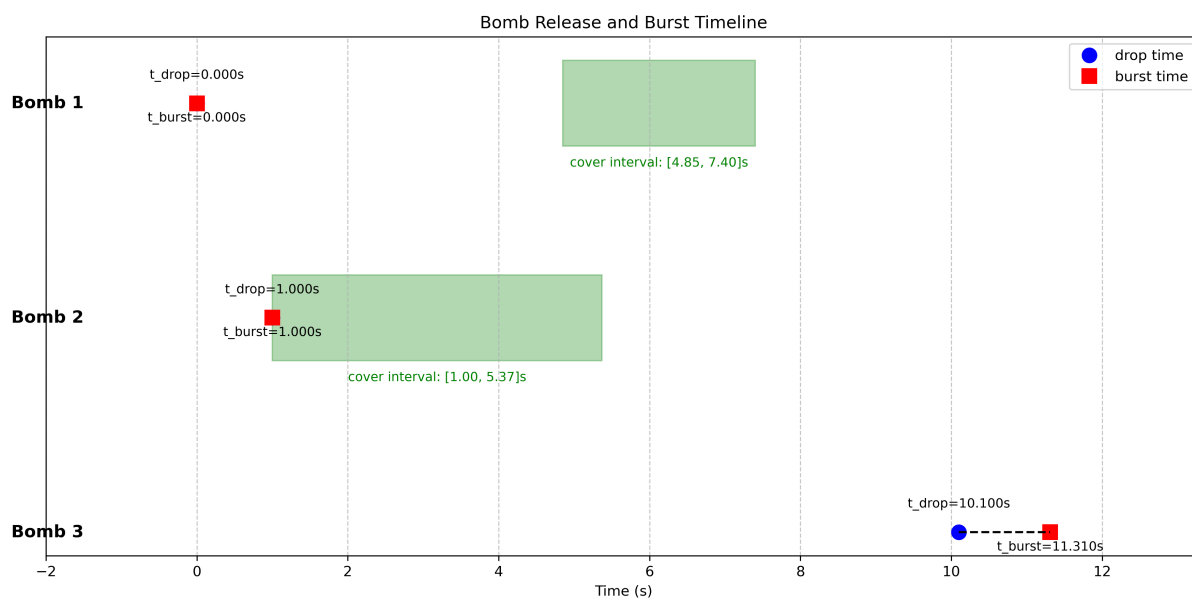


图 13 问题 3 投弹时间线

4.4 问题四模型的建立与求解

4.4.1 模型的建立

问题 4 是三架无人机各携带一枚烟幕干扰弹，目标是通过寻找合适的每架无人机的飞行速度，航向角，投弹时间，延时起爆时间，以期达到对单枚导弹 M1 遮蔽时间的最大化。本题采用三架无人机协同思路，先通过寻找每家无人机单独执行任务时最大化遮

蔽时间的最优解，再通过检验遮蔽时间是否重叠，如若未重叠，那么三个局部最优解一定为全局最优解；如若重叠，再在此基础上筛选其他更优解。

单机决策与可行域

对第 i 架无人机，决策为

$$x^{(i)} = (v_{\text{drone}}^{(i)}, \theta^{(i)}, t_{\text{drop}}^{(i)}, \text{delay}^{(i)}), \quad t_{\text{burst}}^{(i)} = t_{\text{drop}}^{(i)} + \text{delay}^{(i)}.$$

物理与边界约束逐行写为

$$v_{\min} \leq v_{\text{drone}}^{(i)} \leq v_{\max}, \quad (33)$$

$$-\pi \leq \theta^{(i)} \leq \pi, \quad (34)$$

$$0 \leq t_{\text{drop}}^{(i)} \leq T_{\text{end}}, \quad (35)$$

$$0 \leq \text{delay}^{(i)} \leq T_{\text{effective}}, \quad (36)$$

$$t_{\text{burst}}^{(i)} = t_{\text{drop}}^{(i)} + \text{delay}^{(i)} \leq T_{\text{end}}. \quad (37)$$

若出现 $t_{\text{drop}}^{(i)} + \text{delay}^{(i)} > T_{\text{end}}$ ，在解码阶段作可行化修复：

$$\text{delay}^{(i)} \leftarrow \max(0, T_{\text{end}} - t_{\text{drop}}^{(i)}), \quad t_{\text{burst}}^{(i)} = t_{\text{drop}}^{(i)} + \text{delay}^{(i)}.$$

运动学与云团演化

导弹从 M_1^0 出发、以 v_{missile} 指向 fake 匀速直线飞行：

$$M_1^t = M_1^0 + v_{\text{missile}} t \frac{\text{fake} - M_1^0}{\|\text{fake} - M_1^0\|}. \quad (38)$$

第 i 架无人机等高匀速、航向角为 $\theta^{(i)}$ 。平抛可得其起爆位置

$$P_{\text{burst}}^{(i)} = FY_i^0 + v_{\text{drone}}^{(i)} t_{\text{burst}}^{(i)} (\cos \theta^{(i)}, \sin \theta^{(i)}, 0) + (0, 0, -\frac{1}{2}g [t_{\text{burst}}^{(i)} - t_{\text{drop}}^{(i)}]^2). \quad (39)$$

起爆后云团竖直下沉，评估时窗与云团中心为

$$t \in [t_{\text{burst}}^{(i)}, \min(t_{\text{burst}}^{(i)} + T_{\text{effective}}, T_{\text{end}})], \quad (40)$$

$$C^{(i)}(t) = (P_{\text{burst},x}^{(i)}, P_{\text{burst},y}^{(i)}, P_{\text{burst},z}^{(i)} - v_{\text{cloud}} [t - t_{\text{burst}}^{(i)}]). \quad (41)$$

遮蔽判据与单机时长

沿用问题一的“严格全覆盖”判据：对目标圆柱（底面中心 $real$ 、半径 R 、高度 H ）

的可见表面均匀采样。若在某时刻 t 对每个采样点 B 都有

$$\min_{s \in [0,1]} \left\| s M_1^t + (1-s) B - C^{(i)}(t) \right\| \leq R_{\text{effective}}, \quad (42)$$

则认为该时刻被完全遮蔽。把满足条件的时刻并成区间集合 $\{[s_j^{(i)}, e_j^{(i)}]\}$ ，定义单机有效遮蔽时长

$$\text{CoverTime}^{(i)} = \sum_j (e_j^{(i)} - s_j^{(i)}). \quad (43)$$

三机联合指标（并集） FY1 采用问题二的最优方案，FY2/FY3 为本题所得最优方案。
三机遮蔽区间并集

$$\mathcal{U}_{\text{all}} = \left(\bigcup_j [s_j^{(1)}, e_j^{(1)}] \right) \cup \left(\bigcup_j [s_j^{(2)}, e_j^{(2)}] \right) \cup \left(\bigcup_j [s_j^{(3)}, e_j^{(3)}] \right). \quad (44)$$

将 $\mathcal{U}_{\text{all}}$ 合并排序为若干不相交的区间 $[S_\ell, E_\ell]$ ，则

$$\text{CoverTime}_{\text{all}} = \sum_\ell (E_\ell - S_\ell). \quad (45)$$

4.4.2 模型的求解

先找到每架无人机单独执行任务时的最优解，再通过检验遮蔽时间是否重叠，如若未重叠，那么三个局部最优解一定为全局最优解；如若重叠，再在此基础上去筛选其他更优解。

单机优化（ $i = 1, 2, 3$ 独立求解）

$$\max_{v_{\text{drone}}^{(i)}, \theta^{(i)}, t_{\text{drop}}^{(i)}, \text{delay}^{(i)}} \text{CoverTime}^{(i)} \quad \text{s.t. 上述边界与可行化约束。} \quad (46)$$

实现采用实数编码的遗传算法：染色体

$$[v_{\text{drone}}^{(i)}, \theta^{(i)}, t_{\text{drop}}^{(i)}, \text{delay}^{(i)}],$$

解码阶段计算 $t_{\text{burst}}^{(i)}$ 并执行可行化修复（保证 $t_{\text{burst}}^{(i)} \leq T_{\text{end}}$ ）；适应度由严格判据下的区间集合 $\{[s_j^{(i)}, e_j^{(i)}]\}$ 求得 $\text{CoverTime}^{(i)}$ 。为兼顾效率与精度，采用“粗评步长 + 精评步长”的两阶段评估：先用较大步长快速筛选，再对候选（含精英）以较小步长重评；进化操作为锦标赛选择、SBX 交叉与退火式高斯变异，并配以物理启发的初始种子以加速收敛。

并集合并与结果导出

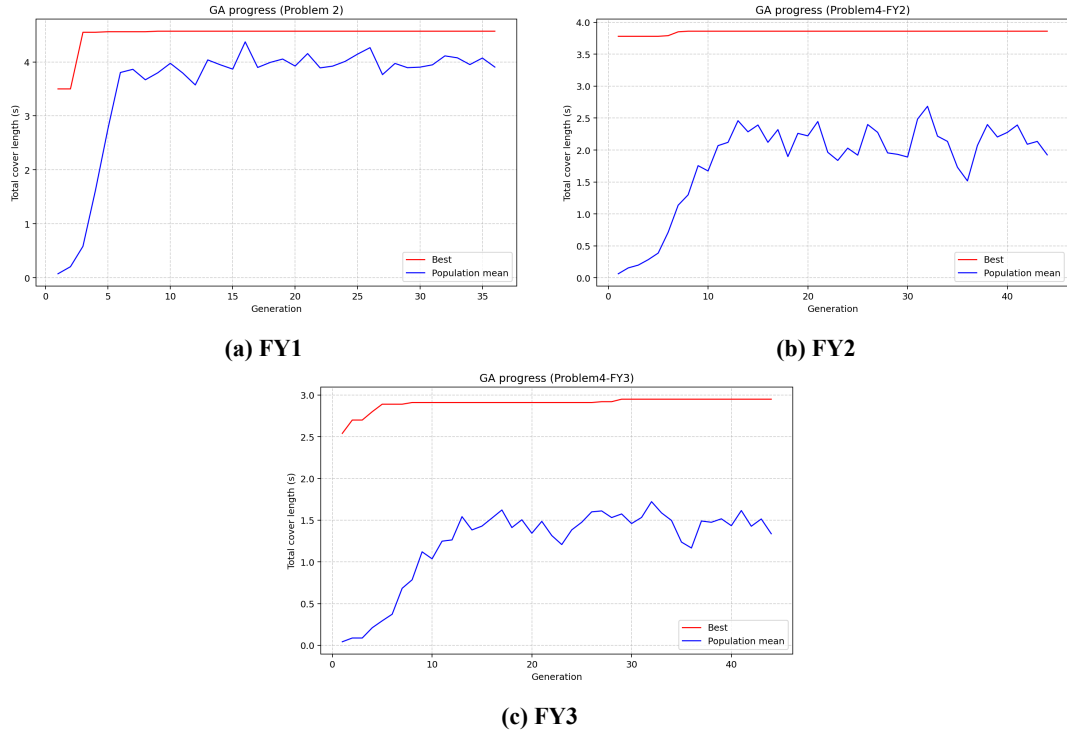


图 14 问题 4 遗传算法进展图

得到 FY1-FY3 的最优区间后，按时间排序有序合并得到 $\mathcal{U}_{\text{all}}$ 与 $\text{CoverTime}_{\text{all}}$ ；同时导出每架无人机的

$$(v_{\text{drone}}^{(i)}, \theta^{(i)}, t_{\text{drop}}^{(i)}, \text{delay}^{(i)}, t_{\text{burst}}^{(i)})$$

及其投放/起爆三维坐标（与问题二相同的直线—平抛公式）用于展示与复现：

$$(x, y)_{\text{drop}}^{(i)} = (FY_{i,x}^0, FY_{i,y}^0) + v_{\text{drone}}^{(i)} t_{\text{drop}}^{(i)} (\cos \theta^{(i)}, \sin \theta^{(i)}), \quad (47)$$

$$(x, y)_{\text{burst}}^{(i)} = (FY_{i,x}^0, FY_{i,y}^0) + v_{\text{drone}}^{(i)} t_{\text{burst}}^{(i)} (\cos \theta^{(i)}, \sin \theta^{(i)}), \quad (48)$$

$$z_{\text{drop}}^{(i)} = FY_{i,z}^0, \quad z_{\text{burst}}^{(i)} = FY_{i,z}^0 - \frac{1}{2}g[t_{\text{burst}}^{(i)} - t_{\text{drop}}^{(i)}]^2. \quad (49)$$

最终结果为：

最终对 M1 导弹的总有效遮蔽时间是 **11.38 s**。

问题四总体参数

表 3 炸弹参数数据

炸弹类型	覆盖长度 (s)	速度 (m/s)	航向 (°)	投弹时间 (s)	延迟 (s)	爆炸时间 (s)
FY1	4.570	74.623	6.371	0.461	0.793	1.254
FY2	3.860	140.000	-44.883	11.067	2.980	14.047
FY3	2.950	110.016	89.460	23.833	4.268	28.101

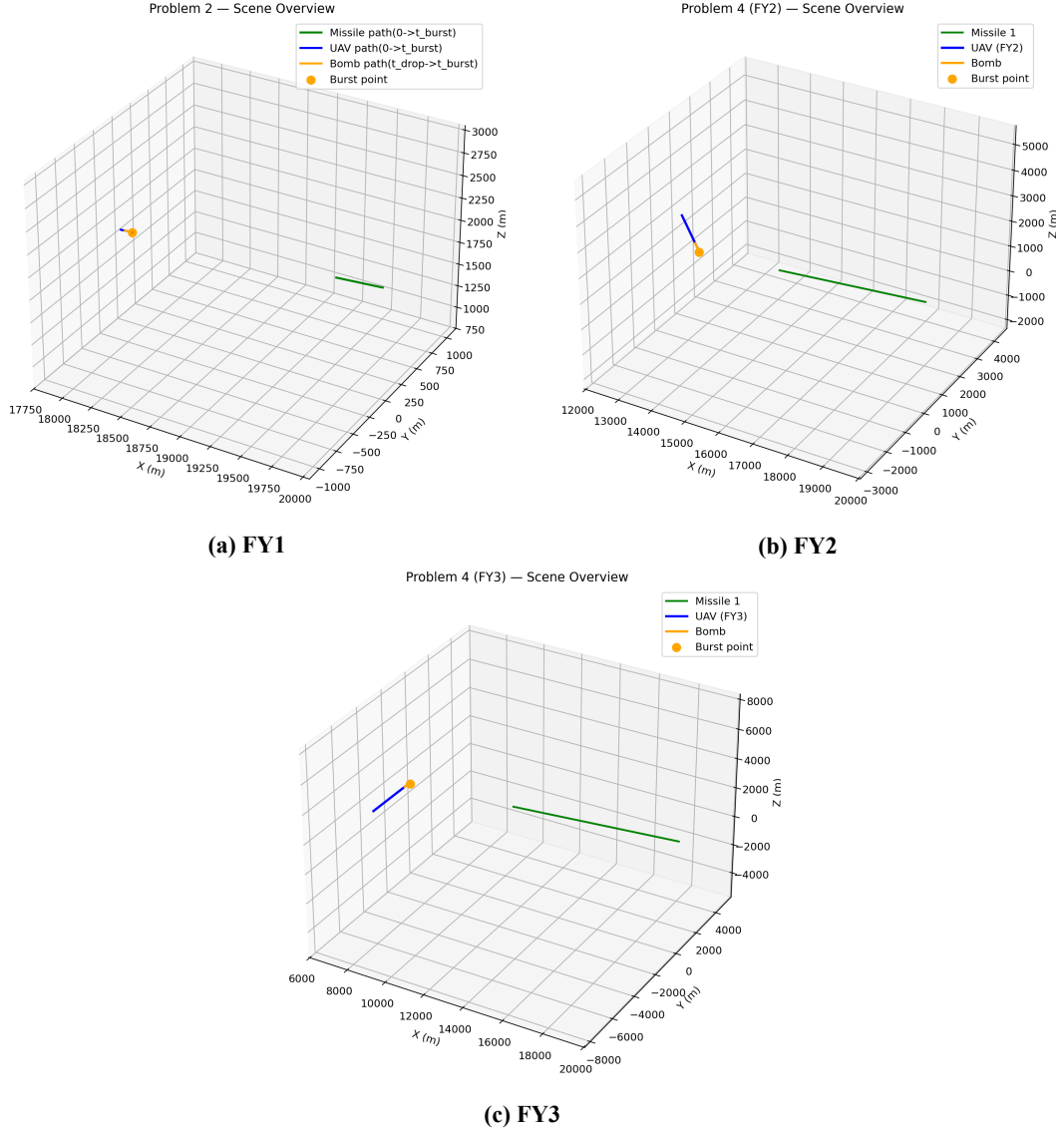


图 15 问题 4 各无人机投弹轨迹

4.5 问题五：模型的建立与求解

4.5.1 模型的建立

本题是“多机 × 多弹 × 多导弹”的复杂题目，先从单架无人机的三次投放出发，定义运动学、评估时间窗与遮蔽判据；随后给出“按导弹分组取并集并求和”的计分方式，最后对五架无人机求和得到总目标。上标⁽ⁱ⁾表示第*i*架无人机 ($i = 1, \dots, 5$)，上标^(k)表示第*k*次投放 ($k = 1, 2, 3$)，导弹编号为 $m = 1, 2, 3$ 。

对象与时间轴

三枚导弹均做等高匀速直线运动：

$$M_m^t = M_m^0 + v_{\text{missile}} t \frac{\text{fake} - M_m^0}{\|\text{fake} - M_m^0\|}, \quad m = 1, 2, 3. \quad (50)$$

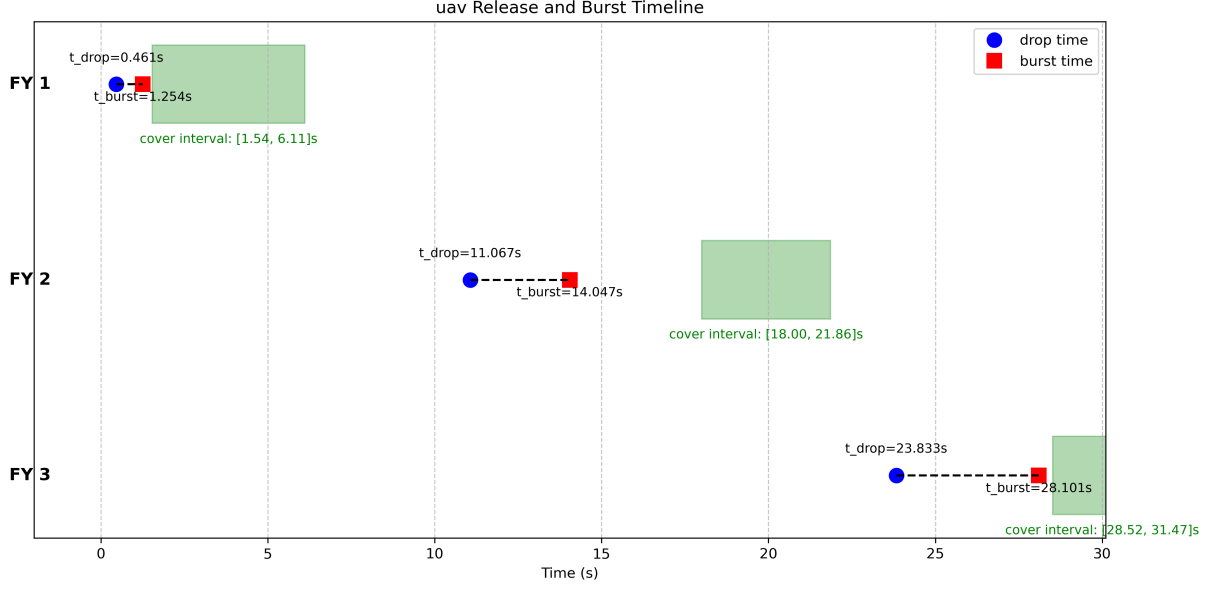


图 16 问题 4 投弹时间线

第 i 架无人机的决策量与起爆时刻:

$$x^{(i)} = \left(v_{\text{drone}}^{(i)}, \theta^{(i)}, \{t_{\text{rel}}^{(i,k)}, \text{delay}^{(i,k)}\}_{k=1}^3 \right), \quad t_{\text{burst}}^{(i,k)} = t_{\text{rel}}^{(i,k)} + \text{delay}^{(i,k)}. \quad (51)$$

起爆点（平抛运动）:

$$P_{\text{burst}}^{(i,k)} = FY_i^0 + v_{\text{drone}}^{(i)} t_{\text{burst}}^{(i,k)} (\cos \theta^{(i)}, \sin \theta^{(i)}, 0) + \left(0, 0, -\frac{1}{2}g [t_{\text{burst}}^{(i,k)} - t_{\text{rel}}^{(i,k)}]^2 \right). \quad (52)$$

云团中心与评估时间窗（为避免无效计算，引入统一上限 $T_{\cap}$ ）:

$$C^{(i,k)}(t) = \left(P_x^{(i,k)}, P_y^{(i,k)}, P_z^{(i,k)} - v_{\text{cloud}} [t - t_{\text{burst}}^{(i,k)}] \right), \quad (53)$$

$$t \in \left[t_{\text{burst}}^{(i,k)}, \min(t_{\text{burst}}^{(i,k)} + T_{\text{effective}}, T_{\text{end}}) \right] \cap [0, T_{\cap}], \quad (54)$$

其中

$$t_{\text{arr}}^{(m)} = \frac{\|\text{fake} - M_m^0\|}{v_{\text{missile}}}, \quad T_{\cap} = \min\left(T_{\text{end}}, \max_{m \in \{1,2,3\}} t_{\text{arr}}^{(m)}\right). \quad (55)$$

约束与可行化（与实现一致）

$$v_{\min} \leq v_{\text{drone}}^{(i)} \leq v_{\max}, \quad (56)$$

$$-\pi \leq \theta^{(i)} \leq \pi, \quad (57)$$

$$0 \leq t_{\text{rel}}^{(i,1)}, \quad (58)$$

$$t_{\text{rel}}^{(i,2)} - t_{\text{rel}}^{(i,1)} \geq \text{interval}, \quad (59)$$

$$t_{\text{rel}}^{(i,3)} - t_{\text{rel}}^{(i,2)} \geq \text{interval}, \quad (60)$$

$$0 \leq \text{delay}^{(i,k)} \leq T_{\text{effective}}, \quad k = 1, 2, 3, \quad (61)$$

$$t_{\text{burst}}^{(i,k)} = t_{\text{rel}}^{(i,k)} + \text{delay}^{(i,k)} \leq \min(T_{\text{end}}, T_{\cap}). \quad (62)$$

为稳健起见，投放时刻采用差分编码并在解码时保证“严格间隔”：

$$t_{\text{rel}}^{(i,1)} = d_1, \quad t_{\text{rel}}^{(i,2)} = t_{\text{rel}}^{(i,1)} + \text{interval} + d_2, \quad t_{\text{rel}}^{(i,3)} = t_{\text{rel}}^{(i,2)} + \text{interval} + d_3, \quad d_2, d_3 \geq 0, \quad (63)$$

若 $t_{\text{rel}}^{(i,k)} + \text{delay}^{(i,k)}$ 超出窗口上限，则在解码阶段对 $\text{delay}^{(i,k)}$ 做裁剪修复以满足可行性。

遮蔽判据（严格全覆盖）与导弹分组并集

对圆柱目标（底面中心 $real$ 、半径 R 、高度 H ）可见表面离散采样。若在时刻 t 对每个采样点 B 均有

$$\min_{s \in [0,1]} \left\| s M_m^t + (1-s) B - C^{(i,k)}(t) \right\| \leq R_{\text{effective}}, \quad (64)$$

则判定“第 i 架无人机的第 k 次起爆在时刻 t 对导弹 m 形成完全遮蔽”，并将这些时刻并为若干区间。对固定 (i, m) ，把三次起爆的区间取并集

$$\mathcal{U}_m^{(i)} = \bigcup_{k=1}^3 \bigcup_j [s_j^{(i,k,m)}, e_j^{(i,k,m)}], \quad (65)$$

其时长记为 $\text{len}(\mathcal{U}_m^{(i)})$ 。单机目标定义为三枚导弹的并集时长之和

$$\sum_{m=1}^3 \text{len}(\mathcal{U}_m^{(i)}), \quad (66)$$

而总体目标为五架无人机之和

$$\sum_{i=1}^5 \sum_{m=1}^3 \text{len}(\mathcal{U}_m^{(i)}). \quad (67)$$

4.5.2 模型的求解

求解策略沿用前文：把不可微、强几何约束的评估核固定在内层，外层以遗传算法（GA）在连续可行域上进行全局搜索；为控制代价且保证稳健，采用“粗评先筛、精评定分”的两阶段评估，并配合缓存与上界剪枝。

五个单机子问题（互相独立）

$$\max_{v_{\text{drone}}^{(i)}, \theta^{(i)}, \{t_{\text{rel}}^{(i,k)}, \text{delay}^{(i,k)}\}_{k=1}^3} \sum_{m=1}^3 \text{len}(\mathcal{U}_m^{(i)}) \quad \text{s.t. 上述边界与可行化约束}, \quad i = 1, \dots, 5. \quad (68)$$

实现细节：染色体

$$[\theta^{(i)}, v_{\text{drone}}^{(i)}, d_1, \text{delay}^{(i,1)}, d_2, \text{delay}^{(i,2)}, d_3, \text{delay}^{(i,3)}],$$

解码生成 $t_{\text{rel}}^{(i,k)}$ 、 $t_{\text{burst}}^{(i,k)}$ 并作可行化修复；粗评用较大步长并启用快速充分判据预筛，精评用较小步长并以严格判据重评精英/候选；进化采用锦标赛选择、SBX 交叉与退火式高斯变异，并以物理可行的初始种子加速收敛。

结果汇总与导出

五个子问题最优后，直接求和得到总体目标；同时得到各无人机的最优参数

$$(v_{\text{drone}}^{(i)}, \theta^{(i)}, t_{\text{rel}}^{(i,k)}, \text{delay}^{(i,k)}, t_{\text{burst}}^{(i,k)})$$

及投放/起爆三维坐标（与前题相同的直线—平抛公式），并给出按导弹分组的区间并集时长明细，用于复现与对比。

最终结果为：

问题五遮蔽时间汇总：

表 4 问题五汇总

指标	数值
FY1 总并集时长 (s)	6.4
FY2 总并集时长 (s)	5.28
FY3 总并集时长 (s)	5.41
FY4 总并集时长 (s)	0.0
FY5 总并集时长 (s)	5.18
总计 (s)	22.27

表 5 问题五结果

第一行：姿态与时序					
无人机	航向角 ($^{\circ}$)	速度 (m/s)	弹号	投放时刻 $t_{\text{rel}}(\text{s})$	延迟 $\tau(\text{s})$
第二行：坐标与指标					
投放点坐标 (m)		起爆点坐标 (m)		有效时长 (s) / 干扰导弹	
FY1	5.012915	140.000000	1	0.000295	0.000000
(17800, 0, 1800)		(17800, 0, 1800)		2.55 / 1	
FY1	5.012915	140.000000	2	1.001805	0.000000
(17937.12, 11.88975, 1800)		(17937.12, 11.88975, 1800)		4.37 / 1	
FY1	5.012915	140.000000	3	10.097400	1.208209
(18192.04, 33.99482, 1800)		(18365.23, 49.01288, 1792.182)		0.00 / —	
FY2	295.073264	127.020000	1	7.910000	0.610000
(12424.5, 489.2, 1400)		(12457.1, 419.5, 1398.18)		0.00 / —	
FY2	295.073264	127.020000	2	13.140000	2.680000
(12705, -112, 1400)		(12849, -421, 1364.81)		3.81 / 2	
FY2	295.073264	127.020000	3	25.330000	6.500000
(13359, -1516, 1400)		(13708, -2263, 1192.98)		1.47 / 3	
FY3	91.394820	109.999400	1	23.774860	4.232581
(5937.5, -384, 700)		(5926.4, 80, 612.24)		2.82 / 1	
FY3	91.394820	109.999400	2	27.194320	2.342137
(5928.5, -10, 700)		(5922.4, 250, 673.12)		2.59 / 2	
FY3	91.394820	109.999400	3	28.356410	1.848465
(5925.5, 120, 700)		(5920.7, 320, 683.26)		0.00 / —	
FY4	0.000000	70.000000	1	0.920000	0.500000
(11064.4, 2000, 1800)		(11099.4, 2000, 1798.78)		0.00 / —	
FY4	0.000000	70.000000	2	1.920001	1.340000
(11134.4, 2000, 1800)		(11228.2, 2000, 1791.2)		0.00 / —	
FY4	0.000000	70.000000	3	2.920002	0.500000
(11204.4, 2000, 1800)		(11239.4, 2000, 1798.78)		0.00 / —	

Continued on next page

第一行：姿态与时序（续）					
无人机	航向角 (°)	速度 (m/s)	弹号	投放时刻 $t_{\text{rel}}(\text{s})$	延迟 $\tau(\text{s})$
第二行：坐标与指标					
投放点坐标 (m)			起爆点坐标 (m)		有效时长 (s) / 干扰导弹
FY5	120.108365	104.780075	1	15.962982	1.695641
(12161.5, -555, 1300)			(12072, -400, 1285.91)		3.55 / 1
FY5	120.108365	104.780075	2	17.493743	4.666900
(12080, -415, 1300)			(11835, 0, 1193.28)		1.63 / 2
FY5	120.108365	104.780075	3	23.308527	1.379184
(11775, 100, 1300)			(11705, 240, 1290.68)		0.00 / —

五、模型的评价与推广

5.1 模型的评价

5.1.1 模型的优点

1 模型并没有对真目标进行质点近似，且对有效遮蔽的判据进行了严格设计，相当精确地对投弹过程中的运动模型进行了数学描述，利用运动学公式和几何关系，能准确地在确定投弹策略的前提下计算出有效遮蔽时间。

2 求解时使用了遗传算法作为主要优化方法，这种算法可适应不同目标函数的初始条件，能准确地找到较优的投弹策略。

3 在问题四与问题五中采用了贪心算法进行优化，极大减少了直接求解的复杂度。

5.1.2 模型的不足

对复杂情景有较大的计算量，可能导致较高的计算时间，在现实中，缺乏反导系统需要的及时性。但是考虑军事用途，这些计算量对于军用级别的计算机来说是可以接受的。

5.2 模型的推广

基于遗传算法，问题五中的基因变量包含了五架无人机的航向、速度、三枚烟幕干扰弹的投放时间、引爆时间。过多的基因变量会使遗传算法的计算量陡增、计算时间过长和计算准确度下降，因此我们首先利用贪心算法将问题初步分解为五个独立的单机遮

蔽三弹的优化子问题，再将结果汇总处理，不断迭代，使三枚导弹的有效遮蔽时间并集尽可能长

参考文献

- [1] E. Mezura-Montes and C. A. Coello Coello, “Constraint-handling in nature-inspired numerical optimization: Past, present and future,” *Swarm and Evolutionary Computation*, vol. 1, no. 4, pp. 173–194, 2011.
- [2] 姜启源. 数学模型 [M]. 北京: 高等教育出版社, 2007 年.
- [3] K. Deb and R. B. Agrawal, “Simulated Binary Crossover for Continuous Search Space,” *Complex Systems*, vol. 9, no. 2, pp. 115–148, 1995.
- [4] DeepSeek-V3, DeepSeek-V3, 深度求索 (DeepSeek), 2025-09-07.

附录 A 支撑材料的文件列表

- code
 - evaluator.py 评估模块
 - funcs.py 通用函数
 - ga.py 遗传算法模块
 - params.py 参数文件
 - problem1.py 问题一代码
 - problem2.py 问题二代码
 - problem3.py 问题三代码
 - problem4_fy2.py 问题四 FY2 代码
 - problem4_fy3.py 问题四 FY3 代码
 - problem4.py 问题四代码
 - problem5.py 问题五代码
 - figure.py 绘图代码
 - timeline1.py 问题一时间线
 - timeline2.py 问题二时间线
 - timeline3.py 问题三时间线
 - timeline4.py 问题四时间线
- result
 - result1.xlsx 问题一结果
 - result2.xlsx 问题二结果
 - result3.xlsx 问题三结果
- 数据结果.pdf
- AI 工具使用详情.pdf

附录 B 源代码

以下是参数与通用函数代码：

```
1 from __future__ import annotations
2 import math
3 from functools import lru_cache
4 from typing import List, Tuple
5 import numpy as np
6 import params as p
7 from funcs import unit, seg_point_min_dist
8
```

```

9 # 开关: 为避免误杀, 这里关闭快速充分判据预筛 (需要速度再打开)
10 USE_QUICK_PRECHECK = False
11
12
13 def sample_cylinder_surface(R=p.R, H=p.H, n_theta=160, n_z=25, n_r=8) -> np.ndarray:
14     """
15     采样圆柱体表面点
16     """
17     pts = []
18     angle_1, angle_2 = -0.75*np.pi, 0.75*np.pi # 只采前 270° 贴题意, 同时减少计算量
19
20     # 前侧面
21     for i in range(n_theta):
22         th = angle_1 + (angle_2 - angle_1) * i / n_theta
23         x = p.real[0] + R * np.cos(th)
24         y = p.real[1] + R * np.sin(th)
25         for j in range(n_z + 1):
26             z = p.real[2] + H * j / n_z
27             pts.append([x, y, z])
28
29     # 上端面
30     z = p.real[2] + H
31     for ir in range(n_r + 1):
32         r = R * ir / n_r
33         for i in range(n_theta):
34             th = 2 * np.pi * i / n_theta
35             x = p.real[0] + r * np.cos(th)
36             y = p.real[1] + r * np.sin(th)
37             pts.append([x, y, z])
38
39     return np.asarray(pts, float)
40
41
42 SURF = sample_cylinder_surface()
43
44
45 """
46 基本运动学函数
47 """
48 def missile_dir(M0: np.ndarray) -> np.ndarray:
49     return unit(p.fake - M0)
50
51 def missile_pos_general(M0: np.ndarray, t: float) -> np.ndarray:
52     return M0 + p.v_missile * t * missile_dir(M0)
53
54 def munition_pos_general(FY0: np.ndarray, v: float, dir_xy: np.ndarray, t_drop: float, t:
    float) -> np.ndarray:

```

```

55     if t < t_drop:
56         raise ValueError("t 必须 < t_drop")
57     FY_drop = FY0 + v * t_drop * dir_xy
58     dt = t - t_drop
59     pos = FY_drop + v * dt * dir_xy
60     pos = pos.copy()
61     pos[2] = FY_drop[2] - 0.5 * p.g * dt**2
62     return pos
63
64 def cloud_pos(t: float, t_burst: float, P_burst: np.ndarray) -> np.ndarray:
65     if t < t_burst:
66         raise ValueError("t 必须 < t_burst")
67     return np.array([P_burst[0], P_burst[1], P_burst[2] - p.v_cloud * (t - t_burst)], float)
68
69
70 def covered_strict_at_time(t: float, t_burst: float, P_burst: np.ndarray,
71                           M0: np.ndarray, Psurf: np.ndarray) -> bool:
72     """
73     严格判据: t 时刻导弹位置 Mt 是否被云雾覆盖
74     """
75     Mt = missile_pos_general(M0, t)
76     Ct = cloud_pos(t, t_burst, P_burst)
77     d = seg_point_min_dist(Ct, Mt, Psurf)
78     return np.max(d - p.R_effective) <= 1e-6
79
80
81 Tb = np.array([p.real[0], p.real[1], p.real[2] + p.H/2.0], float) # 圆柱体中心点
82 rb = float(np.sqrt(p.R**2 + (p.H/2.0)**2)) # 圆柱体外接球半径
83
84
85 def quick_sufficient_cover(Mt: np.ndarray, Ct: np.ndarray) -> bool:
86     """
87     以圆柱体外接球代替圆柱体的快速充分判据
88     仅当云雾中心 Ct 在导弹 Mt 与圆柱体外接球连线的“导弹侧”时,才可能遮蔽
89     可能误杀(即判定不遮蔽但实际上遮蔽了),但不会误判
90     """
91     v1, v2 = Tb - Mt, Ct - Mt
92     d1, d2 = np.linalg.norm(v1), np.linalg.norm(v2)
93
94     if d1 <= rb:
95         return False
96     if d2 <= p.R_effective:
97         return True
98
99     alpha = math.asin(min(1.0, rb / d1))
100     beta = math.asin(min(1.0, p.R_effective / d2))
101     cosd = float(np.dot(v1, v2) / (d1 * d2)); cosd = max(-1.0, min(1.0, cosd))

```

```

102     delta = math.acos(cosd)
103     return beta >= alpha + delta
104
105
106 def quick_has_any(M0: np.ndarray, P_burst: np.ndarray, t_burst: float,
107                  dt: float, t_cap: float) -> bool:
108     """
109     只要有一个采样点被遮蔽就返回 True
110     """
111     dt = float(dt if dt > 0 else 1e-6)
112     for t in np.arange(t_burst, min(t_burst + p.t_effective, t_cap) + 1e-12, dt):
113         Ct = cloud_pos(t, t_burst, P_burst)
114         Mt = missile_pos_general(M0, t)
115         if quick_sufficient_cover(Mt, Ct):
116             return True
117     return False
118
119
120 def _q(x: float, step: float) -> float:
121     return float(round(x / step) * step)
122
123 def _pos_dt(x: float) -> float:
124     return float(x if x > 0 else 1e-6)
125
126
127 @lru_cache(maxsize=120000)
128 def _core_intervals_q(
129     dt_q: float, v_q: float, theta_q: float, t_drop_q: float, t_burst_q: float,
130     FYx: float, FYy: float, FYz: float, Mx: float, My: float, Mz: float
131 ) -> Tuple[Tuple[Tuple[float, float], ...], float]:
132     dt_q = _pos_dt(dt_q)
133     FY0 = np.array([FYx, FYy, FYz], float)
134     M0 = np.array([Mx, My, Mz ], float)
135     dir_xy = unit(np.array([math.cos(theta_q), math.sin(theta_q), 0.0], float))
136
137     P_burst = munition_pos_general(FY0, v_q, dir_xy, t_drop_q, t_burst_q)
138     T_cap = min(getattr(p, "T_end", float("inf")), float(np.linalg.norm(p.fake - M0) /
139                    p.v_missile))
139     t0, t1 = t_burst_q, min(t_burst_q + p.t_effective, T_cap)
140     if t1 <= t0:
141         return tuple(), 0.0
142
143     if USE_QUICK_PRECHECK:
144         if not quick_has_any(M0, P_burst, t_burst_q, max(0.12, 3.0*dt_q), T_cap):
145             return tuple(), 0.0
146
147     times = np.arange(t0, t1 + 1e-12, dt_q)

```



```

148 flags = [covered_strict_at_time(tt, t_burst_q, P_burst, M0, SURF) for tt in times]
149
150 ints: List[Tuple[float, float]] = []
151 in_run, s = False, None
152 for i, ok in enumerate(flags):
153     if ok and not in_run:
154         in_run, s = True, times[i]
155     elif (not ok) and in_run:
156         ints.append((float(s), float(times[i-1])))
157         in_run = False
158 if in_run:
159     ints.append((float(s), float(times[-1])))
160
161 total = float(sum(e - s for s, e in ints))
162 return tuple(ints), total
163
164
165 def cover_intervals_cached_general(
166     dt: float, v_drone: float, direction: np.ndarray,
167     t_drop: float, t_burst: float, FY0: np.ndarray, M0: np.ndarray,
168     *, stage: int = 0, precheck: bool = False
169 ) -> Tuple[List[Tuple[float, float]], float]:
170     """
171     量化参数后调用核心函数计算覆盖区间
172     """
173     # 分阶段量化（精评更细）
174     theta_step = math.radians(0.25 if stage == 1 else 0.5)
175     v_step = 0.25 if stage == 1 else 0.5
176     t_step = 0.01 if stage == 1 else 0.02
177     dt_step = 0.005 if stage == 1 else 0.01
178
179     dir_xy = unit(np.array([direction[0], direction[1], 0.0], float))
180     theta = math.atan2(dir_xy[1], dir_xy[0])
181
182     dt_q = _pos_dt(_q(dt, dt_step))
183     v_q = _q(v_drone, v_step)
184     theta_q = _q(theta, theta_step)
185     t_drop_q = _q(t_drop, t_step)
186     t_burst_q = _q(t_burst, t_step)
187
188     # 快速充分判据预筛
189     if precheck and stage == 0:
190         P_burst = munition_pos_general(FY0, v_q, unit(np.array([math.cos(theta_q),
191             math.sin(theta_q), 0.0], float)),
192             t_drop_q, t_burst_q)
193         T_cap = min(getattr(p, "T_end", float("inf")),
194             float(np.linalg.norm(p.fake - M0) / p.v_missile))

```

```

194         if not quick_has_any(M0, P_burst, t_burst_q, max(0.12, 3.0*dt_q), T_cap):
195             return [], 0.0
196
197     ints_t, total = _core_intervals_q(
198         dt_q, v_q, theta_q, t_drop_q, t_burst_q,
199         float(FY0[0]), float(FY0[1]), float(FY0[2]),
200         float(M0[0]), float(M0[1]), float(M0[2]),
201     )
202     return list(ints_t), float(total)
203
204
205 # 兼容: FY1/M1
206 def cover_intervals(dt: float, v_drone: float, direction: np.ndarray,
207                   t_drop: float, t_burst: float) -> Tuple[List[Tuple[float,float]], float]:
208     return cover_intervals_cached_general(dt, v_drone, direction, t_drop, t_burst, p.FY1_0,
209                                         p.M1_0, stage=1)

```

```

1  import numpy as np
2
3
4  def wrap_deg_0_360(deg: float) -> float:
5      """
6      将角度归一化到 [0, 360)
7      """
8      x = deg % 360.0
9      return x + 360.0 if x < 0 else x
10
11
12  def unit(vec: np.ndarray) -> np.ndarray:
13      """
14      返回单位向量
15      """
16      n = np.linalg.norm(vec)
17      return vec / n if n > 0 else vec
18
19
20  def seg_point_min_dist(c: np.ndarray, a: np.ndarray, b: np.ndarray) -> np.ndarray:
21      """
22      计算点 c 到线段 ab 的最短距离（向量化）
23      """
24      ab = b - a[None, :]
25      d2 = np.einsum("ij,ij->i", ab, ab)
26      d2 = np.where(d2 == 0.0, 1e-12, d2)
27      s = np.einsum("j,ij->i", c - a, ab) / d2
28      s = np.clip(s, 0.0, 1.0)
29      closest = a[None, :] + ab * s[:, None] # 投影点

```

```

30     return np.linalg.norm(c[None, :] - closest, axis=1)
31
32
33 def union_length(intervals):
34     """
35     计算多个区间的并集长度
36     """
37     valid = [(s, e) for (s, e) in intervals if (s is not None and e is not None and e > s)]
38     if not valid:
39         return 0.0, []
40     valid.sort()
41     merged = []
42     cs, ce = valid[0]
43     for s, e in valid[1:]:
44         if s <= ce:
45             ce = max(ce, e)
46         else:
47             merged.append((cs, ce))
48             cs, ce = s, e
49     merged.append((cs, ce))
50     return sum(e - s for s, e in merged), merged
51
52
53 def set_axes_equal_3d(ax):
54     """
55     让三轴缩放一致，球体/圆柱不被拉伸
56     """
57     x_limits = ax.get_xlim3d()
58     y_limits = ax.get_ylim3d()
59     z_limits = ax.get_zlim3d()
60     x_range = abs(x_limits[1] - x_limits[0])
61     y_range = abs(y_limits[1] - y_limits[0])
62     z_range = abs(z_limits[1] - z_limits[0])
63     max_range = max([x_range, y_range, z_range])
64     x_middle = np.mean(x_limits)
65     y_middle = np.mean(y_limits)
66     z_middle = np.mean(z_limits)
67     ax.set_xlim3d([x_middle - max_range/2, x_middle + max_range/2])
68     ax.set_ylim3d([y_middle - max_range/2, y_middle + max_range/2])
69     ax.set_zlim3d([z_middle - max_range/2, z_middle + max_range/2])

```

```

1  from __future__ import annotations
2  from typing import Callable, Optional, Tuple, List
3  import numpy as np
4  import random
5

```

```

6 Array = np.ndarray
7
8 class GeneticAlgorithm:
9     def __init__(
10         self,
11         lb: Array,
12         ub: Array,
13         pop_size: int = 32,
14         n_gen: int = 40,
15         seed: int = 123,
16         tournament_k: int = 3,
17         elite_frac: float = 0.10,
18         sbx_eta: float = 12.0,
19         sbx_prob: float = 0.9,
20         mut_rate: float = 0.25,
21         mut_scale: float = 0.12,
22         objective: str = "max",
23     ) -> None:
24         assert lb.shape == ub.shape, "lb/ub shape mismatch"
25         self.lb = lb.astype(float)
26         self.ub = ub.astype(float)
27         self.dim = lb.size
28
29         self.pop_size = int(pop_size)
30         self.n_gen = int(n_gen)
31         self.k = int(tournament_k)
32         self.elite_frac = float(elite_frac)
33
34         self.sbx_eta = float(sbx_eta)
35         self.sbx_prob = float(sbx_prob)
36         self.mut_rate = float(mut_rate)
37         self.mut_scale = float(mut_scale)
38
39         assert objective in ("max", "min")
40         self.objective = objective
41
42         self.rng = np.random.default_rng(int(seed))
43         random.seed(int(seed))
44
45     def _clip(self, x: Array) -> Array:
46         return np.minimum(self.ub, np.maximum(self.lb, x))
47
48     def _init_population(self) -> Array:
49         return self.rng.uniform(self.lb, self.ub, size=(self.pop_size, self.dim))
50
51     def _is_better(self, s_new: float, s_old: float) -> bool:
52         if self.objective == "max":

```

```

53         return s_new > s_old
54     else:
55         return s_new < s_old
56
57 def _tournament_select(self, pop: Array, scores: Array) -> Array:
58     idxs = self.rng.integers(0, self.pop_size, size=self.k)
59     if self.objective == "max":
60         best = idxs[np.argmax(scores[idxs])]
61     else:
62         best = idxs[np.argmin(scores[idxs])]
63     return pop[best].copy()
64
65 def _sbx(self, p1: Array, p2: Array) -> Tuple[Array, Array]:
66     """
67     SBX交叉
68     """
69     if random.random() > self.sbx_prob:
70         return p1.copy(), p2.copy()
71     c1, c2 = p1.copy(), p2.copy()
72     for j in range(self.dim):
73         u = random.random()
74         if u <= 0.5:
75             beta = (2*u)**(1.0/(self.sbx_eta+1.0))
76         else:
77             beta = (1/(2*(1-u)))** (1.0/(self.sbx_eta+1.0))
78             c1[j] = 0.5*((1+beta)*p1[j] + (1-beta)*p2[j])
79             c2[j] = 0.5*((1-beta)*p1[j] + (1+beta)*p2[j])
80     return self._clip(c1), self._clip(c2)
81
82 def _mutate(self, x: Array, sigma: Array) -> Array:
83     """
84     高斯变异
85     """
86     mask = self.rng.random(self.dim) < self.mut_rate
87     noise = self.rng.normal(0.0, 1.0, size=self.dim) * sigma
88     x2 = np.where(mask, x + noise, x)
89     return self._clip(x2)
90
91 def evolve(
92     self,
93     fitness_fn: Callable[[Array], float],
94     reeval_fn: Optional[Callable[[Array], float]] = None,
95     reeval_frac: float = 0.20,
96     verbose_every: int = 5,
97     log_fn: Optional[Callable[[int, float, Array], None]] = None,
98     repair_fn: Optional[Callable[[Array], Array]] = None,
99     seed_population: Optional[List[Array]] = None,

```

```

100     sigma_fn: Optional[Callable[[int, int, Array, Array], Array]] = None,
101     stats_hook: Optional[Callable[[int, float, float], None]] = None,
102 ) -> Tuple[Array, float]:
103
104     # init pop
105     pop = self._init_population()
106     if seed_population:
107         k = min(len(seed_population), self.pop_size)
108         for i in range(k):
109             pop[i, :] = self._clip(seed_population[i])
110
111     # initial repair (optional)
112     if repair_fn is not None:
113         for i in range(self.pop_size):
114             pop[i, :] = self._clip(repair_fn(pop[i, :]))
115
116     # evaluate
117     scores = np.array([fitness_fn(ind) for ind in pop], dtype=float)
118
119     # best so far
120     best_idx = int(np.argmax(scores) if self.objective == "max" else np.argmin(scores))
121     best_x = pop[best_idx].copy()
122     best_s = float(scores[best_idx])
123
124     elite_k = max(2, int(round(self.pop_size * self.elite_frac)))
125     reeval_k = max(2, int(round(self.pop_size * reeval_frac))) if reeval_fn else 0
126
127     for gen in range(1, self.n_gen + 1):
128         # per-gen sigma (anneal by default)
129         if sigma_fn is None:
130             anneal = max(0.3, 1.0 - gen / self.n_gen)
131             sigma = self.mut_scale * (self.ub - self.lb) * anneal
132         else:
133             sigma = sigma_fn(gen, self.n_gen, self.lb, self.ub)
134
135     # optional: refine top-p% with reeval_fn (only improve)
136     if reeval_k > 0:
137         if self.objective == "max":
138             order_tmp = np.argsort(scores)[::-1]
139         else:
140             order_tmp = np.argsort(scores)
141         top_idx = order_tmp[:reeval_k]
142         for i in top_idx:
143             s2 = reeval_fn(pop[i, :])
144             if self._is_better(s2, scores[i]):
145                 scores[i] = s2
146

```

```

147     # update best
148     curr_best_idx = int(np.argmax(scores) if self.objective == "max" else
149                        np.argmin(scores))
150
151     if self._is_better(scores[curr_best_idx], best_s):
152         best_x = pop[curr_best_idx].copy()
153         best_s = float(scores[curr_best_idx])
154
155     mean_s = float(np.mean(scores))
156
157     if stats_hook is not None:
158         stats_hook(gen, best_s, mean_s)
159
160     if verbose_every > 0 and (gen % verbose_every == 0):
161         if log_fn:
162             log_fn(gen, best_s, best_x)
163
164     # selection + create next population (elitism + SBX + mutation)
165     if self.objective == "max":
166         order = np.argsort(scores)[::-1]
167     else:
168         order = np.argsort(scores)
169     new_pop = [pop[i, :].copy() for i in order[:elite_k]]
170
171     while len(new_pop) < self.pop_size:
172         p1 = self._tournament_select(pop, scores)
173         p2 = self._tournament_select(pop, scores)
174         c1, c2 = self._sbx(p1, p2)
175         c1 = self._mutate(c1, sigma)
176         c2 = self._mutate(c2, sigma)
177         if repair_fn is not None:
178             c1 = self._clip(repair_fn(c1))
179             c2 = self._clip(repair_fn(c2))
180         new_pop.append(c1)
181         if len(new_pop) < self.pop_size:
182             new_pop.append(c2)
183
184     pop = np.vstack(new_pop)
185     scores = np.array([fitness_fn(ind) for ind in pop], dtype=float)
186
187     # final refine (optional)
188     if reeval_k > 0 and reeval_fn is not None:
189         if self.objective == "max":
190             order_tmp = np.argsort(scores)[::-1]
191         else:
192             order_tmp = np.argsort(scores)
193         top_idx = order_tmp[:reeval_k]
194         for i in top_idx:
195             s2 = reeval_fn(pop[i, :])

```

```

193         if self._is_better(s2, scores[i]):
194             scores[i] = s2
195
196         # final best
197         final_idx = int(np.argmax(scores) if self.objective == "max" else np.argmin(scores))
198         if self._is_better(scores[final_idx], best_s):
199             best_x = pop[final_idx].copy()
200             best_s = float(scores[final_idx])
201
202         return best_x, best_s

```

```

1  import numpy as np
2
3
4  g = 9.81          # 重力加速度
5  v_missile = 300.0 # 导弹恒速
6  v_cloud = 3.0     # 云团下沉速度
7  v_min = 70.0      # 无人机最低速度
8  v_max = 140.0     # 无人机最高速度
9  R_effective = 10.0 # 有效遮蔽半径
10 t_effective = 20.0 # 有效遮蔽时间窗长度
11 interval = 1.0    # 最短投弹时间间隔
12
13 M1_0 = np.array([20000.0, 0.0, 2000.0]) # 导弹 M1
14 M2_0 = np.array([19000.0, 600.0, 2100.0]) # 导弹 M2
15 M3_0 = np.array([18000.0, -600.0, 1900.0]) # 导弹 M3
16 FY1_0 = np.array([17800.0, 0.0, 1800.0]) # 无人机 FY1
17 FY2_0 = np.array([12000.0, 1400.0, 1400.0]) # 无人机 FY2
18 FY3_0 = np.array([6000.0, -3000.0, 700.0]) # 无人机 FY3
19 FY4_0 = np.array([11000.0, 2000.0, 1800.0]) # 无人机 FY4
20 FY5_0 = np.array([13000.0, -2000.0, 1300.0]) # 无人机 FY5
21 fake = np.array([0.0, 0.0, 0.0])          # 假目标
22 real = np.array([0.0, 200.0, 0.0])        # 真目标圆柱底面中心
23
24 R = 7.0          # 圆柱半径
25 H = 10.0         # 圆柱高度
26
27 T_end = float(np.linalg.norm(fake - M1_0) / v_missile) # 模拟终止时间（导弹到达假目标时刻）

```

以下是问题一的代码：

```

1  from __future__ import annotations
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  import params as p

```



```

6 from evaluator import cover_intervals_cached_general, munition_pos_general,
   missile_pos_general, cloud_pos
7 from funcs import unit, set_axes_equal_3d
8
9
10 # FY1的特化函数
11 def munition_pos(v: float, dir_xy: np.ndarray, t_drop: float, t: float) -> np.ndarray:
12     return munition_pos_general(p.FY1_0, v, dir_xy, t_drop, t)
13
14
15 def cover_intervals_FY1(dt: float, v: float, direction: np.ndarray,
16                         t_drop: float, t_burst: float, *, stage: int = 1, precheck: bool = False):
17     dir_xy = unit(np.array([direction[0], direction[1], 0.0], float))
18     return cover_intervals_cached_general(
19         dt, v, dir_xy, t_drop, t_burst, p.FY1_0, p.M1_0, stage=stage, precheck=precheck
20     )
21
22
23 def total_cover_length_FY1(dt: float, v: float, direction: np.ndarray,
24                           t_drop: float, t_burst: float, *, stage: int = 1, precheck: bool =
25                               False) -> float:
26     intervals, total = cover_intervals_FY1(dt, v, direction, t_drop, t_burst, stage=stage,
27                                             precheck=precheck)
28     return float(total)
29
30 def plot_problem1_3d(v_drone: float, direction: np.ndarray,
31                    t_drop: float, delay: float, dt: float = 0.05) -> None:
32     """
33     画出从 t=0 到 t_burst 的 3D 轨迹，并标注云团（球体半径 = p.R_effective）。
34     direction: 任意 3D 向量，仅取 XY 分量（自动单位化）
35     save_path: 保存图片路径（None 则不保存）
36     """
37     # --- 基本时间轴 ---
38     t_burst = float(t_drop + delay)
39     t_vec0 = np.arange(0.0, t_burst + 1e-12, max(1e-3, dt))
40     t_vec_bomb = np.arange(t_drop, t_burst + 1e-12, max(1e-3, dt))
41
42     dir_xy = unit(np.array([direction[0], direction[1], 0.0], float)) # 只取平面分量
43
44     # --- 轨迹计算 ---
45     # 1) 无人机：等速直线到 t_burst
46     FY_line = p.FY1_0[None, :3] + (v_drone * t_vec0[:, None]) * dir_xy[None, :]
47     FY_line[:, 2] = p.FY1_0[2]
48
49     # 2) 干扰弹：从 t_drop 到 t_burst 的抛物线
50     bomb_traj = np.vstack([

```

```

50     munition_pos_general(p.FY1_0, v_drone, dir_xy, t_drop, t)
51     for t in t_vec_bomb
52 ]))
53
54 # 3) 导弹 M1: 从起始到 t_burst
55 mis_traj = np.vstack([
56     missile_pos_general(p.M1_0, t)
57     for t in t_vec0
58 ])
59
60 # 起爆点与云团
61 P_burst = munition_pos_general(p.FY1_0, v_drone, dir_xy, t_drop, t_burst)
62 R_cloud = float(p.R_effective)
63
64 # 作图
65 fig = plt.figure(figsize=(8, 7))
66 ax = fig.add_subplot(111, projection='3d')
67
68 # 无人机轨迹 (蓝)
69 ax.plot(FY_line[:, 0], FY_line[:, 1], FY_line[:, 2], linewidth=2.0, color='blue',
70         label="UAV path (0→t_burst)")
71
72 # 干扰弹轨迹 (橙)
73 ax.plot(bomb_traj[:, 0], bomb_traj[:, 1], bomb_traj[:, 2], linewidth=2.0, color='orange',
74         label="Bomb path (t_drop→t_burst)")
75
76 # 导弹轨迹 (绿)
77 ax.plot(mis_traj[:, 0], mis_traj[:, 1], mis_traj[:, 2], linewidth=2.0, color='green',
78         label="Missile path (0→t_burst)")
79
80 # 起爆点
81 ax.scatter([P_burst[0]], [P_burst[1]], [P_burst[2]], s=60, marker='o', color='orange',
82           label="Burst point")
83
84 # 云团球体 (橙色 + 半透明)
85 u = np.linspace(0, 2*np.pi, 40)
86 v = np.linspace(0, np.pi, 20)
87 xs = P_burst[0] + R_cloud * np.outer(np.cos(u), np.sin(v))
88 ys = P_burst[1] + R_cloud * np.outer(np.sin(u), np.sin(v))
89 zs = P_burst[2] + R_cloud * np.outer(np.ones_like(u), np.cos(v))
90 ax.plot_surface(xs, ys, zs, color="orange", alpha=0.25, edgecolor="none")
91
92 # 轴/标题
93 ax.set_xlabel("X (m)")
94 ax.set_ylabel("Y (m)")
95 ax.set_zlabel("Z (m)")
96 ax.set_title("Problem 1 - Scene Overview")

```

```

93     ax.legend(loc="best")
94
95     # 视窗与等比例
96     xs_all = np.concatenate([FY_line[:,0], bomb_traj[:,0], mis_traj[:,0], [P_burst[0]]])
97     ys_all = np.concatenate([FY_line[:,1], bomb_traj[:,1], mis_traj[:,1], [P_burst[1]]])
98     zs_all = np.concatenate([FY_line[:,2], bomb_traj[:,2], mis_traj[:,2], [P_burst[2]]])
99     ax.set_xlim(xs_all.min()-50, xs_all.max()+50)
100    ax.set_ylim(ys_all.min()-50, ys_all.max()+50)
101    ax.set_zlim(max(0.0, zs_all.min()-50), zs_all.max()+50)
102    set_axes_equal_3d(ax)
103
104    plt.tight_layout()
105
106    plt.savefig("./figures/problem1_3d.png", dpi=220, bbox_inches="tight")
107
108    plt.show()
109    plt.close(fig)
110
111
112    if __name__ == "__main__":
113
114        v_drone = 120.0
115        t_drop = 1.5
116        delay = 3.6
117        t_burst = t_drop + delay
118
119        # 朝向“假目标”的单位方向（仅用 XY 分量）
120        direction = unit(np.array([p.fake[0]-p.FY1_0[0], p.fake[1]-p.FY1_0[1], 0.0], float))
121
122        # 保留原有的数值评估/打印（如不需要可注释）
123        from evaluator import cover_intervals_cached_general
124        ints, total = cover_intervals_cached_general(
125            dt=0.01, v_drone=v_drone, direction=direction,
126            t_drop=t_drop, t_burst=t_burst, FY0=p.FY1_0, M0=p.M1_0, stage=1, precheck=False
127        )
128        P_burst = munition_pos_general(p.FY1_0, v_drone, direction, t_drop, t_burst)
129        print("Burst point (m): x=%.3f, y=%.3f, z=%.3f" % (P_burst[0], P_burst[1], P_burst[2]))
130        print("[Total] dur=%.3f intervals=%s" % (total, ints))
131
132        # 画 3D 轨迹图
133        plot_problem1_3d(v_drone, direction, t_drop, delay, dt=0.05)

```

以下是问题二的代码：

```

1  from __future__ import annotations
2  import numpy as np
3  import math, time

```

```

4 import matplotlib.pyplot as plt
5
6 import params as p
7 import problem1 as P1
8 from ga import GeneticAlgorithm
9 from evaluator import missile_pos_general, munition_pos_general, cover_intervals_cached_general
10 from funcs import set_axes_equal_3d
11
12
13 # 基因: [v, theta(rad), t_drop, delay]
14 LB = np.array([p.v_min, -math.pi, 0.0, 0.0], dtype=float)
15 UB = np.array([p.v_max, math.pi, 30.0, p.t_effective], dtype=float)
16
17
18 def decode(x: np.ndarray):
19     """
20     解码基因为可读参数
21     """
22     v = float(x[0])
23     theta = float(x[1])
24     dir_xy = np.array([math.cos(theta), math.sin(theta), 0.0], dtype=float)
25     n = np.linalg.norm(dir_xy); dir_xy = dir_xy / n if n > 0 else np.array([1.0, 0.0, 0.0])
26     t_drop = float(x[2]); delay = float(x[3]); t_burst = t_drop + delay
27     T_end = getattr(p, "T_end", float("inf"))
28     if t_burst > T_end:
29         delay = max(0.0, T_end - t_drop)
30         t_burst = t_drop + delay
31     return v, dir_xy, t_drop, delay, t_burst
32
33
34 def eval_total(x: np.ndarray, dt: float, stage: int, *, precheck: bool) -> float:
35     v, dir_xy, t_drop, delay, t_burst = decode(x)
36     return P1.total_cover_length_FY1(dt, v, dir_xy, t_drop, t_burst, stage=stage,
37                                     precheck=precheck)
38
39
40 def plot_problem2_3d(FY0, v_drone, dir_xy, t_drop, delay, save_path, dt=0.05):
41     """
42     绘制三维场景图
43     """
44     # 时间轴
45     t_burst = float(t_drop + delay)
46     t_uav_mis = np.arange(0.0, t_burst + 1e-12, max(1e-3, dt))
47     t_bomb = np.arange(t_drop, t_burst + 1e-12, max(1e-3, dt))
48
49     # UAV: 等速直线 (仅 XY, Z 恒定)

```

```

50 FY_line = FY0[None, :3] + (v_drone * t_uav_mis[:, None]) * dir_xy[None, :]
51 FY_line[:, 2] = FY0[2]
52
53 # Bomb: 抛物线
54 bomb_traj = np.vstack([
55     munition_pos_general(FY0, v_drone, dir_xy, t_drop, t) for t in t_bomb
56 ])
57
58 # Missile: 直线
59 mis1 = np.vstack([
60     missile_pos_general(p.M1_0, t) for t in t_uav_mis
61 ])
62
63 # 起爆点 & 云团
64 P_burst = munition_pos_general(FY0, v_drone, dir_xy, t_drop, t_burst)
65 R_cloud = float(getattr(p, "R_effective", 10.0))
66
67 # 作图
68 fig = plt.figure(figsize=(9, 7))
69 ax = fig.add_subplot(111, projection="3d")
70
71 # 轨迹
72 ax.plot(mis1[:,0], mis1[:,1], mis1[:,2], color="green", linewidth=2.0, label="Missile
73         path(0->t_burst)")
74 ax.plot(FY_line[:,0], FY_line[:,1], FY_line[:,2], color="blue", linewidth=2.0, label="UAV
75         path(0->t_burst)")
76 ax.plot(bomb_traj[:,0], bomb_traj[:,1], bomb_traj[:,2], color="orange", linewidth=2.0,
77         label="Bomb path(t_drop->t_burst)")
78
79 # 起爆点
80 ax.scatter([P_burst[0]], [P_burst[1]], [P_burst[2]], s=60, marker='o', color="orange",
81         label="Burst point")
82
83 # 云团 (橙色半透明)
84 u = np.linspace(0, 2*np.pi, 40); v = np.linspace(0, np.pi, 20)
85 xs = P_burst[0] + R_cloud * np.outer(np.cos(u), np.sin(v))
86 ys = P_burst[1] + R_cloud * np.outer(np.sin(u), np.sin(v))
87 zs = P_burst[2] + R_cloud * np.outer(np.ones_like(u), np.cos(v))
88 ax.plot_surface(xs, ys, zs, color="orange", alpha=0.25, edgecolor="none")
89
90 # 轴/标题
91 ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)"); ax.set_zlabel("Z (m)")
92 ax.set_title("Problem 2 — Scene Overview")
93 ax.legend(loc="best")
94
95 # 视窗范围 (只基于三条轨迹和起爆点)
96 xs_all = np.concatenate([mis1[:,0], FY_line[:,0], bomb_traj[:,0], [P_burst[0]]])

```

```

93 ys_all = np.concatenate([mis1[:,1], FY_line[:,1], bomb_traj[:,1], [P_burst[1]]])
94 zs_all = np.concatenate([mis1[:,2], FY_line[:,2], bomb_traj[:,2], [P_burst[2]]])
95 ax.set_xlim(xs_all.min()-50, xs_all.max()+50)
96 ax.set_ylim(ys_all.min()-50, ys_all.max()+50)
97 ax.set_zlim(max(0.0, zs_all.min()-50), zs_all.max()+50)
98 set_axes_equal_3d(ax)
99
100 plt.tight_layout()
101 plt.savefig(save_path, dpi=220, bbox_inches="tight")
102 plt.show()
103 plt.close(fig)
104
105
106 def run_problem2(cfg):
107
108     ga = GeneticAlgorithm(
109         lb=LB, ub=UB,
110         pop_size=cfg["pop"], n_gen=cfg["gen"], seed=cfg["seed"],
111         elite_frac=cfg["elite_frac"], tournament_k=3,
112         sbx_eta=12.0, sbx_prob=0.9, mut_rate=0.20, mut_scale=0.10,
113         objective="max",
114     )
115
116     # 退火变异幅度
117     def sigma_fn(gen: int, n_gen: int, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
118         anneal = max(0.45, 1.0 - gen / n_gen)
119         return np.array([
120             (p.v_max - p.v_min)*0.05, # v
121             0.06*math.pi,             # theta
122             0.6*anneal,                 # t_drop
123             0.5*anneal,                 # delay
124         ], dtype=float)
125
126     fitness_fn = lambda x: eval_total(x, cfg["ga_dt"], stage=0, precheck=True)
127     reeval_fn = lambda x: eval_total(x, cfg["final_dt"], stage=1, precheck=False)
128
129     hist_gen, hist_best, hist_mean = [], [], []
130
131     def stats_hook(gen: int, best_s: float, mean_s: float):
132         hist_gen.append(int(gen))
133         hist_best.append(float(best_s))
134         hist_mean.append(float(mean_s))
135
136     def log_fn(gen: int, best_score: float, best_x: np.ndarray):
137         if gen % 4 == 0:
138             print(f"Gen {gen:02d}: best {best_score:.3f}s")
139

```

```

140 seeds = [
141     np.array([80.0, math.radians( 5.0), 0.60, 1.00], float),
142     np.array([75.0, math.radians( 10.0), 1.20, 0.80], float),
143     np.array([86.0, math.radians(-40.0), 0.40, 1.20], float),
144     np.array([92.0, math.radians( 25.0), 1.60, 0.60], float),
145 ]
146
147 t0 = time.time()
148 x_best, _ = ga.evolve(
149     fitness_fn=fitness_fn,
150     reeval_fn=reeval_fn, reeval_frac=cfg["reeval_frac"],
151     verbose_every=4, log_fn=log_fn,
152     seed_population=seeds, sigma_fn=sigma_fn,
153     # 关键: 把"均值"也回调出来
154     stats_hook=stats_hook,
155 )
156 t1 = time.time()
157
158 length = eval_total(x_best, cfg["final_dt"], stage=1, precheck=False)
159 v, dir_xy, t_drop, delay, t_burst = decode(x_best)
160 theta_deg = math.degrees(math.atan2(dir_xy[1], dir_xy[0]))
161 try:
162     burst_pos = P1.munition_pos(v, dir_xy, t_drop, t_burst)
163 except Exception:
164     burst_pos = (float("nan"),)*3
165
166 print("\n=== Problem 2 — GA Best Plan (final, total-length objective) ===")
167 print(f"Total cover length: {length:.3f} s")
168 print(f"Speed v = {v:.3f} m/s, Heading = {theta_deg:.3f}°")
169 print(f"Drop time: {t_drop:.3f} s, Delay: {delay:.3f} s, Burst: {t_burst:.3f} s")
170 print(f"Burst pos: ({burst_pos[0]:.3f}, {burst_pos[1]:.3f}, {burst_pos[2]:.3f}) m")
171 print("Elapsed: %.2f s" % (t1 - t0))
172
173 intervals, total_length = cover_intervals_cached_general(
174     dt=cfg["final_dt"],
175     v_drone=v,
176     direction=dir_xy,
177     t_drop=t_drop,
178     t_burst=t_burst,
179     FY0=p.FY1_0,
180     M0=p.M1_0,
181     stage=1,
182     precheck=False
183 )
184
185 if intervals:
186     print("\nEffective cover intervals:")

```

```

187     for i, (start, end) in enumerate(intervals, 1):
188         print(f" Interval {i}: {start:.3f} s to {end:.3f} s (duration: {end-start:.3f} s)")
189     else:
190         print("\nNo effective cover intervals found.")
191
192     # 可视化
193     fig, ax = plt.subplots(1, 1, figsize=(8, 5))
194     ax.plot(hist_gen, hist_best, 'r-', linewidth=1.2, label="Best")
195     ax.plot(hist_gen, hist_mean, 'b-', linewidth=1.2, label="Population mean")
196     ax.set_xlabel("Generation")
197     ax.set_ylabel("Total cover length (s)")
198     ax.set_title("GA progress (Problem 2)")
199     ax.grid(True, linestyle='--', alpha=0.6)
200     ax.legend()
201     fig.tight_layout()
202     fig.savefig("../figures/problem2_ga_progress.png", dpi=180, bbox_inches="tight")
203     plt.close(fig)
204
205     plot_problem2_3d(
206         FY0=p.FY1_0,
207         v_drone=v,
208         dir_xy=dir_xy,
209         t_drop=t_drop,
210         delay=delay,
211         save_path="../figures/problem2_3d.png",
212         dt=0.05
213     )
214
215
216     # 遗传算法配置
217     CFG = dict(
218         pop=48, gen=36, seed=202509,
219         elite_frac=0.16, reeval_frac=0.22,
220         ga_dt=0.06, final_dt=0.01
221     )
222
223
224     if __name__ == "__main__":
225
226         run_problem2(CFG)

```

以下是问题三的代码：

```

1 import numpy as np, math, time, random
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 from funcs import set_axes_equal_3d

```



```

5
6
7 # 场景参数
8 FY1_0 = np.array([17800.0, 0.0, 1800.0])
9 g = 9.8
10 R_cyl, H_cyl = 7.0, 10.0
11 Y_axis = 200.0
12 R_effective = 10.0
13 SINK = 3.0
14 CLOUD_LIFETIME = 20.0
15 vM = 300.0
16 M0 = np.array([20000.0, 0.0, 2000.0])
17 O = np.array([0.0, 0.0, 0.0])
18 uM = (O - M0) / np.linalg.norm(O - M0)
19 t_arrival = float(np.linalg.norm(O - M0) / vM)
20
21 def M(t):
22     return M0 + uM * (vM * t)
23
24
25 # 圆柱闭合表面采样（侧面+上下底）
26 def sample_cylinder(n_theta, n_z, cap_radial_steps):
27
28     pts=[]
29     thetas=np.linspace(0, 2*np.pi, n_theta, endpoint=False)
30     zs=np.linspace(0.0, H_cyl, n_z)
31
32     # 侧面
33     for z in zs:
34         for th in thetas:
35             pts.append([R_cyl*math.cos(th), Y_axis + R_cyl*math.sin(th), z])
36
37     # 上下底（圆盘内部）
38     for z in [0.0, H_cyl]:
39         for r in np.linspace(0, R_cyl, cap_radial_steps+1):
40             for th in thetas:
41                 pts.append([r*math.cos(th), Y_axis + r*math.sin(th), z])
42     return np.asarray(pts, float)
43
44
45 # 评估档位: GA 粗评 / 最终高精
46 P_fit = sample_cylinder(36, 9, 5) # 更稀, 配合大 dt_fit, 加速
47 P_ref = sample_cylinder(160, 25, 8) # 高精用于精英复评 & 最终
48
49
50 def seg_point_min_dist(c, m, P):
51     pm=P-m[None,:]
```

```

52     d2=np.einsum("ij,ij->i", pm, pm); d2=np.where(d2==0.0,1e-12,d2)
53     s =np.einsum("j,ij->i", c-m, pm)/d2; s=np.clip(s,0.0,1.0)
54     closest = m[None,:] + pm*s[:,None]
55     return np.linalg.norm(c[None,:]-closest,axis=1)
56
57 def fully_occluded_strict(t, P_burst, t_burst, P, eps=1e-6):
58     if not (t_burst <= t <= t_burst + CLOUD_LIFETIME):
59         return False
60     m = M(t)
61     c = P_burst + np.array([0.0,0.0,-SINK*(t - t_burst)])
62     return np.max(seg_point_min_dist(c, m, P) - R_effective) <= eps
63
64 def intervals_strict(P_burst, t_burst, P, dt=0.02, tmax=12.0, eps=1e-6):
65     t0, t1 = t_burst, min(t_burst + CLOUD_LIFETIME, t_burst + tmax, t_arrival)
66     if t1 <= t0:
67         return [], 0.0
68     times = np.arange(t0, t1 + 1e-12, max(dt, 1e-6))
69     flags=[ fully_occluded_strict(t, P_burst, t_burst, P, eps=eps) for t in times ]
70     intervals=[]; in_seg=False; start=None
71     for i, ok in enumerate(flags):
72         if ok and not in_seg:
73             in_seg=True; start=times[i]
74         elif not ok and in_seg:
75             intervals.append((start, times[i-1])); in_seg=False
76     if in_seg:
77         intervals.append((start, times[-1]))
78     return intervals, sum(b-a for a,b in intervals)
79
80
81 USE_QUICK = True
82 T_b = np.array([0.0, 200.0, H_cyl/2.0])
83 r_b = math.sqrt(R_cyl**2 + (H_cyl/2.0)**2)
84 def quick_sufficient_cover(Mt, Ct):
85     v1, v2 = T_b - Mt, Ct - Mt
86     d1, d2 = np.linalg.norm(v1), np.linalg.norm(v2)
87     if d1 <= r_b: return False
88     if d2 <= R_effective: return True
89     alpha = math.asin(min(1.0, r_b/d1))
90     beta = math.asin(min(1.0, R_effective/d2))
91     cosd = float(np.dot(v1, v2)/(d1*d2)); cosd = max(-1.0, min(1.0, cosd))
92     delta = math.acos(cosd)
93     return beta >= alpha + delta
94
95 def quick_has_any(P_burst, t_burst, dt=0.12):
96     t0, t1 = t_burst, min(t_burst + CLOUD_LIFETIME, t_arrival)
97     for t in np.arange(t0, t1 + 1e-12, max(dt,1e-6)):
98         Ct = P_burst + np.array([0,0,-SINK*(t - t_burst)])

```

```

99         if quick_sufficient_cover(M(t), Ct):
100             return True
101         return False
102
103
104 def uav_pos(FY0, t, v, heading_deg):
105     rad=math.radians(heading_deg)
106     return FY0 + np.array([math.cos(rad), math.sin(rad), 0.0])*v*t
107
108 def detonation_point(FY0, t_burst, v, heading_deg, t_drop):
109     xy = uav_pos(FY0, t_burst, v, heading_deg)[:2]
110     if t_drop > t_burst:
111         z = FY0[2]
112     else:
113         z = FY0[2] - 0.5*g*(t_burst - t_drop)**2
114     return np.array([xy[0], xy[1], z])
115
116
117 def drop_point(FY0, t_drop, v, heading_deg):
118     xy = uav_pos(FY0, t_drop, v, heading_deg)[:2]
119     return np.array([xy[0], xy[1], FY0[2]], float)
120
121
122 def union_length(intervals):
123     if not intervals: return 0.0, []
124     intervals = sorted(intervals)
125     merged=[]; cs,ce=intervals[0]
126     for s,e in intervals[1:]:
127         if s<=ce: ce=max(ce,e)
128         else: merged.append((cs,ce)); cs,ce=s,e
129     merged.append((cs,ce))
130     return sum(e-s for s,e in merged), merged
131
132
133 # 染色体: [theta_deg, v, d1, tau1, d2, tau2, d3, tau3]
134 def wrap_deg(th):
135     return th % 360.0
136
137 def clamp(x, lo, hi):
138     return max(lo, min(hi, x))
139
140 def decode_intervals(ch):
141     theta, v, d1, tau1, d2, tau2, d3, tau3 = ch
142     tr1 = max(0.0, d1)
143     tr2 = tr1 + 1.0 + max(0.0, d2)
144     tr3 = tr2 + 1.0 + max(0.0, d3)
145     tau1 = clamp(tau1, 0.0, 6.0)

```

```

146     tau2 = clamp(tau2, 0.0, 6.0)
147     tau3 = clamp(tau3, 0.0, 6.0)
148     return theta, v, tr1, tau1, tr2, tau2, tr3, tau3
149
150 def repair_gene(ch):
151     theta, v, tr1, tau1, tr2, tau2, tr3, tau3 = decode_intervals(ch)
152     theta = wrap_deg(theta)
153     v = clamp(v, 70.0, 140.0)
154     tdet1 = min(tr1 + tau1, t_arrival)
155     tdet2 = min(tr2 + tau2, t_arrival)
156     tdet3 = min(tr3 + tau3, t_arrival)
157     tau1 = max(0.0, tdet1 - tr1)
158     tau2 = max(0.0, tdet2 - tr2)
159     tau3 = max(0.0, tdet3 - tr3)
160     return [theta, v, tr1, tau1, tr2, tau2, tr3, tau3]
161
162
163 def eval_three(theta, v, tr1, tau1, tr2, tau2, tr3, tau3, P, dt=0.03, use_quick=True):
164     """
165     返回：三团“所有片段”的并集覆盖时间（严格判据）
166     """
167     inters_all=[]; per=[]
168     for tr, tau in [(tr1,tau1),(tr2,tau2),(tr3,tau3)]:
169         tdet = tr + tau
170         P_burst = detonation_point(FY1_0, tdet, v, theta, tr)
171         if P_burst[2] < 0.0 or tdet > t_arrival:
172             inters, L = [], 0.0
173         else:
174             if use_quick and (not quick_has_any(P_burst, tdet, dt=0.12)):
175                 inters, L = [], 0.0
176             else:
177                 inters, L = intervals_strict(P_burst, tdet, P, dt=dt, tmax=12.0)
178             per.append((inters, L, tdet, P_burst))
179             inters_all += inters
180     L_union, merged = union_length(inters_all)
181     return L_union, merged, per
182
183 def fitness(ch, P, dt_fit=0.04, use_quick=True):
184     theta, v, tr1, tau1, tr2, tau2, tr3, tau3 = repair_gene(ch)
185     L_union, merged, per = eval_three(theta, v, tr1, tau1, tr2, tau2, tr3, tau3, P, dt=dt_fit,
186                                     use_quick=use_quick)
187     return L_union, merged, per, [theta, v, tr1, tau1, tr2, tau2, tr3, tau3]
188
189 def tournament_select(pop, fits, k=3):
190     n=len(pop); idxs=np.random.randint(0, n, size=k)
191     best_i = max(idxs, key=lambda i: fits[i])
192     return pop[best_i].copy()

```

```

192
193 def sbx_crossover(p1, p2, eta=10.0, prob=0.9):
194     if random.random() > prob: return p1.copy(), p2.copy()
195     c1, c2 = p1.copy(), p2.copy()
196     for j in range(len(p1)):
197         u = random.random()
198         if u <= 0.5: beta = (2*u)**(1.0/(eta+1.0))
199         else:      beta = (1/(2*(1-u)))** (1.0/(eta+1.0))
200         c1[j] = 0.5*((1+beta)*p1[j] + (1-beta)*p2[j])
201         c2[j] = 0.5*((1-beta)*p1[j] + (1+beta)*p2[j])
202     return c1, c2
203
204 def gaussian_mutation(c, sigmas, prob=0.25):
205     for j in range(len(c)):
206         if random.random() < prob:
207             c[j] += random.gauss(0.0, sigmas[j])
208     return c
209
210
211 def plot_problem3_3d(theta_deg, v, bombs, *, dt=0.05, save_path):
212     """
213     只绘制: Missile1(0→t_max)、UAV(0→t_max)、三颗干扰弹(t_drop→t_burst) 的 3D 轨迹;
214     三个起爆点分别绘制橙色半透明云团 (半径 = R_effective) 。
215     - 导弹: 绿色
216     - 无人机: 蓝色
217     - 三颗干扰弹: 橙色
218     - 云团: 橙色 + alpha=0.25
219     bombs: [(t_drop1, tau1), (t_drop2, tau2), (t_drop3, tau3)]
220     """
221
222     # 统一时间轴: 到三枚弹的最晚起爆时刻
223     t_bursts = [tr+tau for (tr, tau) in bombs]
224     t_max = max(1e-6, max(t_bursts))
225     t_uav_mis = np.arange(0.0, t_max + 1e-12, max(1e-3, dt))
226
227     # UAV 航向向量 (由航向角度)
228     rad = math.radians(float(theta_deg))
229     dir_xy = np.array([math.cos(rad), math.sin(rad), 0.0], float)
230
231     # UAV: 等速直线
232     uav = FY1_0[None, :3] + (v * t_uav_mis[:, None]) * dir_xy[None, :]
233     uav[:, 2] = FY1_0[2]
234
235     # Missile1: 直线 (指向原点)
236     mis = np.vstack([M(t) for t in t_uav_mis])
237
238     # Bombs: 每枚弹从 t_drop 到 t_burst 的抛物线 (XY 随 UAV 匀速, Z 做自由落体)

```

```

239 bomb_trajs = []
240 burst_points = []
241 for (tr, tau) in bombs:
242     t_b = float(tr + tau)
243     if t_b <= 0.0: # 不可行的直接跳过
244         bomb_trajs.append(None)
245         burst_points.append(None)
246         continue
247     t_bomb = np.arange(tr, t_b + 1e-12, max(1e-3, dt))
248     xy = FY1_0[:2][None, :] + (v * t_bomb[:, None]) * dir_xy[None, :2]
249     z = FY1_0[2] - 0.5 * g * (t_bomb - tr)**2
250     bomb_trajs.append(np.column_stack([xy[:,0], xy[:,1], z]))
251     burst_points.append(np.array([xy[-1,0], xy[-1,1], z[-1]], float))
252
253 # 绘图
254 fig = plt.figure(figsize=(9, 7))
255 ax = fig.add_subplot(111, projection="3d")
256
257 # 轨迹（颜色规范）
258 ax.plot(mis[:,0], mis[:,1], mis[:,2], color="green", linewidth=2.0, label="Missile
    path(0->t_burst)")
259 ax.plot(uav[:,0], uav[:,1], uav[:,2], color="blue", linewidth=2.0, label="UAV
    path(0->t_burst)")
260
261 # 修改 bomb 轨迹的标签设置
262 for i, traj in enumerate(bomb_trajs, 1):
263     if traj is None: continue
264     ax.plot(traj[:,0], traj[:,1], traj[:,2], color="orange", linewidth=2.0, label="Bomb
    path" if i==1 else None) # 所有 bomb 使用同一个标签
265
266 # 修改起爆点的标签设置
267 for i, P_burst in enumerate(burst_points, 1):
268     if P_burst is None: continue
269     ax.scatter([P_burst[0]], [P_burst[1]], [P_burst[2]], s=60, marker='o', color="orange",
    label="Burst point" if i==1 else None) # 所有起爆点使用同一个标签
270
271 # 轴与视窗
272 ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)"); ax.set_zlabel("Z (m)")
273 ax.set_title("Problem 3 — Scene Overview")
274 ax.legend(loc="best")
275
276 # 自适应范围
277 xs = [mis[:,0], uav[:,0]] + [t[:,0] for t in bomb_trajs if t is not None] +
    [np.array([p[0]]) for p in burst_points if p is not None]
278 ys = [mis[:,1], uav[:,1]] + [t[:,1] for t in bomb_trajs if t is not None] +
    [np.array([p[1]]) for p in burst_points if p is not None]
279 zs = [mis[:,2], uav[:,2]] + [t[:,2] for t in bomb_trajs if t is not None] +

```

```

280     [np.array([p[2]]) for p in burst_points if p is not None]
281     xs_all = np.concatenate(xs); ys_all = np.concatenate(ys); zs_all = np.concatenate(zs)
282     ax.set_xlim(xs_all.min()-50, xs_all.max()+50)
283     ax.set_ylim(ys_all.min()-50, ys_all.max()+50)
284     ax.set_zlim(max(0.0, zs_all.min()-50), zs_all.max()+50)
285     set_axes_equal_3d(ax)
286
287     plt.tight_layout()
288     plt.savefig(save_path, dpi=220, bbox_inches="tight")
289     plt.show()
290     plt.close(fig)
291
292 def run_problem3(seed=314159, pop_size=90, n_gen=40,
293                 elite_frac=0.12, elite_eval_frac=0.20,
294                 dt_fit=0.04, dt_ref=0.01, use_quick=USE_QUICK):
295
296     np.random.seed(seed); random.seed(seed)
297
298     # 初始种群 (含启发式解)
299     seeds = []
300     seeds.append([5.0, 70.0, 0.20, 1.20, 1.00, 0.00, 1.00, 0.20])
301     seeds.append([3.8, 80.0, 0.60, 1.00, 1.00, 0.00, 1.20, 0.20])
302     seeds.append([300.0, 85.0, 0.50, 0.40, 1.50, 0.30, 1.50, 0.60])
303     seeds.append([25.0, 90.0, 1.20, 0.20, 0.80, 0.40, 1.20, 0.30])
304
305     pop=[s.copy() for s in seeds]
306     while len(pop) < pop_size:
307         theta = np.random.uniform(0.0, 360.0)
308         v = np.random.uniform(70.0, 140.0)
309         d1 = np.random.uniform(0.0, 3.0)
310         d2 = np.random.uniform(0.0, 2.0)
311         d3 = np.random.uniform(0.0, 2.0)
312         tau1 = np.random.uniform(0.0, 1.6)
313         tau2 = np.random.uniform(0.0, 1.6)
314         tau3 = np.random.uniform(0.0, 1.6)
315         pop.append([theta, v, d1, tau1, d2, tau2, d3, tau3])
316
317     best=None; best_pack=None
318     elite_k = max(2, int(pop_size*elite_frac))
319
320     hist_gen, hist_best, hist_mean = [], [], []
321
322     for gen in range(1, n_gen+1):
323         anneal = max(0.3, 1.0 - gen/n_gen)
324         sigmas = [ 360.0*0.05, (140-70)*0.05, 0.5*anneal, 0.3*anneal,
325                   0.6*anneal, 0.3*anneal, 0.8*anneal, 0.3*anneal ]

```

```

326
327     fits=[]; infos=[]
328     for ind in pop:
329         L, merged, per, repaired = fitness(ind, P_fit, dt_fit=dt_fit, use_quick=use_quick)
330         fits.append(L); infos.append((merged, per, repaired))
331
332     order_tmp = np.argsort(fits)[::-1]
333     top_eval = order_tmp[:max(2, int(pop_size*elite_eval_frac))]
334     for i in top_eval:
335         rep_ch = infos[i][2]
336         theta, v, tr1, tau1, tr2, tau2, tr3, tau3 = rep_ch
337         L2, merged2, per2 = eval_three(theta, v, tr1, tau1, tr2, tau2, tr3, tau3,
338                                     P_ref, dt=0.015, use_quick=False)
339         if L2 > fits[i] + 1e-6:
340             fits[i] = L2
341             infos[i] = (merged2, per2, rep_ch)
342
343     g_best = int(np.argmax(fits))
344     if (best is None) or (fits[g_best] > best + 1e-9):
345         best = fits[g_best]; best_pack = (pop[g_best].copy(), infos[g_best])
346
347     hist_gen.append(gen)
348     hist_best.append(float(fits[g_best]))
349     hist_mean.append(float(np.mean(fits)))
350
351     print(f"Gen {gen:02d}: best {fits[g_best]:.3f} s, avg={np.mean(fits):.3f} s")
352
353     order = np.argsort(fits)[::-1]
354     new_pop = [ pop[i].copy() for i in order[:elite_k] ]
355     while len(new_pop) < pop_size:
356         p1 = tournament_select(pop, fits, k=3)
357         p2 = tournament_select(pop, fits, k=3)
358         c1, c2 = sbx_crossover(p1, p2, eta=12.0, prob=0.9)
359         c1 = gaussian_mutation(c1, sigmas, prob=0.25)
360         c2 = gaussian_mutation(c2, sigmas, prob=0.25)
361         new_pop.append(c1)
362         if len(new_pop) < pop_size: new_pop.append(c2)
363     pop = new_pop
364
365     chrom_best, (merged_fit, per_fit, repaired) = best_pack
366     theta, v, tr1, tau1, tr2, tau2, tr3, tau3 = repaired
367
368     details=[]; inters_all=[]
369     bombs = [(tr1,tau1),(tr2,tau2),(tr3,tau3)]
370     for (tr,tau) in bombs:
371         td = tr + tau
372         P_burst = detonation_point(FY1_0, td, v, theta, tr)

```



```

373     inters, L = intervals_strict(P_burst, td, P_ref, dt=dt_ref, tmax=12.0)
374     details.append((tr, td, tau, P_burst, L, inters))
375     inters_all += inters
376 L_union_ref, merged_ref = union_length(inters_all)
377
378 print("\n=== 遗传算法 · 问题3 · 最优解（高精评估） ===")
379 print(f"三团并集（严格）= {L_union_ref:.3f} s")
380 print("并集区间 =", merged_ref)
381 print(f"航向={theta:.2f}°, 速度={v:.1f} m/s")
382 for k, (tr, td, tau, P_burst, L, inters) in enumerate(details, 1):
383     print(f" 弹#{k}: t_drop={tr:.2f} s, t_burst={td:.2f} s, ={tau:.2f} s | 单窗={L:.2f} s")
384     print(f"      P_burst=({P_burst[0]:.2f}, {P_burst[1]:.2f}, {P_burst[2]:.2f}) |
      区间={inters}")
385
386
387 # 可视化
388 fig, ax = plt.subplots(1, 1, figsize=(8, 5))
389 ax.plot(hist_gen, hist_best, 'r-', linewidth=1.2, label="Best")
390 ax.plot(hist_gen, hist_mean, 'b-', linewidth=1.2, label="Population mean")
391 ax.set_xlabel("Generation")
392 ax.set_ylabel("Union cover length (s)")
393 ax.set_title("GA progress (Problem 3)")
394 ax.grid(True, linestyle='--', alpha=0.6)
395 ax.legend()
396 fig.tight_layout()
397 fig.savefig("../figures/problem3_ga_progress.png", dpi=180, bbox_inches="tight")
398 plt.show()
399
400 # Excel 导出
401 cols = [
402     "无人机运动方向",
403     "无人机运动速度 (m/s)",
404     "烟幕干扰弹编号",
405     "烟幕干扰弹投放点的x坐标 (m)",
406     "烟幕干扰弹投放点的y坐标 (m)",
407     "烟幕干扰弹投放点的z坐标 (m)",
408     "烟幕干扰弹起爆点的x坐标 (m)",
409     "烟幕干扰弹起爆点的y坐标 (m)",
410     "烟幕干扰弹起爆点的z坐标 (m)",
411     "有效干扰时长 (s)",
412 ]
413 rows = []
414 for idx, (tr, td, tau, P_burst, L, _) in enumerate(details, 1):
415     P_drop = drop_point(FY1_0, tr, v, theta)
416     rows.append({
417         "无人机运动方向": float(theta % 360.0),
418         "无人机运动速度 (m/s)": float(v),

```

```

419         "烟幕干扰弹编号": int(idx),
420         "烟幕干扰弹投放点的x坐标 (m)": float(P_drop[0]),
421         "烟幕干扰弹投放点的y坐标 (m)": float(P_drop[1]),
422         "烟幕干扰弹投放点的z坐标 (m)": float(P_drop[2]),
423         "烟幕干扰弹起爆点的x坐标 (m)": float(P_burst[0]),
424         "烟幕干扰弹起爆点的y坐标 (m)": float(P_burst[1]),
425         "烟幕干扰弹起爆点的z坐标 (m)": float(P_burst[2]),
426         "有效干扰时长 (s)": float(L),
427     })
428
429     # 追加空行与注释行
430     blank = {c: np.nan for c in cols}
431     note = {c: np.nan for c in cols}
432     note["无人机运动方向"] = "注: 以x轴为正向, 逆时针方向为正, 取值0~360 (度)。"
433     df_out = pd.DataFrame(rows, columns=cols)
434     df_out = pd.concat([df_out, pd.DataFrame([blank, note])], ignore_index=True)
435
436     with pd.ExcelWriter("../result/result1.xlsx") as w:
437         df_out.to_excel(w, index=False, sheet_name="Sheet1")
438
439     # 3D 可视化
440     plot_problem3_3d(
441         theta_deg=theta,
442         v=v,
443         bombs=[(tr1, tau1), (tr2, tau2), (tr3, tau3)],
444         dt=0.05,
445         save_path="../figures/problem3_3d.png"
446     )
447
448
449 if __name__ == "__main__":
450
451     res = run_problem3(
452         seed=314159, pop_size=90, n_gen=40,
453         elite_frac=0.12, elite_eval_frac=0.20,
454         dt_fit=0.04, dt_ref=0.01, use_quick=True
455     )

```

以下是问题四的代码:

```

1 from __future__ import annotations
2 from typing import Dict, Any, List
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import math, time
6
7 import params as p

```

```

8 from evaluator import cover_intervals_cached_general, munition_pos_general, missile_pos_general
9 from ga import GeneticAlgorithm
10 from funcs import set_axes_equal_3d
11
12
13 T_UP = float(getattr(p, "T_end", 60.0))
14 # [v, theta, t_drop, delay]
15 LB = np.array([p.v_min, -math.pi, 0.0, 0.0], dtype=float)
16 UB = np.array([p.v_max, math.pi, T_UP, p.t_effective], dtype=float)
17
18
19 def _decode(x: np.ndarray):
20     """
21     解码基因为可读参数
22     """
23     v = float(x[0])
24     theta = float(x[1])
25     dir_xy = np.array([math.cos(theta), math.sin(theta), 0.0], float)
26     n = np.linalg.norm(dir_xy); dir_xy = dir_xy/n if n>0 else np.array([1.0,0.0,0.0], float)
27     t_drop = float(x[2])
28     delay = float(min(max(0.0, x[3]), float(p.t_effective)))
29     t_burst = t_drop + delay
30     if t_burst > T_UP:
31         delay = max(0.0, T_UP - t_drop)
32         t_burst = t_drop + delay
33     return v, dir_xy, t_drop, delay, t_burst
34
35
36 def _cover_FY2(dt: float, v: float, dir_xy: np.ndarray,
37               t_drop: float, t_burst: float,
38               *, stage: int, precheck: bool):
39     """
40     兼容 evaluator 是否带 precheck 参数的两种签名。
41     """
42     try:
43         return cover_intervals_cached_general(dt, v, dir_xy, t_drop, t_burst,
44                                               p.FY2_0, p.M1_0, stage=stage, precheck=precheck)
45     except TypeError:
46         return cover_intervals_cached_general(dt, v, dir_xy, t_drop, t_burst,
47                                               p.FY2_0, p.M1_0, stage=stage)
48
49
50 def _eval_total(x: np.ndarray, dt: float, stage: int, precheck: bool) -> float:
51     v, dir_xy, t_drop, delay, t_burst = _decode(x)
52     _, total = _cover_FY2(dt, v, dir_xy, t_drop, t_burst, stage=stage, precheck=precheck)
53     return float(total)
54

```

```

55
56 def _smart_seeds_for_FY2_nominal() -> List[np.ndarray]:
57     """
58     生成一组智能初始种子，覆盖典型的投放参数组合。
59     """
60     NOM_TH_DEG = -45.0
61     NOM_V      = 140.0
62     NOM_TDROD = 11.0
63     NOM_DELAY = 3.0
64
65     th0 = math.radians(NOM_TH_DEG)
66     def _clip_v(v): return float(max(p.v_min, min(p.v_max, v)))
67     def _clip_td(td): return float(max(0.0, min(T_UP, td)))
68     def _make(v, th, td, dl): return np.array([_clip_v(v), th, _clip_td(td), float(dl)], float)
69
70     td_set = [NOM_TDROD + d for d in (-3.0, -1.5, 0.0, +1.2, +3.0)]
71     dl_set = [NOM_DELAY + d for d in (-1.0, -0.6, 0.0, +0.6, +1.0)]
72     v_set = [NOM_V + d for d in (0.0, -6.0, +6.0, -12.0, +12.0)]
73
74     seeds: List[np.ndarray] = []
75     for th in [th0,
76               th0 + math.radians(7), th0 - math.radians(7),
77               th0 + math.radians(12), th0 - math.radians(12),
78               th0 + math.radians(18), th0 - math.radians(18)]:
79         seeds.append(_make(NOM_V, th, NOM_TDROD, NOM_DELAY))
80     for td in td_set:
81         for dl in [NOM_DELAY-0.6, NOM_DELAY, NOM_DELAY+0.6]:
82             seeds.append(_make(NOM_V, th0, td, dl))
83     for v in v_set:
84         seeds.append(_make(v, th0, NOM_TDROD, NOM_DELAY))
85     for th in [th0 + math.radians(24), th0 - math.radians(24),
86               th0 + math.radians(30), th0 - math.radians(30)]:
87         seeds.append(_make(NOM_V, th, NOM_TDROD + 3.0, NOM_DELAY + 0.6))
88     for th in [th0 + math.radians(12), th0 - math.radians(12)]:
89         for v in [NOM_V-12, NOM_V-6, NOM_V+6, NOM_V+12]:
90             seeds.append(_make(v, th, NOM_TDROD + 1.2, NOM_DELAY))
91
92     uniq: List[np.ndarray] = []
93     for s in seeds:
94         if not any(np.allclose(s, t) for t in uniq):
95             uniq.append(s)
96     return uniq[:32]
97
98
99 def plot_fy2_3d(FY0, v_drone, dir_xy, t_drop, delay, save_path, dt=0.05):
100     """
101     绘制三维场景图

```

```

102 """
103
104 # 时间轴
105 t_burst = float(t_drop + delay)
106 t_uav_mis = np.arange(0.0, t_burst + 1e-12, max(1e-3, dt))
107 t_bomb = np.arange(t_drop, t_burst + 1e-12, max(1e-3, dt))
108
109 # UAV: 等速直线 (XY 按航向, Z 恒定)
110 FY_line = FY0[None, :3] + (v_drone * t_uav_mis[:, None]) * dir_xy[None, :]
111 FY_line[:, 2] = FY0[2]
112
113 # Bomb: 抛物线 (投放到起爆)
114 bomb_traj = np.vstack([munition_pos_general(FY0, v_drone, dir_xy, t_drop, t) for t in
115                        t_bomb])
116
117 # Missile1: 直线 (朝原点)
118 mis1 = np.vstack([missile_pos_general(p.M1_0, t) for t in t_uav_mis])
119
120 # 起爆点 & 云团
121 P_burst = munition_pos_general(FY0, v_drone, dir_xy, t_drop, t_burst)
122 R_cloud = float(getattr(p, "R_effective", 10.0))
123
124 # 绘图
125 fig = plt.figure(figsize=(9, 7))
126 ax = fig.add_subplot(111, projection="3d")
127
128 ax.plot(mis1[:,0], mis1[:,1], mis1[:,2], color="green", linewidth=1.8, label="Missile 1")
129 ax.plot(FY_line[:,0], FY_line[:,1], FY_line[:,2], color="blue", linewidth=2.2, label="UAV
130        (FY2)")
131 ax.plot(bomb_traj[:,0], bomb_traj[:,1], bomb_traj[:,2], color="orange", linewidth=2.2,
132        label="Bomb")
133
134 ax.scatter([P_burst[0]], [P_burst[1]], [P_burst[2]], s=60, marker='o', color="orange",
135        label="Burst point")
136
137 # 云团 (橙色半透明)
138 u = np.linspace(0, 2*np.pi, 40); v_ = np.linspace(0, np.pi, 20)
139 xs = P_burst[0] + R_cloud * np.outer(np.cos(u), np.sin(v_))
140 ys = P_burst[1] + R_cloud * np.outer(np.sin(u), np.sin(v_))
141 zs = P_burst[2] + R_cloud * np.outer(np.ones_like(u), np.cos(v_))
142 ax.plot_surface(xs, ys, zs, color="orange", alpha=0.25, edgecolor="none")
143
144 ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)"); ax.set_zlabel("Z (m)")
145 ax.set_title("Problem 4 (FY2) — Scene Overview")
146 ax.legend(loc="best")
147
148 # 自适应视窗 & 等比例

```

```

145 xs_all = np.concatenate([mis1[:,0], FY_line[:,0], bomb_traj[:,0], [P_burst[0]]])
146 ys_all = np.concatenate([mis1[:,1], FY_line[:,1], bomb_traj[:,1], [P_burst[1]]])
147 zs_all = np.concatenate([mis1[:,2], FY_line[:,2], bomb_traj[:,2], [P_burst[2]]])
148 ax.set_xlim(xs_all.min()-50, xs_all.max()+50)
149 ax.set_ylim(ys_all.min()-50, ys_all.max()+50)
150 ax.set_zlim(max(0.0, zs_all.min()-50), zs_all.max()+50)
151 set_axes_equal_3d(ax)
152
153 plt.tight_layout()
154 plt.savefig(save_path, dpi=220, bbox_inches="tight")
155 plt.show()
156 plt.close(fig)
157
158
159 def run_problem4_FY2(cfg: Dict[str, Any]) -> None:
160
161     ga = GeneticAlgorithm(
162         lb=LB, ub=UB,
163         pop_size=cfg["pop"], n_gen=cfg["gen"], seed=cfg["seed"],
164         elite_frac=cfg["elite_frac"], tournament_k=3,
165         sbx_eta=12.0, sbx_prob=0.90,
166         mut_rate=0.20, mut_scale=0.10,
167         objective="max",
168     )
169
170     # 自适应扰动幅度（随迭代进程逐渐减小）
171     def sigma_fn(g: int, G: int, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
172         phase = g / G
173         anneal = max(0.40, 1.0 - phase)
174         ang_scale = (0.12 if phase < 0.5 else 0.08) * math.pi # 早大后小
175         return np.array([
176             (p.v_max - p.v_min)*0.05, # v
177             ang_scale, # theta
178             0.10*T_UP*anneal, # t_drop
179             0.60*anneal, # delay
180         ], float)
181
182     fitness_fn = lambda x: _eval_total(x, cfg["ga_dt"], stage=0,
183         precheck=cfg["precheck_stage0"])
184     reeval_fn = lambda x: _eval_total(x, cfg["final_dt"], stage=1, precheck=False)
185
186     seeds = _smart_seeds_for_FY2_nominal()
187
188     hist_gen, hist_best, hist_mean = [], [], []
189     def stats_hook(g: int, best: float, mean: float):
190         hist_gen.append(int(g)); hist_best.append(float(best)); hist_mean.append(float(mean))
191     def _log(g, best, x):

```

```

191     if g % 4 == 0:
192         print(f"[FY2] Gen {g:02d} best {best:.3f}s")
193
194     t0 = time.time()
195     x_best, _ = ga.evolve(fitness_fn=fitness_fn, reeval_fn=reeval_fn,
196                          reeval_frac=cfg["reeval_frac"],
197                          verbose_every=4, log_fn=_log,
198                          seed_population=seeds, sigma_fn=sigma_fn,
199                          stats_hook=stats_hook)
200
201     t1 = time.time()
202
203     val = _eval_total(x_best, cfg["final_dt"], stage=1, precheck=False)
204     v, dir_xy, t_drop, delay, t_burst = _decode(x_best)
205     theta_deg = math.degrees(math.atan2(dir_xy[1], dir_xy[0]))
206
207     print("\n=== FY2 — Single Bomb (Total-Length Objective) ===")
208     print(f"Total cover length: {val:.3f} s")
209     print(f"Speed v = {v:.3f} m/s, Heading = {theta_deg:.3f}°")
210     print(f"Drop: {t_drop:.3f}s, Delay: {delay:.3f}s, Burst: {t_burst:.3f}s")
211     print(f"Elapsed: %.2f s" % (t1 - t0))
212
213     intervals, total_length = _cover_FY2(
214         dt=cfg["final_dt"],
215         v=v,
216         dir_xy=dir_xy,
217         t_drop=t_drop,
218         t_burst=t_burst,
219         stage=1,
220         precheck=False
221     )
222
223     if intervals:
224         print("\nEffective cover intervals:")
225         for i, (start, end) in enumerate(intervals, 1):
226             print(f"Interval {i}: {start:.3f} s to {end:.3f} s (duration: {end-start:.3f} s)")
227     else:
228         print("\nNo effective cover intervals found.")
229
230     # 可视化
231     fig, ax = plt.subplots(1, 1, figsize=(8, 5))
232     ax.plot(hist_gen, hist_best, 'r-', linewidth=1.2, label="Best")
233     ax.plot(hist_gen, hist_mean, 'b-', linewidth=1.2, label="Population mean")
234     ax.set_xlabel("Generation")
235     ax.set_ylabel("Total cover length (s)")
236     ax.set_title("GA progress (Problem4-FY2)")
237     ax.grid(True, linestyle='--', alpha=0.6)
238     ax.legend()

```

```

237     fig.tight_layout()
238     fig.savefig("../figures/fy2_ga_progress.png", dpi=180, bbox_inches="tight")
239     plt.show()
240
241     # 3D 可视化
242     plot_fy2_3d(
243         FY0=p.FY2_0,
244         v_drone=v,
245         dir_xy=dir_xy,
246         t_drop=t_drop,
247         delay=delay,
248         save_path="../figures/problem4_fy2_3d.png",
249         dt=0.05
250     )
251
252
253     # 遗传算法配置
254     CFG = dict(
255         pop=60, gen=44, seed=202509,
256         elite_frac=0.18, reeval_frac=0.32,
257         ga_dt=0.06, final_dt=0.01, precheck_stage0=True
258     )
259
260
261     if __name__ == "__main__":
262
263         run_problem4_FY2(CFG)

```

```

1  from __future__ import annotations
2  from typing import Dict, Any, List
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import math, time
6
7  import params as p
8  from evaluator import cover_intervals_cached_general, missile_pos_general, munition_pos_general
9  from ga import GeneticAlgorithm
10 from funcs import set_axes_equal_3d
11
12
13 T_UP = float(getattr(p, "T_end", 60.0))
14 # [v, theta, t_drop, delay]
15 LB = np.array([p.v_min, -math.pi, 0.0, 0.0], dtype=float)
16 UB = np.array([p.v_max, math.pi, T_UP, p.t_effective], dtype=float)
17
18

```



```

19 def _decode(x: np.ndarray):
20     """
21     解码基因为可读参数
22     """
23     v = float(x[0])
24     theta = float(x[1])
25     dir_xy = np.array([math.cos(theta), math.sin(theta), 0.0], float)
26     n = np.linalg.norm(dir_xy); dir_xy = dir_xy/n if n>0 else np.array([1.0,0.0,0.0], float)
27     t_drop = float(x[2])
28     delay = float(min(max(0.0, x[3]), float(p.t_effective)))
29     t_burst = t_drop + delay
30     if t_burst > T_UP:
31         delay = max(0.0, T_UP - t_drop)
32         t_burst = t_drop + delay
33     return v, dir_xy, t_drop, delay, t_burst
34
35
36 def _cover_FY3(dt: float, v: float, dir_xy: np.ndarray,
37               t_drop: float, t_burst: float,
38               *, stage: int, precheck: bool):
39     try:
40         return cover_intervals_cached_general(dt, v, dir_xy, t_drop, t_burst,
41                                              p.FY3_0, p.M1_0, stage=stage, precheck=precheck)
42     except TypeError:
43         return cover_intervals_cached_general(dt, v, dir_xy, t_drop, t_burst,
44                                              p.FY3_0, p.M1_0, stage=stage)
45
46
47 def _eval_total(x: np.ndarray, dt: float, stage: int, precheck: bool) -> float:
48     v, dir_xy, t_drop, delay, t_burst = _decode(x)
49     _, total = _cover_FY3(dt, v, dir_xy, t_drop, t_burst, stage=stage, precheck=precheck)
50     return float(total)
51
52
53 def _smart_seeds_for_FY3_nominal() -> List[np.ndarray]:
54     """
55     基于"指定近似最优点的智能种子（最多32个）
56     """
57
58     NOM_TH_DEG = 90.0
59     NOM_V = 110.0
60     NOM_TDROD = 23.8
61     NOM_DELAY = 4.2
62
63     th0 = math.radians(NOM_TH_DEG)
64     def _clip_v(v): return float(max(p.v_min, min(p.v_max, v)))
65     def _clip_td(td): return float(max(0.0, min(T_UP, td)))

```

```

66 def _make(v, th, td, dl): return np.array([_clip_v(v), th, _clip_td(td), float(dl)], float)
67
68 td_set = [NOM_TDROP + d for d in (-3.0, -1.5, 0.0, +1.2, +3.0)]
69 dl_set = [NOM_DELAY + d for d in (-1.0, -0.4, 0.0, +0.4, +1.0)]
70 v_set = [NOM_V, NOM_V-6, NOM_V+6, NOM_V-12, NOM_V+12]
71
72 seeds: List[np.ndarray] = []
73 for th in [th0,
74            th0 + math.radians(7), th0 - math.radians(7),
75            th0 + math.radians(12), th0 - math.radians(12),
76            th0 + math.radians(18), th0 - math.radians(18)]:
77     seeds.append(_make(NOM_V, th, NOM_TDROP, NOM_DELAY))
78 for td in td_set:
79     for dl in [NOM_DELAY-0.4, NOM_DELAY, NOM_DELAY+0.4]:
80         seeds.append(_make(NOM_V, th0, td, dl))
81 for v in v_set:
82     seeds.append(_make(v, th0, NOM_TDROP, NOM_DELAY))
83 for th in [th0 + math.radians(24), th0 - math.radians(24),
84            th0 + math.radians(30), th0 - math.radians(30)]:
85     seeds.append(_make(NOM_V, th, NOM_TDROP + 3.5, NOM_DELAY + 0.8))
86 for th in [th0 + math.radians(12), th0 - math.radians(12)]:
87     for v in [NOM_V-12, NOM_V+12]:
88         seeds.append(_make(v, th, NOM_TDROP + 1.2, NOM_DELAY))
89
90 uniq: List[np.ndarray] = []
91 for s in seeds:
92     if not any(np.allclose(s, t) for t in uniq):
93         uniq.append(s)
94 return uniq[:32]
95
96
97 def plot_fy3_3d(FY0, v_drone, dir_xy, t_drop, delay, save_path, dt=0.05):
98     """
99     绘制三维场景图
100     """
101
102     # 时间轴
103     t_burst = float(t_drop + delay)
104     t_uav_mis = np.arange(0.0, t_burst + 1e-12, max(1e-3, dt))
105     t_bomb = np.arange(t_drop, t_burst + 1e-12, max(1e-3, dt))
106
107     # UAV: 等速直线 (XY 按航向, Z 恒定)
108     FY_line = FY0[None, :3] + (v_drone * t_uav_mis[:, None]) * dir_xy[None, :]
109     FY_line[:, 2] = FY0[2]
110
111     # Bomb: 抛物线 (投放到起爆)
112     bomb_traj = np.vstack([munition_pos_general(FY0, v_drone, dir_xy, t_drop, t) for t in

```

```

        t_bomb])

113
114 # Missile1: 直线 (朝原点)
115 mis1 = np.vstack([missile_pos_general(p.M1_0, t) for t in t_uav_mis])
116
117 # 起爆点 & 云团
118 P_burst = munition_pos_general(FY0, v_drone, dir_xy, t_drop, t_burst)
119 R_cloud = float(getattr(p, "R_effective", 10.0))
120
121 # 绘图
122 fig = plt.figure(figsize=(9, 7))
123 ax = fig.add_subplot(111, projection="3d")
124
125 # 三条轨迹
126 ax.plot(mis1[:,0], mis1[:,1], mis1[:,2], color="green", linewidth=1.8, label="Missile 1")
127 ax.plot(FY_line[:,0], FY_line[:,1], FY_line[:,2], color="blue", linewidth=2.2, label="UAV
    (FY3)")
128 ax.plot(bomb_traj[:,0], bomb_traj[:,1], bomb_traj[:,2], color="orange", linewidth=2.2,
    label="Bomb")
129
130 # 起爆点
131 ax.scatter([P_burst[0]], [P_burst[1]], [P_burst[2]], s=60, marker='o', color="orange",
    label="Burst point")
132
133 # 云团 (橙色半透明)
134 u = np.linspace(0, 2*np.pi, 40); v_ = np.linspace(0, np.pi, 20)
135 xs = P_burst[0] + R_cloud * np.outer(np.cos(u), np.sin(v_))
136 ys = P_burst[1] + R_cloud * np.outer(np.sin(u), np.sin(v_))
137 zs = P_burst[2] + R_cloud * np.outer(np.ones_like(u), np.cos(v_))
138 ax.plot_surface(xs, ys, zs, color="orange", alpha=0.25, edgecolor="none")
139
140 # 坐标轴与范围
141 ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)"); ax.set_zlabel("Z (m)")
142 ax.set_title("Problem 4 (FY3) — Scene Overview")
143 ax.legend(loc="best")
144
145 xs_all = np.concatenate([mis1[:,0], FY_line[:,0], bomb_traj[:,0], [P_burst[0]]])
146 ys_all = np.concatenate([mis1[:,1], FY_line[:,1], bomb_traj[:,1], [P_burst[1]]])
147 zs_all = np.concatenate([mis1[:,2], FY_line[:,2], bomb_traj[:,2], [P_burst[2]]])
148 ax.set_xlim(xs_all.min()-50, xs_all.max()+50)
149 ax.set_ylim(ys_all.min()-50, ys_all.max()+50)
150 ax.set_zlim(max(0.0, zs_all.min()-50), zs_all.max()+50)
151 set_axes_equal_3d(ax)
152
153 plt.tight_layout()
154 plt.savefig(save_path, dpi=220, bbox_inches="tight")
155 plt.show()

```

```

156 plt.close(fig)
157
158
159 def run_problem4_FY3(cfg: Dict[str, Any]):
160
161     ga = GeneticAlgorithm(
162         lb=LB, ub=UB,
163         pop_size=cfg["pop"], n_gen=cfg["gen"], seed=cfg["seed"],
164         elite_frac=cfg["elite_frac"], tournament_k=3,
165         sbx_eta=12.0, sbx_prob=0.90,
166         mut_rate=0.20, mut_scale=0.10,
167         objective="max",
168     )
169
170     # 自适应扰动幅度（随迭代进程逐渐减小）
171     def sigma_fn(g: int, G: int, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
172         phase = g / G
173         anneal = max(0.40, 1.0 - phase)
174         ang_scale = (0.12 if phase < 0.5 else 0.08) * math.pi # 早大后小
175         return np.array([
176             (p.v_max - p.v_min)*0.05, # v
177             ang_scale, # theta
178             0.10*T_UP*anneal, # t_drop
179             0.60*anneal, # delay
180         ], float)
181
182     fitness_fn = lambda x: _eval_total(x, cfg["ga_dt"], stage=0,
183         precheck=cfg["precheck_stage0"])
184
185     reeval_fn = lambda x: _eval_total(x, cfg["final_dt"], stage=1, precheck=False)
186
187     seeds = _smart_seeds_for_FY3_nominal()
188
189     hist_gen, hist_best, hist_mean = [], [], []
190     def stats_hook(g: int, best: float, mean: float):
191         hist_gen.append(int(g)); hist_best.append(float(best)); hist_mean.append(float(mean))
192     def _log(g, best, x):
193         if g % 4 == 0:
194             print(f"[FY3] Gen {g:02d} best {best:.3f}s")
195
196     t0 = time.time()
197     x_best, _ = ga.evolve(fitness_fn=fitness_fn, reeval_fn=reeval_fn,
198         reeval_frac=cfg["reeval_frac"],
199         verbose_every=4, log_fn=_log,
200         seed_population=seeds, sigma_fn=sigma_fn,
201         stats_hook=stats_hook)
202     t1 = time.time()

```

```

201 val = _eval_total(x_best, cfg["final_dt"], stage=1, precheck=False)
202 v, dir_xy, t_drop, delay, t_burst = _decode(x_best)
203 theta_deg = math.degrees(math.atan2(dir_xy[1], dir_xy[0]))
204
205 print("\n=== FY3 — Single Bomb (Total-Length Objective) ===")
206 print(f"Total cover length: {val:.3f} s")
207 print(f"Speed v = {v:.3f} m/s, Heading = {theta_deg:.3f}°")
208 print(f"Drop: {t_drop:.3f}s, Delay: {delay:.3f}s, Burst: {t_burst:.3f}s")
209 print("Elapsed: %.2f s" % (t1 - t0))
210
211 intervals, total_length = _cover_FY3(
212     dt=cfg["final_dt"],
213     v=v,
214     dir_xy=dir_xy,
215     t_drop=t_drop,
216     t_burst=t_burst,
217     stage=1,
218     precheck=False
219 )
220
221 if intervals:
222     print("\nEffective cover intervals:")
223     for i, (start, end) in enumerate(intervals, 1):
224         print(f"Interval {i}: {start:.3f} s to {end:.3f} s (duration: {end-start:.3f} s)")
225 else:
226     print("\nNo effective cover intervals found.")
227
228 # 可视化
229 fig, ax = plt.subplots(1, 1, figsize=(8, 5))
230 ax.plot(hist_gen, hist_best, 'r-', linewidth=1.2, label="Best")
231 ax.plot(hist_gen, hist_mean, 'b-', linewidth=1.2, label="Population mean")
232 ax.set_xlabel("Generation")
233 ax.set_ylabel("Total cover length (s)")
234 ax.set_title("GA progress (Problem4-FY3)")
235 ax.grid(True, linestyle='--', alpha=0.6)
236 ax.legend()
237 fig.tight_layout()
238 fig.savefig("../figures/fy3_ga_progress.png", dpi=180, bbox_inches="tight")
239 plt.show()
240
241 # 3D 可视化
242 plot_fy3_3d(
243     FY0=p.FY3_0,
244     v_drone=v,
245     dir_xy=dir_xy,
246     t_drop=t_drop,
247     delay=delay,

```

```

248     dt=0.05,
249     save_path="../figures/problem4_fy3_3d.png"
250 )
251
252
253 # 遗传算法配置
254 CFG = dict(
255     pop=60, gen=44, seed=202509,
256     elite_frac=0.18, reeval_frac=0.32,
257     ga_dt=0.06, final_dt=0.01, precheck_stage0=True
258 )
259
260
261 if __name__ == "__main__":
262
263     run_problem4_FY3(CFG)

```

```

1  from __future__ import annotations
2  import io, re, importlib, math
3  from contextlib import redirect_stdout
4  from typing import Dict, Any, Tuple, Optional
5  import pandas as pd
6  import numpy as np
7  import params as p
8  from funcs import wrap_deg_0_360
9
10 # 需要汇总的模块与主函数、以及各自 FY 初始点在 params 中的变量名
11 ITEMS = [
12     ("FY1", "problem2", "run_problem2", "FY1_0"),
13     ("FY2", "problem4_fy2", "run_problem4_FY2", "FY2_0"),
14     ("FY3", "problem4_fy3", "run_problem4_FY3", "FY3_0"),
15 ]
16
17
18 def safe_import(modname: str):
19     """
20     安全导入模块；失败时返回异常对象。
21     """
22     try:
23         return importlib.import_module(modname)
24     except Exception as e:
25         return e
26
27
28 def call_entry(mod, func_name: str) -> Tuple[Optional[Dict[str, Any]], str, Optional[str]]:
29     """

```

调用模块的入口函数:

- 优先调用无参; 若报 `TypeError`, 再尝试传入默认 `CFG`;
- 捕获 `stdout` 便于后续解析;

返回 (`ret`, `raw_log`, `error`)

- `ret`: 函数返回 (可能是 `dict`, 也可能是 `None`)
- `raw_log`: 捕获的标准输出文本
- `error`: 若出错, 返回错误信息字符串

```
"""
raw_log = ""
try:
    f = getattr(mod, func_name)
except AttributeError:
    return None, raw_log, f"Function {func_name} not found in {getattr(mod, '__name__', str(mod))}"

try:
    buf = io.StringIO()
    with redirect_stdout(buf):
        # 首先尝试无参数调用
        ret = f()
    raw_log = buf.getvalue()
    return ret, raw_log, None
except TypeError as e:
    # 如果函数需要参数, 尝试获取参数并调用
    try:
        # 检查函数签名, 看看需要什么参数
        import inspect
        sig = inspect.signature(f)
        params = sig.parameters

        # 准备参数
        kwargs = {}
        for name, param in params.items():
            if name == 'cfg':
                # 尝试从模块获取 CFG 配置
                cfg = getattr(mod, "CFG", None)
                if cfg is not None:
                    kwargs[name] = cfg
        # 可以添加其他参数的处理逻辑

        buf = io.StringIO()
        with redirect_stdout(buf):
            ret = f(**kwargs)
        raw_log = buf.getvalue()
        return ret, raw_log, None
    except Exception as inner_e:
        return None, raw_log, f"Execution error with parameters: {repr(inner_e)}"
```

```

76     except Exception as e:
77         return None, raw_log, f"Execution error: {repr(e)}"
78
79
80 def parse_stdout_metrics(raw_log: str) -> Dict[str, float]:
81     """
82     从打印文本中解析关键数值指标，返回字典。
83     Total cover length, Speed v, Heading , Drop/Delay/Burst
84     """
85     m: Dict[str, float] = {}
86     # Total cover length: 1.234 s
87     r = re.search(r"Total cover length:\s*([0-9.+-eE+])\s*s", raw_log)
88     if r:
89         m["total_cover_s"] = float(r.group(1))
90     # Speed v = 80.000 m/s, Heading = 12.345°
91     r = re.search(r"Speed v\s*=\s*([0-9.+-eE+])\s*m/s,\s*Heading\s*.*?=\s*([0-9.+-eE+])°",
92                   raw_log)
93     if r:
94         m["speed_mps"] = float(r.group(1))
95         m["heading_deg"] = float(r.group(2))
96     # Drop time: ... / 或 Drop: ...
97     r = re.search(r"(?:Drop
98                   time|Drop):\s*([0-9.+-eE+])\s*s,\s*Delay:\s*([0-9.+-eE+])\s*s,\s*Burst:\s*([0-9.+-eE+])\s*s",
99                   raw_log)
100     if r:
101         m["t_drop_s"] = float(r.group(1))
102         m["delay_s"] = float(r.group(2))
103         m["t_burst_s"] = float(r.group(3))
104     return m
105
106
107 def xy_at(FY0: np.ndarray, v: float, theta_deg: float, t: float) -> Tuple[float, float]:
108     """
109     给定 FY 初始点、速度、航向角（度）、时刻 t，计算 XY 坐标。
110     """
111     th = math.radians(theta_deg)
112     x = float(FY0[0] + math.cos(th) * v * t)
113     y = float(FY0[1] + math.sin(th) * v * t)
114     return x, y
115
116
117 def drop_point(FY0: np.ndarray, v: float, theta_deg: float, t_drop: float) -> Tuple[float,
118                                             float, float]:
119     x, y = xy_at(FY0, v, theta_deg, t_drop)
120     z = float(FY0[2])
121     return x, y, z
122
123
124 def burst_point(FY0: np.ndarray, v: float, theta_deg: float, t_drop: float, t_burst: float) ->

```



```

    Tuple[float, float, float]:
119 x, y = xy_at(FY0, v, theta_deg, t_burst)
120 dt = max(0.0, float(t_burst - t_drop))
121 z = float(FY0[2] - 0.5 * p.g * dt * dt)
122 return x, y, z
123
124 def collect_one(label: str, modname: str, func_name: str, fy_attr: str) -> Dict[str, Any]:
125     """
126     对单个 UAV 运行模块、采集并计算输出行。
127     """
128     row: Dict[str, Any] = {
129         "无人机编号": label,
130         "无人机运动方向": None,
131         "无人机运动速度 (m/s)": None,
132         "烟幕干扰弹投放点的x坐标 (m)": None,
133         "烟幕干扰弹投放点的y坐标 (m)": None,
134         "烟幕干扰弹投放点的z坐标 (m)": None,
135         "烟幕干扰弹起爆点的x坐标 (m)": None,
136         "烟幕干扰弹起爆点的y坐标 (m)": None,
137         "烟幕干扰弹起爆点的z坐标 (m)": None,
138         "有效干扰时长 (s)": None,
139         "error": None,
140     }
141
142     mod = safe_import(modname)
143     if isinstance(mod, Exception):
144         row["error"] = f"Import error: {repr(mod)}"
145         return row
146
147     ret, log, err = call_entry(mod, func_name)
148     if err:
149         row["error"] = err
150         return row
151
152     # 1) 优先读取返回的 dict
153     speed = heading = t_drop = delay = t_burst = total = None
154     if isinstance(ret, dict):
155         total = ret.get("total_cover_s", total)
156         plan = ret.get("plan", {})
157         if isinstance(plan, dict):
158             speed = plan.get("speed_mps", speed)
159             heading = plan.get("heading_deg", heading)
160             t_drop = plan.get("t_drop_s", t_drop)
161             delay = plan.get("delay_s", delay)
162             t_burst = plan.get("t_burst_s", t_burst)
163
164     # 2) stdout 兜底解析

```

```

165 parsed = parse_stdout_metrics(log or "")
166 total = float(parsed.get("total_cover_s", total)) if parsed.get("total_cover_s") is not
    None else total
167 speed = float(parsed.get("speed_mps", speed)) if parsed.get("speed_mps") is not None else
    speed
168 heading = float(parsed.get("heading_deg", heading)) if parsed.get("heading_deg") is not
    None else heading
169 t_drop = float(parsed.get("t_drop_s", t_drop)) if parsed.get("t_drop_s") is not None else
    t_drop
170 delay = float(parsed.get("delay_s", delay)) if parsed.get("delay_s") is not None else
    delay
171 t_burst = float(parsed.get("t_burst_s", t_burst)) if parsed.get("t_burst_s") is not None
    else t_burst
172
173 # 3) 计算坐标并填充行
174 row["有效干扰时长 (s)"] = float(total) if total is not None else None
175 row["无人机运动速度 (m/s)"] = float(speed) if speed is not None else None
176 if heading is not None:
177     heading = wrap_deg_0_360(float(heading))
178     row["无人机运动方向"] = heading
179
180 if all(v is not None for v in [speed, heading, t_drop, t_burst]):
181     FY0 = getattr(p, fy_attr) # FY 初始点
182     x1, y1, z1 = drop_point(FY0, float(speed), heading, float(t_drop))
183     x2, y2, z2 = burst_point(FY0, float(speed), heading, float(t_drop), float(t_burst))
184     row["烟幕干扰弹投放点的x坐标 (m)"] = x1
185     row["烟幕干扰弹投放点的y坐标 (m)"] = y1
186     row["烟幕干扰弹投放点的z坐标 (m)"] = z1
187     row["烟幕干扰弹起爆点的x坐标 (m)"] = x2
188     row["烟幕干扰弹起爆点的y坐标 (m)"] = y2
189     row["烟幕干扰弹起爆点的z坐标 (m)"] = z2
190
191 return row
192
193
194 if __name__ == "__main__":
195     """
196     执行汇总并输出到 Excel, 返回输出路径。
197     """
198     rows = []
199     for label, modname, func_name, fy_attr in ITEMS:
200         rows.append(collect_one(label, modname, func_name, fy_attr))
201
202     # 构建 DataFrame, 维持列顺序
203     cols = [
204         "无人机编号",
205         "无人机运动方向",

```

```

206     "无人机运动速度 (m/s)",
207     "烟幕干扰弹投放点的x坐标 (m)",
208     "烟幕干扰弹投放点的y坐标 (m)",
209     "烟幕干扰弹投放点的z坐标 (m)",
210     "烟幕干扰弹起爆点的x坐标 (m)",
211     "烟幕干扰弹起爆点的y坐标 (m)",
212     "烟幕干扰弹起爆点的z坐标 (m)",
213     "有效干扰时长 (s)",
214     "error",
215 ]
216 df = pd.DataFrame(rows, columns=cols)
217
218 # 追加：空行 + 注释行
219 blank = {c: np.nan for c in cols}
220 note = {c: np.nan for c in cols}
221 note["无人机运动方向"] = "注：以x轴为正向，逆时针方向为正，取值0~360（度）。"
222 df_out = pd.concat([df, pd.DataFrame([blank, note])], ignore_index=True)
223
224 with pd.ExcelWriter("../result/result2.xlsx") as writer:
225     df_out.to_excel(writer, index=False, sheet_name="Sheet1")

```

以下是问题五的代码：

```

1  from __future__ import annotations
2  from typing import Dict, Any, List, Tuple
3  import numpy as np
4  import math, os, time
5  import pandas as pd
6
7  import params as p
8  from funcs import unit, union_length
9  from ga import GeneticAlgorithm
10 from evaluator import cover_intervals_cached_general, munition_pos_general
11
12
13 # 赛题常量
14 _YAXIS = float(getattr(p, "y_axis", 200.0))
15 _R_CYL = float(getattr(p, "R_cyl", 7.0))
16 _H_CYL = float(getattr(p, "H_cyl", 10.0))
17 _R_SMOKE = float(getattr(p, "R_smoke", 10.0))
18 _SINK = float(getattr(p, "SINK", 3.0))
19 _g = float(getattr(p, "g", 9.8))
20 t_effective = float(getattr(p, "cloud_lifetime", 20.0))
21 V_MIN, V_MAX = float(getattr(p, "v_min", 70.0)), float(getattr(p, "v_max", 140.0))
22 T_END = float(getattr(p, "T_end", 60.0))
23
24 # 运动学

```

```

25 def _arrival_time(M0: np.ndarray) -> float:
26     return float(np.linalg.norm(p.fake - M0) / p.v_missile)
27
28 def _heading_vec(theta_rad: float) -> np.ndarray:
29     return unit(np.array([math.cos(theta_rad), math.sin(theta_rad), 0.0], float))
30
31 def _missile_pos(M0: np.ndarray, t: float) -> np.ndarray:
32     u = unit(p.fake - M0)
33     return M0 + u * (p.v_missile * float(t))
34
35 def _uav_xy(FY0: np.ndarray, v: float, theta: float, t: float) -> np.ndarray:
36     dir_xy = _heading_vec(theta)
37     return FY0[:2] + dir_xy[:2] * (v * t)
38
39 def _burst_xyz(FY0: np.ndarray, v: float, theta: float, t_drop: float, delay: float) ->
    np.ndarray:
40     xy = _uav_xy(FY0, v, theta, t_drop + delay)
41     z = FY0[2] - 0.5*_g*(delay**2) if delay > 0 else FY0[2]
42     return np.array([xy[0], xy[1], z], float)
43
44
45 # 快速充分判据（预筛）
46 TB = np.array([0.0, _YAXIS, _H_CYL/2.0], float)
47 RB = float((_R_CYL**2 + (_H_CYL/2.0)**2) ** 0.5)
48
49 def quick_sufficient_cover(Mt: np.ndarray, Ct: np.ndarray) -> bool:
50     v1, v2 = TB - Mt, Ct - Mt
51     d1, d2 = np.linalg.norm(v1), np.linalg.norm(v2)
52     if d1 <= RB: return False
53     if d2 <= _R_SMOKE: return True
54     alpha = math.asin(min(1.0, RB/d1))
55     beta = math.asin(min(1.0, _R_SMOKE/d2))
56     cosd = float(np.dot(v1, v2)/(d1*d2)); cosd = max(-1.0, min(1.0, cosd))
57     delta = math.acos(cosd)
58     return beta >= alpha + delta
59
60 def quick_has_any(FY0: np.ndarray, v: float, theta: float,
61     t_drop: float, delay: float,
62     M0: np.ndarray, T_cap: float) -> bool:
63     t_burst = t_drop + delay
64     if t_burst <= 0.0 or t_burst > T_cap: return False
65     t_end = min(t_burst + t_effective, _arrival_time(M0), T_cap)
66     if t_end <= t_burst: return False
67
68     Ct0 = _burst_xyz(FY0, v, theta, t_drop, delay)
69     span = float(t_end - t_burst)
70     steps = max(5, min(10, int(round(span / 0.7))))

```

```

71     step = float(max(0.5, min(1.0, span / steps)))
72
73     t = t_burst
74     while t <= t_end + 1e-9:
75         Mt = _missile_pos(M0, t)
76         Ct = Ct0.copy(); Ct[2] = Ct0[2] - _SINK * (t - t_burst)
77         if quick_sufficient_cover(Mt, Ct): return True
78         t += step
79     return False
80
81 # 缓存 + 上界剪枝
82 class FitnessCache:
83     def __init__(self, quant: float = 1e-3):
84         self.q=quant
85         self._map: Dict[Tuple, float]={}
86     def key(self, arr: List[float], dt: float, stage: int) -> Tuple:
87         return tuple(int(round(float(v)/self.q)) for v in (arr+[dt,stage]))
88     def get(self, arr: List[float], dt: float, stage: int):
89         return self._map.get(self.key(arr, dt, stage))
90     def put(self, arr: List[float], dt: float, stage: int, val: float):
91         self._map[self.key(arr, dt, stage)] = float(val)
92
93 class SegmentCache:
94     def __init__(self, quant: float = 2e-3): self.q=quant; self._map: Dict[Tuple,
95         List[Tuple[float,float]]]={}
96     def key(self, FY0, M0, v, th, tr, delay, dt, stage):
97         arr = list(FY0)+list(M0)+[v, th, tr, delay, dt, stage]
98         return tuple(int(round(float(a)/self.q)) for a in arr)
99     def get(self, FY0, M0, v, th, tr, delay, dt, stage):
100         return self._map.get(self.key(FY0, M0, v, th, tr, delay, dt, stage))
101     def put(self, FY0, M0, v, th, tr, delay, dt, stage, segs):
102         self._map[self.key(FY0, M0, v, th, tr, delay, dt, stage)] = segs
103
104 FIT_CACHE = FitnessCache(1e-3)
105 SEG_CACHE = SegmentCache(2e-3)
106
107 def _max_possible_sum(M_list: List[np.ndarray], tdet_list: List[float], T_cap: float) -> float:
108     total = 0.0
109     for tdet in tdet_list:
110         if tdet <= 0.0 or tdet > T_cap: continue
111         spans = [max(0.0, min(t_effective, _arrival_time(M0) - tdet, T_cap - tdet)) for M0 in
112             M_list]
113         if spans: total += 3.0 * (sum(spans) / len(spans))
114     return float(total)
115
116 # 基因编码: tr2=tr1+(1+Δ2), tr3=tr2+(1+Δ3), Δ>0 (严格 >1s)
117 _GAP_EPS = 1e-6

```

```

116
117 def gene_from_seed_radians(theta_rad: float, v: float,
118                             rel: List[float], delay: List[float]) -> np.ndarray:
119     tr1, tr2, tr3 = map(float, rel)
120     delay1, delay2, delay3 = map(float, delay)
121     d1 = max(0.0, tr1)
122     d2e = max(_GAP_EPS, (tr2 - tr1) - 1.0)
123     d3e = max(_GAP_EPS, (tr3 - tr2) - 1.0)
124     return np.array([theta_rad, float(v), d1, delay1, d2e, delay2, d3e, delay3], float)
125
126 def decode_gene(x: np.ndarray, T_cap: float, delay_min: float, delay_max: float):
127     th = float(x[0])
128     v = float(np.clip(x[1], V_MIN, V_MAX))
129     d1 = max(0.0, float(x[2]))
130     d2 = max(_GAP_EPS, float(x[4]))
131     d3 = max(_GAP_EPS, float(x[6]))
132     tr1 = d1
133     tr2 = tr1 + 1.0 + d2
134     tr3 = tr2 + 1.0 + d3
135     delay1 = float(np.clip(x[3], delay_min, delay_max))
136     delay2 = float(np.clip(x[5], delay_min, delay_max))
137     delay3 = float(np.clip(x[7], delay_min, delay_max))
138
139     # 裁剪起爆到窗口内
140     for (tr, delayv_name) in [(tr1, "delay1"), (tr2, "delay2"), (tr3, "delay3")]:
141         tdet = tr + locals()[delayv_name]
142         if tdet > T_cap:
143             locals()[delayv_name] = max(0.0, min(delay_max, T_cap - tr)) if tr < T_cap else 0.0
144
145     t1 = min(tr1 + delay1, T_cap); delay1 = max(delay_min, min(delay_max, t1 - tr1))
146     t2 = min(tr2 + delay2, T_cap); delay2 = max(delay_min, min(delay_max, t2 - tr2))
147     t3 = min(tr3 + delay3, T_cap); delay3 = max(delay_min, min(delay_max, t3 - tr3))
148     return th, v, tr1, delay1, tr2, delay2, tr3, delay3
149
150
151 # 单 UAV × 三导弹
152 def eval_uav_three(
153     FY0: np.ndarray, M_list: List[np.ndarray],
154     theta: float, v: float,
155     tr1: float, delay1: float, tr2: float, delay2: float, tr3: float, delay3: float,
156     *, dt: float, stage: int, precheck: bool, use_quick: bool, T_cap: float,
157     aspiration: float = -1.0, margin: float = 0.08
158 ) -> Tuple[float, Dict[str, Any]]:
159
160     tdet_list = [tr1+delay1, tr2+delay2, tr3+delay3]
161     ub = _max_possible_sum(M_list, tdet_list, T_cap)
162     if aspiration > 0.0 and ub + 1e-9 <= aspiration - margin:

```

```

163         return 0.0, dict(per_missile_len=[0.0,0.0,0.0], per_missile_merged={0:[],1:[],2:[]})
164
165     dir_vec = _heading_vec(theta)
166     by_missile: Dict[int, List[Tuple[float,float]]] = {0:[],1:[],2:[]}
167
168     for t_drop, delay in [(tr1,delay1),(tr2,delay2),(tr3,delay3)]:
169         t_burst = t_drop + delay
170         if t_burst <= 0.0 or t_burst > T_cap: continue
171         z0 = FY0[2] - 0.5*_g*(delay**2) if delay>0 else FY0[2]
172         if z0 < -1e-6: continue
173
174         for mi, MO in enumerate(M_list):
175             if use_quick and not quick_has_any(FY0, v, theta, t_drop, delay, MO, T_cap):
176                 continue
177             segs = SEG_CACHE.get(FY0, MO, v, theta, t_drop, delay, dt, stage)
178             if segs is None:
179                 try:
180                     segs, _ = cover_intervals_cached_general(dt, v, dir_vec, t_drop, t_burst,
181                                                             FY0, MO, stage=stage, precheck=precheck)
182                 except TypeError:
183                     segs, _ = cover_intervals_cached_general(dt, v, dir_vec, t_drop, t_burst,
184                                                             FY0, MO, stage=stage)
185             SEG_CACHE.put(FY0, MO, v, theta, t_drop, delay, dt, stage, segs)
186             if segs: by_missile[mi].extend(segs)
187
188     per_len, per_merged = [], {}
189     for mi in range(3):
190         L, merged = union_length(by_missile[mi])
191         per_len.append(float(L)); per_merged[mi] = merged
192     return float(sum(per_len)), dict(per_missile_len=per_len, per_missile_merged=per_merged)
193
194 # 种子
195 UAV_PARAMS = {
196     "FY1": dict(direction_angle=5.01, speed=140.00,
197                 release_time=[0.00, 1.00, 10.10],
198                 delay_time=[0.00, 0.00, 1.21]),
199     "FY2": dict(direction_angle=5.15, speed=127.02,
200                 release_time=[7.91, 13.14, 25.33],
201                 delay_time=[0.61, 2.68, 7.32]),
202     "FY3": dict(direction_angle=91.40, speed=110.00,
203                 release_time=[23.80, 27.20, 28.40],
204                 delay_time=[4.20, 2.30, 1.80]),
205     "FY4": dict(direction_angle=0.00, speed=70.00,
206                 release_time=[0.92, 1.92, 2.92],
207                 delay_time=[0.50, 1.34, 0.50]),
208     "FY5": dict(direction_angle=2.10, speed=104.78,
209                 release_time=[16.04, 17.59, 21.57],

```

```

208         delay_time=[1.65, 4.69, 1.89]),
209     }
210
211     FY0_MAP = {"FY1": p.FY1_0, "FY2": p.FY2_0, "FY3": p.FY3_0, "FY4": p.FY4_0, "FY5": p.FY5_0}
212
213     def build_seed_from_params(name: str) -> np.ndarray:
214         prm = UAV_PARAMS[name]
215         return gene_from_seed_radians(float(prm["direction_angle"]),
216                                       float(prm["speed"]),
217                                       list(map(float, prm["release_time"])),
218                                       list(map(float, prm["delay_time"])))
219
220     def add_gaussian_jitter(x: np.ndarray, rng: np.random.Generator,
221                           th_sigma_deg=6.0, v_sigma=8.0, t_sigma=1.2, delay_sigma=0.8):
222         j = np.array([
223             math.radians(th_sigma_deg)*rng.standard_normal(),
224             v_sigma*rng.standard_normal(),
225             t_sigma*rng.standard_normal(), delay_sigma*rng.standard_normal(),
226             max(0.15, abs(t_sigma*rng.standard_normal())), delay_sigma*rng.standard_normal(),
227             max(0.20, abs(t_sigma*rng.standard_normal())), delay_sigma*rng.standard_normal(),
228         ], float)
229         xj = x + j
230         xj[4] = max(_GAP_EPS, xj[4]); xj[6] = max(_GAP_EPS, xj[6])
231         return xj
232
233
234     # 单 UAV 子问题
235     def solve_uav_subproblem(
236         name: str, FY0: np.ndarray, M_list: List[np.ndarray], *,
237         seed: int, seed_gene: np.ndarray,
238         theta_center: float, theta_jitter_deg: float,
239         v_center: float | None, v_jitter: float,
240         d1_min: float, delay_min: float, delay_max: float,
241         d_extra_max: float = 16.0,
242         pop_size: int = 44, n_gen: int = 22,
243         elite_frac: float = 0.28, reeval_frac: float = 0.18,
244         dt_fit: float = 0.18, dt_ref: float = 0.010,
245         use_quick: bool = True
246     ) -> Dict[str, Any]:
247
248         T_cap = min(T_END, max(_arrival_time(M0) for M0 in M_list))
249
250         # 盒约束
251         th_lb, th_ub = theta_center - math.radians(theta_jitter_deg), theta_center +
            math.radians(theta_jitter_deg)
252         if v_center is None:
253             v_lb, v_ub = V_MIN, V_MAX

```



```

254 else:
255     v_lb = max(V_MIN, v_center - v_jitter)
256     v_ub = min(V_MAX, v_center + v_jitter)
257
258 LB = np.array([th_lb, v_lb, max(0.0, d1_min), delay_min, _GAP_EPS, delay_min, _GAP_EPS,
259               delay_min], float)
260
261 UB = np.array([th_ub, v_ub, d1_min+25.0, delay_max, d_extra_max, delay_max, d_extra_max,
262               delay_max], float)
263
264 ga = GeneticAlgorithm(lb=LB, ub=UB, pop_size=pop_size, n_gen=n_gen, seed=seed,
265                       elite_frac=elite_frac, tournament_k=4, sbx_eta=12.0, sbx_prob=0.90,
266                       mut_rate=0.22, mut_scale=0.12, objective="max")
267
268 rng = np.random.default_rng(seed ^ 0xC0FFEE)
269 seeds = [np.minimum(UB, np.maximum(LB, seed_gene.copy()))]
270 for _ in range(5):
271     seeds.append(np.minimum(UB, np.maximum(LB, add_gaussian_jitter(seed_gene, rng))))
272
273 for de in [0.3, 1.0, 2.0, 4.0, 6.0, 10.0]:
274     base = seeds[0].copy(); base[4] = de; base[6] = de
275     seeds.append(np.minimum(UB, np.maximum(LB, base)))
276     seeds.append(np.minimum(UB, np.maximum(LB, add_gaussian_jitter(base, rng,
277                               th_sigma_deg=8.0, v_sigma=10.0, t_sigma=1.6, delay_sigma=0.9))))
278
279 ASP = {"best": 0.0}
280 def core(dt: float, stage: int, precheck: bool, use_quick: bool, x: np.ndarray) -> float:
281     th, v, tr1, delay1, tr2, delay2, tr3, delay3 = decode_gene(x, T_cap, delay_min,
282                       delay_max)
283     key = [th, v, tr1, delay1, tr2, delay2, tr3, delay3]
284     cached = FIT_CACHE.get(key, dt, stage)
285     if cached is not None:
286         if cached > ASP["best"]: ASP["best"] = cached
287         return cached
288     val, _ = eval_uav_three(FY0, M_list, th, v, tr1, delay1, tr2, delay2, tr3, delay3,
289                             dt=dt, stage=stage, precheck=precheck, use_quick=use_quick,
290                             T_cap=T_cap,
291                             aspiration=ASP["best"], margin=0.08)
292     FIT_CACHE.put(key, dt, stage, val)
293     if val > ASP["best"]: ASP["best"] = val
294     return val
295
296 fitness_fn = lambda x: core(dt_fit, stage=0, precheck=True, use_quick=use_quick, x=x)
297 reeval_fn = lambda x: core(dt_ref, stage=1, precheck=False, use_quick=False, x=x)
298
299 def sigma_fn(g: int, G: int, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
300     phase = g/max(1,G)
301     anneal = max(0.45, 1.0 - phase)

```

```

296     th_scale = (ub[0]-lb[0]) * 0.26
297     v_scale = 0.0 if (v_center is not None) else (ub[1]-lb[1]) * 0.28
298     return np.array([th_scale*anneal, v_scale*anneal,
299                     1.5*anneal, 0.70*anneal, 2.0*anneal, 0.70*anneal, 2.6*anneal,
300                     0.70*anneal], float)
301
302 def log_fn(g: int, best: float, x: np.ndarray):
303     if g % 2 == 0:
304         print(f" [{name}] Gen {g:02d} | best {best:.3f}s (fit_cache={len(FIT_CACHE._map)},
305             seg_cache={len(SEG_CACHE._map)})")
306
307     x_best, _ = ga.evolve(fitness_fn=fitness_fn, reeval_fn=reeval_fn, reeval_frac=reeval_frac,
308                          verbose_every=2, log_fn=log_fn,
309                          seed_population=seeds, sigma_fn=sigma_fn)
310
311     th, v, tr1, delay1, tr2, delay2, tr3, delay3 = decode_gene(x_best, T_cap, delay_min,
312                                                                delay_max)
313     best_total, info = eval_uav_three(FY0, M_list, th, v, tr1, delay1, tr2, delay2, tr3, delay3,
314                                       dt=dt_ref, stage=1, precheck=False, use_quick=False,
315                                       T_cap=T_cap)
316
317     return dict(
318         UAV=name,
319         theta_deg=float((math.degrees(th) % 360.0)),
320         speed_mps=float(v),
321         bombs=[dict(t_drop=tr1, delay=delay1), dict(t_drop=tr2, delay=delay2), dict(t_drop=tr3,
322                                             delay=delay3)],
323         per_missile_len=info["per_missile_len"],
324         best_total=float(best_total),
325     )
326
327 # 主流程：五架机全部求解 + 写 Excel
328 def run_problem5(out_path: str = "result3.xlsx",
329                 pop_size: int = 44, n_gen: int = 22,
330                 dt_fit: float = 0.18, dt_ref: float = 0.01):
331     M_list = [p.M1_0, p.M2_0, p.M3_0]
332     rng = np.random.default_rng(20250907)
333
334     UAVS = []
335     for idx, name in enumerate(["FY1", "FY2", "FY3", "FY4", "FY5"], 1):
336         prm = UAV_PARAMS[name]
337         seed_gene = build_seed_from_params(name)
338         UAVS.append(dict(
339             name=name, FY0=FY0_MAP[name], seed=(20250907 ^ (idx<<12)),
340             seed_gene=seed_gene,
341             theta_center=float(prm["direction_angle"]), # 弧度中心
342             theta_jitter_deg=14.0 if name!="FY1" else 12.0, # FY1 略窄
343             v_center=float(prm["speed"]), v_jitter=12.0,

```

```

338         d1_min=0.0 if name in ["FY1","FY4"] else 3.0,
339         delay_min=0.0, delay_max=8.0 if name in ["FY4","FY5"] else 6.5,
340     ))
341
342 details_rows, summary_rows, total_sum = [], [], 0.0
343 t0 = time.time()
344
345 for cfg in UAVS:
346     print(f"\n=== 子问题: {cfg['name']} (严格判据·加速) ===")
347     res = solve_uav_subproblem(M_list=M_list, dt_fit=dt_fit, dt_ref=dt_ref,
348                               pop_size=pop_size, n_gen=n_gen, **cfg)
349     total_sum += res["best_total"]
350     summary_rows.append(dict(UAV=cfg["name"], total_cover_sum_s=res["best_total"]))
351
352     theta = res["theta_deg"]; v = res["speed_mps"]
353     L1, L2, L3 = res["per_missile_len"]
354     b1,b2,b3 = res["bombs"]
355     print(f" -> {cfg['name']}: best sum(M1,M2,M3) = {res['best_total']:.3f}s | per-missile =
356           [{L1:.3f}, {L2:.3f}, {L3:.3f}]")
357     print("   params:  =%.3f° v=%.3f m/s" % (theta, v))
358     print("   bombs : (t_drop=%.3f, delay=%.3f) | (t_drop=%.3f, delay=%.3f) | (t_drop=%.3f,
359           delay=%.3f)"
360           % (b1['t_drop'], b1['delay'], b2['t_drop'], b2['delay'], b3['t_drop'],
361              b3['delay']))
362
363     for idx_b, b in enumerate(res["bombs"], 1):
364         details_rows.append({
365             "无人机": cfg["name"],
366             "航向角(度)": theta,
367             "速度(m/s)": v,
368             "弹号": idx_b,
369             "投放时刻t_drop(s)": float(b["t_drop"]),
370             "起爆延迟delay(s)": float(b["delay"]),
371             "对M1并集时长(s)": float(L1),
372             "对M2并集时长(s)": float(L2),
373             "对M3并集时长(s)": float(L3),
374             "三导弹合计(s)": float(L1 + L2 + L3),
375         })
376
377 t1 = time.time()
378 print("\n=== 问题5 汇总 ===")
379 for r in summary_rows:
380     print(f" {r['UAV']}: {r['total_cover_sum_s']:.3f}s")
381 print(f" 合计: {total_sum:.3f}s 耗时: {(t1 - t0):.2f}s")
382
383 df_detail = pd.DataFrame(details_rows)
384 df_summary = pd.DataFrame(summary_rows + [dict(UAV="TOTAL", total_cover_sum_s=total_sum)])

```

```

382 with pd.ExcelWriter(out_path) as writer:
383     df_detail.to_excel(writer, sheet_name="Details", index=False)
384     df_summary.to_excel(writer, sheet_name="Summary", index=False)
385
386     return dict(total=total_sum, detail_path=os.path.abspath(out_path), elapsed=t1-t0)
387
388
389 if __name__ == "__main__":
390
391     run_problem5(out_path="result3.xlsx", pop_size=44, n_gen=22, dt_fit=0.18, dt_ref=0.01)

```

以下是绘图代码:

```

1 import matplotlib.pyplot as plt
2 import params as p
3 from funcs import unit
4
5
6 def plot_init_pos():
7     """
8     绘制导弹和无人机的初始位置和飞行方向
9     """
10    plt.figure(1, figsize=(12, 8))
11
12    # 绘制真假目标位置
13    plt.plot(p.real[0], p.real[1], 'g*', markersize=6)
14    plt.plot(p.fake[0], p.fake[1], 'm*', markersize=6)
15    plt.text(p.real[0]+50, p.real[1]+50, 'Real Target', fontsize=6)
16    plt.text(p.fake[0]+50, p.fake[1]+50, 'Fake Target', fontsize=6)
17
18    # 绘制导弹位置
19    plt.plot(p.M1_0[0], p.M1_0[1], 'ro', markersize=4)
20    plt.plot(p.M2_0[0], p.M2_0[1], 'ro', markersize=4)
21    plt.plot(p.M3_0[0], p.M3_0[1], 'ro', markersize=4)
22    plt.text(p.M1_0[0]+50, p.M1_0[1]+50, 'M1', fontsize=6)
23    plt.text(p.M2_0[0]+50, p.M2_0[1]+50, 'M2', fontsize=6)
24    plt.text(p.M3_0[0]+50, p.M3_0[1]+50, 'M3', fontsize=6)
25
26    # 绘制无人机位置
27    plt.plot(p.FY1_0[0], p.FY1_0[1], 'bo', markersize=4)
28    plt.plot(p.FY2_0[0], p.FY2_0[1], 'bo', markersize=4)
29    plt.plot(p.FY3_0[0], p.FY3_0[1], 'bo', markersize=4)
30    plt.plot(p.FY4_0[0], p.FY4_0[1], 'bo', markersize=4)
31    plt.plot(p.FY5_0[0], p.FY5_0[1], 'bo', markersize=4)
32    plt.text(p.FY1_0[0]+50, p.FY1_0[1]+50, 'FY1', fontsize=6)
33    plt.text(p.FY2_0[0]+50, p.FY2_0[1]+50, 'FY2', fontsize=6)
34    plt.text(p.FY3_0[0]+50, p.FY3_0[1]+50, 'FY3', fontsize=6)

```

```

35 plt.text(p.FY4_0[0]+50, p.FY4_0[1]+50, 'FY4', fontsize=6)
36 plt.text(p.FY5_0[0]+50, p.FY5_0[1]+50, 'FY5', fontsize=6)
37
38 plt.grid(True, linestyle='--', alpha=0.7)
39
40 # 绘制飞行方向箭头
41 arrow_length = 250 # 箭头长度
42 for i, m in enumerate([p.M1_0, p.M2_0, p.M3_0]):
43     direction = unit(p.fake - m) # 飞行方向单位向量
44     dx, dy = direction[0] * arrow_length, direction[1] * arrow_length
45     plt.arrow(m[0], m[1], dx, dy, head_width=6, head_length=10, fc='red', ec='red',
46               linewidth=3)
47
48 plt.gca().set_aspect('equal', adjustable='box')
49 plt.xlabel('X (m)', fontsize=12)
50 plt.ylabel('Y (m)', fontsize=12)
51 plt.title('Initial Positions of Missiles and Drones', fontsize=14)
52 plt.savefig('./figures/init_pos.png', dpi=300, bbox_inches='tight')
53 plt.show()
54
55 if __name__ == "__main__":
56     plot_init_pos()

```

```

1 import matplotlib.pyplot as plt
2 import matplotlib.patches as patches
3
4
5 # 数据
6 bomb = {
7     "name": "Bomb", "release": 1.5, "delay": 3.6, "burst": 5.1, "cover_start": 8.05,
8     "cover_end": 9.44
9 }
10
11 # 创建图表
12 fig, ax = plt.subplots(figsize=(12, 6))
13
14 y = 1 # y轴位置
15
16 # 投放时刻
17 ax.plot(bomb["release"], y, 'o', markersize=10, color='blue', label='drop time')
18 ax.text(bomb["release"], y + 0.1, f't_drop={bomb["release"]:.3f}s',
19         ha='center', va='bottom', fontsize=9)
20
21 # 起爆时刻

```

```

22 ax.plot(bomb["burst"], y, 's', markersize=10, color='red', label='burst time')
23 ax.text(bomb["burst"], y - 0.1, f't_burst={bomb["burst"]:.3f}s',
24         ha='center', va='bottom', fontsize=9)
25
26 # 投放与起爆之间的连线
27 ax.plot([bomb["release"], bomb["burst"]], [y, y], 'k--', linewidth=1.5)
28
29 # 遮蔽区间（如果有）
30 if bomb["cover_start"] is not None and bomb["cover_end"] is not None:
31     width = bomb["cover_end"] - bomb["cover_start"]
32     rect = patches.Rectangle((bomb["cover_start"], y-0.2), width, 0.4,
33                             linewidth=1, edgecolor='green', facecolor='green', alpha=0.3)
34     ax.add_patch(rect)
35     ax.text(bomb["cover_start"] + width/2, y-0.25,
36             f'cover interval: [{bomb["cover_start"]:.2f}, {bomb["cover_end"]:.2f}]s',
37             ha='center', va='top', fontsize=9, color='green')
38
39 # 弹名称
40 ax.text(-1.5, y, bomb["name"], ha='right', va='center', fontsize=12, fontweight='bold')
41
42 # 设置图表属性
43 ax.set_yticks([])
44 ax.set_xlabel('Time (s)')
45 ax.set_title('Bomb Release and Burst Timeline')
46 ax.grid(True, linestyle='--', alpha=0.7)
47 ax.legend(loc='upper right')
48
49 # 设置x轴范围
50 ax.set_xlim(-2, max(12, bomb["burst"]) + 2)
51
52 plt.tight_layout()
53 plt.savefig('./figures/bomb_timeline_1.png', dpi=300, bbox_inches='tight')
54 plt.show()

```

```

1 import matplotlib.pyplot as plt
2 import matplotlib.patches as patches
3
4
5 # 数据
6 bomb = {
7     "name": "Bomb", "release": 0.461, "delay": 0.793, "burst": 1.254, "cover_start": 1.54,
8     "cover_end": 6.11
9 }
10
11 # 创建图表

```

```

12 fig, ax = plt.subplots(figsize=(12, 6))
13
14 y = 1 # y轴位置
15
16 # 投放时刻
17 ax.plot(bomb["release"], y, 'o', markersize=10, color='blue', label='drop time')
18 ax.text(bomb["release"], y + 0.1, f't_drop={bomb["release"]:.3f}s',
19         ha='center', va='bottom', fontsize=9)
20
21 # 起爆时刻
22 ax.plot(bomb["burst"], y, 's', markersize=10, color='red', label='burst time')
23 ax.text(bomb["burst"], y - 0.1, f't_burst={bomb["burst"]:.3f}s',
24         ha='center', va='bottom', fontsize=9)
25
26 # 投放与起爆之间的连线
27 ax.plot([bomb["release"], bomb["burst"]], [y, y], 'k--', linewidth=1.5)
28
29 # 遮蔽区间 (如果有)
30 if bomb["cover_start"] is not None and bomb["cover_end"] is not None:
31     width = bomb["cover_end"] - bomb["cover_start"]
32     rect = patches.Rectangle((bomb["cover_start"], y-0.2), width, 0.4,
33                             linewidth=1, edgecolor='green', facecolor='green', alpha=0.3)
34     ax.add_patch(rect)
35     ax.text(bomb["cover_start"] + width/2, y-0.25,
36             f'cover interval: [{bomb["cover_start"]:.2f}, {bomb["cover_end"]:.2f}]s',
37             ha='center', va='top', fontsize=9, color='green')
38
39 # 弹名称
40 ax.text(-1.5, y, bomb["name"], ha='right', va='center', fontsize=12, fontweight='bold')
41
42 # 设置图表属性
43 ax.set_yticks([])
44 ax.set_xlabel('Time (s)')
45 ax.set_title('Bomb Release and Burst Timeline')
46 ax.grid(True, linestyle='--', alpha=0.7)
47 ax.legend(loc='upper right')
48
49 # 设置x轴范围
50 ax.set_xlim(-2, max(12, bomb["burst"]) + 2)
51
52 plt.tight_layout()
53 plt.savefig('./figures/bomb_timeline_2.png', dpi=300, bbox_inches='tight')
54 plt.show()

```

```

1 import matplotlib.pyplot as plt
2 import matplotlib.patches as patches

```

```

3
4
5 # 数据
6 bomb_data = [
7     {"name": "Bomb 1", "release": 0, "delay": 0, "burst": 0., "cover_start": 4.85, "cover_end":
8         7.4},
9     {"name": "Bomb 2", "release": 1, "delay": 0, "burst": 1, "cover_start": 1.0, "cover_end":
10         5.37},
11     {"name": "Bomb 3", "release": 10.1, "delay": 1.21, "burst": 11.31, "cover_start": None,
12         "cover_end": None}
13 ]
14
15 # 创建图表
16 fig, ax = plt.subplots(figsize=(12, 6))
17
18 # 设置y轴位置
19 y_positions = [3, 2, 1] # 从顶部开始
20
21 # 绘制每个弹的时间线
22 for i, bomb in enumerate(bomb_data):
23     y = y_positions[i]
24
25     # 投放时刻
26     ax.plot(bomb["release"], y, 'o', markersize=10, color='blue', label='drop time' if i == 0
27             else "")
28     ax.text(bomb["release"], y + 0.1, f't_drop={bomb["release"]:.3f}s',
29             ha='center', va='bottom', fontsize=9)
30
31     # 起爆时刻
32     ax.plot(bomb["burst"], y, 's', markersize=10, color='red', label='burst time' if i == 0
33             else "")
34     ax.text(bomb["burst"], y - 0.1, f't_burst={bomb["burst"]:.3f}s',
35             ha='center', va='bottom', fontsize=9)
36
37     # 投放与起爆之间的连线
38     ax.plot([bomb["release"], bomb["burst"]], [y, y], 'k--', linewidth=1.5)
39
40     # 遮蔽区间（如果有）
41     if bomb["cover_start"] is not None and bomb["cover_end"] is not None:
42         width = bomb["cover_end"] - bomb["cover_start"]
43         rect = patches.Rectangle((bomb["cover_start"], y-0.2), width, 0.4,
44                                 linewidth=1, edgecolor='green', facecolor='green', alpha=0.3)
45         ax.add_patch(rect)
46         ax.text(bomb["cover_start"] + width/2, y-0.25,
47                 f'cover interval: [{bomb["cover_start"]:.2f}, {bomb["cover_end"]:.2f}]s',
48                 ha='center', va='top', fontsize=9, color='green')
49

```



```

45     # 弹名称
46     ax.text(-1.5, y, bomb["name"], ha='right', va='center', fontsize=12, fontweight='bold')
47
48     # 设置图表属性
49     ax.set_yticks([])
50     ax.set_xlabel('Time (s)')
51     ax.set_title('Bomb Release and Burst Timeline')
52     ax.grid(True, linestyle='--', alpha=0.7)
53     ax.legend(loc='upper right')
54
55     # 设置x轴范围
56     ax.set_xlim(-2, max(12, max(bomb["burst"] for bomb in bomb_data) + 2))
57
58     plt.tight_layout()
59     plt.savefig('./figures/bomb_timeline_3.png', dpi=300, bbox_inches='tight')
60     plt.show()

```

```

1  import matplotlib.pyplot as plt
2  import matplotlib.patches as patches
3
4
5  # 数据
6  uav_data = [
7      {"name": "FY 1", "release": 0.461, "delay": 0.793, "burst": 1.254, "cover_start": 1.540,
8        "cover_end": 6.110},
9      {"name": "FY 2", "release": 11.067, "delay": 2.980, "burst": 14.047, "cover_start": 18.000,
10       "cover_end": 21.860},
11      {"name": "FY 3", "release": 23.833, "delay": 4.268, "burst": 28.101, "cover_start": 28.520,
12       "cover_end": 31.470}
13  ]
14
15  # 创建图表
16  fig, ax = plt.subplots(figsize=(12, 6))
17
18  # 设置y轴位置
19  y_positions = [3, 2, 1] # 从顶部开始
20
21  # 绘制每个无人机的时间线
22  for i, uav in enumerate(uav_data):
23      y = y_positions[i]
24
25      # 投放时刻
26      ax.plot(uav["release"], y, 'o', markersize=10, color='blue', label='drop time' if i == 0
27              else "")
28      ax.text(uav["release"], y + 0.1, f't_drop={uav["release"]:.3f}s',
29              ha='center', va='bottom', fontsize=9)

```

```

26
27 # 起爆时刻
28 ax.plot(uav["burst"], y, 's', markersize=10, color='red', label='burst time' if i == 0 else
    "")
29 ax.text(uav["burst"], y - 0.1, f't_burst={uav["burst"]:.3f}s',
30         ha='center', va='bottom', fontsize=9)
31
32 # 投放与起爆之间的连线
33 ax.plot([uav["release"], uav["burst"]], [y, y], 'k--', linewidth=1.5)
34
35 # 遮蔽区间（如果有）
36 if uav["cover_start"] is not None and uav["cover_end"] is not None:
37     width = uav["cover_end"] - uav["cover_start"]
38     rect = patches.Rectangle((uav["cover_start"], y-0.2), width, 0.4,
39                             linewidth=1, edgecolor='green', facecolor='green', alpha=0.3)
40     ax.add_patch(rect)
41     ax.text(uav["cover_start"] + width/2, y-0.25,
42             f'cover interval: [{uav["cover_start"]:.2f}, {uav["cover_end"]:.2f}]s',
43             ha='center', va='top', fontsize=9, color='green')
44
45 # 弹名称
46 ax.text(-1.5, y, uav["name"], ha='right', va='center', fontsize=12, fontweight='bold')
47
48 # 设置图表属性
49 ax.set_yticks([])
50 ax.set_xlabel('Time (s)')
51 ax.set_title('uav Release and Burst Timeline')
52 ax.grid(True, linestyle='--', alpha=0.7)
53 ax.legend(loc='upper right')
54
55 # 设置x轴范围
56 ax.set_xlim(-2, max(12, max(uav["burst"] for uav in uav_data) + 2))
57
58 plt.tight_layout()
59 plt.savefig('./figures/uav_timeline.png', dpi=300, bbox_inches='tight')
60 plt.show()

```